

CROSS-DATASET HUMAN ACTIVITY RECOGNITION THROUGH STORAGE-OPTIMIZED GRAPH NEURAL NETWORKS

by

Nisha Dangol

Thesis submitted in partial fulfillment of the requirements
for the Degree of Master of Science (Computer Science)

Acadia University

Spring Convocation 2027

This thesis by Nisha Dangol was defended successfully on [DATE OF DEFENCE].

The committee for the thesis defence was:

Dr. [Name], Chair

Dr. Jiju Poovvancheri, External Examiner

Dr. [Name], Internal Examiner

Dr. Sazia Mahfuz, Supervisor

Dr. Darcy Benoit, Director

The author retains copyright in this thesis. Any substantial copying or any other actions that exceed fair dealing or other exceptions in the Copyright Act require the permission of the author.

Contents

Abstract	xix
List of Abbreviations and Symbols	xxi
Acknowledgements	xxiii
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	2
1.3 Research Scope	4
1.4 Research Questions and Objectives	5
1.5 Summary of Contributions	6
1.6 Organization of the Thesis	7
2 Background & Literature Review	9
2.1 Human Activity Recognition	9
2.2 Traditional Machine Learning Approaches for HAR	11
2.3 Deep Learning Approaches for HAR	12
2.3.1 CNN-based models	13
2.3.2 Recurrent and hybrid models	13
2.3.3 Transformer-based models	15
2.4 Graph Neural Networks for HAR	15

2.4.1	GNN fundamentals	16
2.4.2	Graph representations of sensor data	18
2.5	Cross-Dataset Generalization and Domain Shift in HAR	19
2.5.1	Domain Adaptation versus Domain Generalization	20
2.5.2	Leave-One-Dataset-Out as an Evaluation Protocol	21
2.5.3	Domain-Adversarial Training	22
2.5.4	Other Domain Adaptation and Domain Generalization Techniques	22
2.6	The DAGHAR Benchmark	24
2.7	Storage and Model Optimization for Deep Learning and Graph Neural Networks .	27
2.8	Research Gap	29
2.9	Summary	31
3	Methodology	33
3.1	Dataset and Data Preparation	33
3.1.1	DAGHAR Benchmark	33
3.1.2	Exploratory Data Analysis (EDA)	35
3.1.3	Data Preprocessing and Feature Preparation	37
3.1.4	Cross-Dataset Evaluation Setting	38
3.2	Proposed Generalization Framework	38
3.2.1	Sensor-as-Node Graph Representation	39
3.2.2	Time-Domain and Frequency-Domain Features	40
3.2.3	CNN-Based Feature Encoding	41
3.2.4	GraphSAGE Backbone	42
3.2.5	Domain-Adversarial Training	42
3.2.6	Adaptive Pearson-Blend Graph Construction	43
3.2.7	ConvNet-Based Complementary Branch and Late Fusion	45
3.3	Storage Optimization Framework	48
3.3.1	Architecture Capacity Reduction	49

3.3.2	Post-Hoc Model Compression	50
3.3.3	Graph-Level Optimization	50
3.3.4	Input-Level Optimization	51
3.3.5	Storage–Generalization Trade-off	51
3.4	Experimental Protocol	52
3.4.1	Leave-One-Dataset-Out Evaluation	52
3.4.2	Model Selection and Validation	53
3.4.3	Multi-Seed Validation	54
3.5	Evaluation Metrics	54
3.5.1	Classification Performance	55
3.5.1.1	Accuracy	55
3.5.1.2	Macro-F1	55
3.5.1.3	Precision and Recall	55
3.5.1.4	Confusion Matrices	55
3.5.2	Storage and Efficiency	55
3.5.2.1	Parameter Count and Deploy Size	56
3.5.2.2	Generalization-per-Byte	56
3.5.2.3	Compression Factor	56
3.5.2.4	Inference Latency	56
3.5.3	Statistical Analysis	57
3.6	Experimental Roadmap	57
3.7	Summary	59
4	Empirical Studies	61
4.1	Overview of the Experimental Arc	61
4.2	Arc I: From Storage Optimization to Domain Generalization	64
4.2.1	Single-Dataset Storage-Optimization Baseline	64
4.2.2	Adoption of the DAGHAR Benchmark	65

4.2.3	LODO Reframing as a Domain Generalization Problem	66
4.3	Arc II: Feature and Graph-Representation Exploration	67
4.3.1	Frequency Features on the Timestep Graph	68
4.3.2	Hybrid Features on the Timestep Graph	69
4.3.3	Sensor-as-Node Graph Rewrite	70
4.3.4	Domain-Adversarial Training and Backbone Convergence	72
4.3.5	Configuration Sweep	73
4.3.6	Recipe Finalization	73
4.4	Arc III: Domain Adaptation and Robustness Attempts	75
4.4.1	Feature, Normalization, and Alignment Variants	75
4.4.2	Convolutional Late Fusion	77
4.4.3	Test-Time and Training-Time Adaptation	78
4.4.4	Transductive Domain Adaptation Family	79
4.5	Arc IV: Adaptive Graph Construction — The Best Model	81
4.5.1	Design and Screening	81
4.5.2	Seed Confirmation and the Keep/Revert Decision	82
4.5.3	Why the Blend Wins and Pure Correlation Does Not	84
4.5.4	The Thesis’s Best Model	86
4.6	Arc V: Final Architecture Search	86
4.6.1	Weight-Aware Message Passing	87
4.6.2	Pseudo-Label Self-Training	89
4.7	Arc VI: Storage Optimization Revisited	91
4.7.1	Architecture Capacity and Post-Hoc Compression	91
4.7.2	Edge-Level and Input-Level Storage Optimization	93
4.7.3	Combined-Axis Results and the Pareto Frontier	94
4.7.4	Gradient-Conflict Diagnostic	95
4.8	Cross-Phase Comparative Discussion	96

4.9	Summary	101
5	Conclusion	103
5.1	Summary of the Work	103
5.2	Research Contributions	105
5.3	Limitations	107
5.4	Future Work	109
5.5	Concluding Remarks	110

List of Tables

2.1	Cross-dataset evaluation collapses within-dataset accuracy on DAGHAR. Six-dataset mean accuracy of representative classifiers from the benchmark’s own suite [1], on the standardized view, under within-dataset evaluation versus the LODO protocol, for time-domain and frequency-domain input. ConvNet denotes the convolutional-recurrent DeepConvLSTM architecture of Ordóñez and Roggen [2].	26
3.1	The six DAGHAR domains used throughout this thesis. Sample counts are for the standardized view; “classes present” lists the activities available in each domain. . .	34
3.2	Regularization and training configuration for the graph branch. These settings are frozen across the model-family and adaptive-graph ablations of Chapter 4.	48
3.3	The storage-optimization operators evaluated on the locked model of Section 3.2, grouped by the axis of the model each one acts on.	52
3.4	Roadmap of the experimental programme retained in this thesis, grouped by the chronological arcs of Chapter 4.	57
4.1	Detailed roadmap of the experimental arc. Each row is one stage of the work described in this chapter; accuracy is the leave-one-dataset-out mean across the six DAGHAR datasets once the benchmark was adopted (Section 4.2), and the within-dataset mean beforehand.	63

4.2	Leave-one-dataset-out accuracy after the benchmark’s first exposure to all six DAGHAR datasets, backbone by storage-optimization operator. Mean and standard deviation are computed across the six leave-one-dataset-out folds.	67
4.3	Leave-one-dataset-out accuracy after switching to frequency-domain node features, still on the timestep-as-node graph. Gains are measured against the time-domain results of Table 4.2.	68
4.4	Leave-one-dataset-out accuracy with concatenated time- and frequency-domain node features (114-dimensional), still on the timestep-as-node graph. Every backbone loses ground relative to the frequency-only features of Table 4.3.	69
4.5	Leave-one-dataset-out accuracy of the sensor-as-node graph, four backbones plus the no-message-passing control, before domain-adversarial training is introduced. .	71
4.6	Leave-one-dataset-out accuracy with domain-adversarial training added to each sensor-graph configuration, and the change relative to the non-adversarial results of Table 4.5.	72
4.7	Leave-one-dataset-out accuracy of the finalized recipe: sensor graph, GraphSAGE, domain-adversarial training, and frequency or hybrid input features with source-only per-channel normalization.	74
4.8	Single-model modifications to Arm A, all measured against the leave-one-dataset-out mean established in Section 4.3.6. Every row reverts.	76
4.9	Convolutional late fusion, compared against each model alone. Two independent training runs are reported; both confirm the same qualitative result.	77
4.10	Transductive domain-adaptation methods, evaluated relative to the target-aware lower bound of the same protocol. Both revert.	80
4.11	Screening sweep over candidate adaptive-graph configurations, single seed, selected on source-validation accuracy only.	82

4.12	Five-seed leave-one-dataset-out accuracy: the fixed-topology control, the winning adaptive blend, and the pure-correlation ablation. Only the blend clears the keep/revert rule.	83
4.13	Per-target leave-one-dataset-out accuracy, fixed topology versus the adopted adaptive blend (fused system, five-seed means). Every target improves.	84
4.14	Leave-one-dataset-out accuracy at each of the project’s three domain-generalization milestones. The adaptive graph is adopted as the thesis’s final system.	86
4.15	Residual Chebyshev backbone versus the adopted GraphSAGE backbone, three-seed means, both using the identical adaptive-graph topology.	87
4.16	Pseudo-label self-training versus the source-only adaptive-graph baseline, three-seed means.	89
4.17	Architecture capacity variants and input-level node coarsening, five-seed means, fused system.	92
4.18	Edge-selection policies at a matched nine-edge budget, applied at inference to the same trained checkpoint, five-seed means.	93
4.19	Every stage covered in this chapter, in order, with its final verdict.	97

List of Figures

- 3.1 Activities separate into energy bands once frequency content is considered. A t-SNE embedding of standardized windows coloured by activity class shows the low-, mid-, and high-energy groupings that motivate frequency-aware node features. 35
- 3.2 Domain shift is driven primarily by device placement. A projection of the windows coloured by source dataset shows overlap for static activities but placement-specific separation for dynamic ones. 36
- 3.3 The proposed system, pursuing both research goals on one locked model. A shared feature-extraction step produces two views of each window: a hybrid time-plus-frequency view consumed by the graph branch, whose edges are chosen by a prior-anchored adaptive rule, and a frequency view consumed by the convolutional branch. The two branches' class probabilities are combined by late fusion, and the resulting system is then the subject of the storage-optimization study. The dotted arrow denotes the training-only adversarial gradient reversed through the GRL; the domain head plays no part in inference. 40
- 3.4 From timestep graphs to a sensor graph. Each of the six accelerometer and gyroscope channels becomes a node, and edges express inter-sensor relationships rather than temporal ones. 41

3.5	The complete graph branch, from raw window to both output heads. The convolutional node encoder (top path) and the adaptive Pearson-blend edge construction (bottom path) feed the two-layer GraphSAGE backbone; the pooled graph embedding is read by an activity classifier and, through the gradient-reversal layer, a domain discriminator trained adversarially against it.	46
3.6	The convolutional complementary branch, a DeepConvLSTM reimplementation. Four convolutional blocks read the window’s frequency view, a recurrent block of stacked LSTM layers aggregates the resulting sequence, and a linear classifier on the final aggregated state produces this branch’s class probabilities.	47
3.7	The storage-optimization framework. Five operator families act on the locked model of Section 3.2, divided by when they act: retrain-based operators (capacity reduction, node coarsening, adaptive windowing) modify the model or its input and are retrained from scratch under the identical protocol, while inference-based operators (post-hoc compression, edge-policy substitution) act on the already-trained model without retraining. All resulting single- and combined-axis operating points feed one storage–generalization trade-off analysis.	49
4.1	Leave-one-dataset-out accuracy across the experimental arc. The early collapse on first exposure to the six-dataset benchmark, the long recovery through representation and robustness work, and the final gain from the adaptive graph are the three inflection points that organize the rest of this chapter.	62

Abstract

Sensor-based human activity recognition (HAR) using smartphone accelerometers and gyroscopes performs well within a single dataset but degrades sharply on unseen datasets with different devices, placements, populations, or protocols. Any model robust enough to close that gap must still be small enough to deploy on resource-constrained devices. This thesis addresses both problems jointly on the DAGHAR benchmark’s six datasets, under a strict leave-one-dataset-out (LODO) protocol in which the held-out dataset is never used for training or selection.

Initial graph neural network baselines confirmed the severity of the problem, reaching only around 60% mean cross-dataset accuracy against over 90% within-dataset accuracy. To close this gap, we develop a sensor-as-node graph representation with a GraphSAGE backbone, domain-adversarial training, and late fusion with an independently trained DeepConvLSTM-based convolutional-recurrent branch. Its principal contribution is an adaptive Pearson-blend graph construction that reconstructs the sensor graph’s edges per input window by combining observed inter-channel correlation with a fixed physical prior over sensor placement. This raised the fused system’s cross-dataset accuracy to 77.96%, matching the strongest baseline reported on the DAGHAR benchmark itself.

To assess deployment cost, we define and apply generalization-per-byte, a joint accuracy-and-storage metric, across five compression axes. The resulting analysis shows that storage cost is highly uneven — some reductions cost almost nothing, others cost a great deal. The clearest case is edge selection: at the same edge budget, choosing the right connections is worth over eight accuracy points on its own. Graph structure, not graph size, drives generalization.

List of Abbreviations and Symbols

AW Adaptive Windowing

CNN Convolutional Neural Network

DA Domain Adaptation

DANN Domain-Adversarial Neural Network

DG Domain Generalization

FFT Fast Fourier Transform

GAT Graph Attention Network

GCN Graph Convolutional Network

GIN Graph Isomorphism Network

GNN Graph Neural Network

GRL Gradient Reversal Layer

GS Graph Sparsification

HAR Human Activity Recognition

LODO Leave-One-Dataset-Out

LSTM Long Short-Term Memory

MMD Maximum Mean Discrepancy

NC Node Coarsening

RED Random Edge Deletion

SO Storage Optimization

SOTA State of the Art

UDA Unsupervised Domain Adaptation

Acknowledgements

[to be drafted]

Chapter 1

Introduction

1.1 Motivation

Human activity recognition (HAR)—the task of automatically identifying what a person is physically doing from sensor measurements—has become a foundational component of mobile and wearable computing. Knowing whether a person is sitting, standing, walking, climbing stairs, or running is a small piece of information on its own, but it covers a wide range of applications: health and rehabilitation monitoring, fall detection and elder care, fitness tracking, and context-aware systems that adapt their behaviour to what the user is doing [3, 4]. What makes HAR broadly deployable at all is the smartphone: modern phones carry the two inertial sensors this task needs most—a tri-axial accelerometer and a tri-axial gyroscope—and they are already in billions of pockets, requiring no additional hardware, no instrumentation of the environment, and no change in user behaviour.

On paper, the problem can appear close to solved. Deep learning models trained and evaluated on a single HAR dataset routinely reach accuracies in the high eighties and nineties [4], and this thesis’s own within-dataset experiments reproduce that picture. But these numbers are earned under quietly favourable conditions: the training and test data come from the same devices, worn in the same body position, by overlapping or demographically similar groups of people, recorded

under one consistent collection protocol. Real deployment offers no such guarantee. A recognition model shipped in an application will run on phones the developers never tested, carried in a pocket by one user and in a waist holster by another, by populations that move differently from the study participants, sampled and pre-processed differently from the data the model was trained on. Each of these differences shifts the distribution of the sensor signal, and together they define the gap between a dataset and the world.

The consequence, well documented in recent benchmark work, is that HAR models often generalize badly: a model that excels on the dataset it was trained on can lose tens of percentage points of accuracy when evaluated on a different dataset of the very same activities [1]. This cross-dataset fragility—not within-dataset accuracy—is the real bottleneck standing between laboratory HAR and dependable deployed HAR, and it is the first motivation of this thesis.

The second motivation follows from where HAR models must actually run. Phones and wearables are storage-, memory-, and energy-constrained, and an activity-recognition model is rarely the application a device exists to serve—it is a background component that must justify every byte it occupies and every millisecond it spends. A model too large to ship, too slow to run continuously, or too costly to update in the field will not be deployed, however accurate it is. Robustness and efficiency are therefore not two independent wishes, but a joint requirement. The two also interact in a specific way: the techniques used to shrink a model can erode the very generalization ability that made it worth deploying in the first place. Understanding that interaction—not merely maximizing either property alone—is what deployment-ready HAR actually demands, and it is the second motivation of this thesis.

1.2 Problem Statement

This thesis is built around two problems that follow directly from the deployment mismatch described above. Stated plainly: cross-dataset generalization in sensor-based HAR remains unsolved at deployable accuracy, and the storage cost of whatever generalization can be achieved is under-

studied—the two halves of the deployment question are almost always investigated separately, on different models, under different protocols.

Problem 1: cross-dataset generalization is not guaranteed. A model that performs well when trained and tested within a single dataset may perform substantially worse on a dataset it has never seen, even though the underlying activities—sitting, standing, walking, climbing stairs—are the same. The differences that separate one HAR dataset from another—device, placement, population, protocol—do not disappear simply because a model fits the data it was trained on. The model must generalize *across* that shift, not merely fit within it. In the formal setting studied throughout this thesis, the unseen dataset is exactly that—unseen: no target data, not even unlabeled, is available at training time, which distinguishes this *domain generalization* problem from the more forgiving domain-adaptation setting.

Problem 2: generalizing is not useful if it is too expensive. Even a model that does generalize well across datasets may be an impractical choice for the phones and wearables HAR is meant to run on, if it is too large to store, too slow to run, or too costly to update in the field. A project that only chases accuracy risks producing a model that nobody could actually deploy. The open question is not simply whether such a model can be made smaller—compression techniques are plentiful—but *what its generalization ability costs to keep*: which reductions are free, which are fatal, and what governs the difference.

Together, these two problems give the thesis two connected goals. The first is to improve how well a sensor-based HAR model generalizes to a completely unseen dataset. The second is to quantify and reduce what that model costs to store and run, without giving up the generalization gained. Both goals are pursued on the *same* underlying model, so that the second is a direct measure of the cost of the first, rather than a separate exercise on a different system. The research questions and concrete objectives that motivate these goals are stated in Section 1.4.

1.3 Research Scope

The two problems above could each absorb a research programme many times the size of a thesis, so the boundaries of this one are drawn explicitly here.

All experiments are conducted on the DAGHAR benchmark [1], a curated collection of six publicly available smartphone-based HAR datasets standardized onto a common format specifically to support cross-dataset research; no new data is collected. The sensing modalities are the ones that DAGHAR provides: tri-axial accelerometer and tri-axial gyroscope. Cross-dataset generalization is studied exclusively under the leave-one-dataset-out (LODO) protocol, in its strict domain-generalization form: models are trained on five datasets and evaluated on the sixth, which contributes nothing to training or model selection—not even unlabeled data. The model families investigated are graph neural networks (operating on a graph over the sensor channels) and an established convolutional–recurrent architecture used both as a comparison point and as a fusion partner; domain-adaptation and domain-generalization techniques from the wider literature are evaluated as candidate components within these families. On the storage side, the scope is the compression of a single, locked model along five axes—architectural capacity, numerical precision, graph edges, graph nodes, and temporal window length—and the characterization of what each axis costs in generalization.

This thesis does not collect or annotate new sensor data, and it does not extend beyond the accelerometer and gyroscope to other modalities (audio, barometer, heart rate, video). It targets the offline recognition setting defined by DAGHAR’s three-second windows. Online or streaming inference, latency-optimal scheduling on real handsets, and energy measurement on physical devices are engineering questions left outside the study; storage and CPU inference time are used instead as efficiency proxies. Finally, the thesis does not attempt person-level personalization or continual learning: every model is trained once on its source datasets and evaluated on the frozen one.

1.4 Research Questions and Objectives

The two connected goals stated in Section 1.2 are translated into two research questions, one per problem, each with concrete objectives.

RQ1 (domain generalization). *To what extent can a graph-based HAR model, trained with no access to its target dataset, generalize to that unseen dataset—and which design choices govern how well it does so?* The objectives under this question are:

1. Establish graph-neural-network baselines on DAGHAR under the LODO protocol to quantify the gap between within-dataset and cross-dataset performance, and design a graph representation and node-feature scheme — covering node definition and signal views (time-domain, frequency-domain, hybrid) — suited to cross-dataset transfer.
2. Incorporate and evaluate domain-invariance mechanisms and representative domain-adaptation techniques under a strictly target-blind selection protocol, and develop an adaptive graph-construction method that chooses edges per window from the data, anchored by a physical prior, evaluating whether it closes the gap to the benchmark’s convolutional state of the art, alone and in fusion with a convolutional branch.
3. Validate every adopted component across multiple random seeds, with target-blind model selection maintained throughout.

RQ2 (storage optimization under domain generalization). *When the resulting model is compressed, how much cross-dataset generalization is lost per unit of storage saved—which reductions are free, which are fatal, and what governs the difference?* The objectives under this question are:

1. Define a storage-optimization framework of compression operators acting on the locked model along five storage axes, each evaluated at matched budgets, and propose a joint accuracy-and-size metric (generalization-per-byte) so operating points can be ranked on one scale.

2. Characterize the storage–generalization trade-off along each axis and in combination, identifying Pareto-efficient operating points, and isolate the effect of graph structure from graph budget by comparing principled and naive edge-selection policies at identical edge counts.
3. Explain, rather than merely report, the observed pattern — including why invariance training helps only modestly — using a training-dynamics diagnostic.

Objectives RQ1.1–RQ1.5 are addressed in Chapters 3 and 4 (the framework and its empirical validation); objectives RQ2.1–RQ2.5 are addressed by the storage-optimization framework of Chapter 3 and the trade-off study of Chapter 4.

1.5 Summary of Contributions

This thesis makes five contributions which are summarized here:.

1. The central methodological contribution is an adaptive graph-construction rule for the sensor graph. For every input window, it scores each candidate sensor pair by blending that window’s own inter-channel correlation with a fixed physical prior over sensor placement, then keeps the highest-scoring pairs as edges. The resulting fused system matches the strongest published convolutional baseline on the DAGHAR benchmark, across all six held-out datasets. A purely data-driven version of the same rule fails on at least one held-out dataset, showing that the physical prior is necessary, not optional.
2. The thesis also proposes generalization-per-byte, a metric that ranks compressed models by the cross-dataset accuracy they retain per unit of storage. This metric is used to run the first systematic accuracy-versus-storage analysis for a graph-based, cross-dataset HAR model, across five separate compression axes evaluated at matched budgets. The analysis finds the cost of compression to be uneven: some reductions, such as dropping a redundant sensor channel, come at almost no cost, while others trade away meaningfully more accuracy for their savings. The most efficient operating points among these are identified explicitly.

3. A further finding is that graph structure, not graph size, drives generalization. At the same edge budget, choosing the right sensor connections outperforms random edge selection by more than eight accuracy points, consistently across every held-out dataset. This shows that which connections a graph keeps is a meaningful design choice in its own right, not simply a way to save storage.
4. The thesis also explains why domain-adversarial training helps only modestly, rather than leaving this as an unexplained result. During training, the model’s activity-classification objective and its domain-invariance objective are shown to pull the shared encoder in opposing directions on most training steps. This gradient conflict accounts both for the limited gains from moderate invariance training and for the regressions seen when the mechanism is made stronger.
5. Finally, the thesis treats a broad set of negative results as evidence in their own right. Additional model capacity, several feature and normalization variants, test-time adaptation, pseudo-labeling, and domain-adaptation methods with access to unlabeled target data were all evaluated under one consistent protocol. None of these alternatives cleared a pre-declared adoption bar. Together, they indicate that the cross-dataset bottleneck in HAR is structural, rather than something more parameters or more information about the target dataset can solve.

1.6 Organization of the Thesis

The remainder of this thesis is organized as follows. Chapter 2 provides the background this work builds on: the HAR task and its traditional and deep-learning treatments, graph neural networks and graph representations of sensor data, domain shift and the distinction between domain adaptation and domain generalization, the DAGHAR benchmark itself, and the storage-optimization literature, closing with the research gap this thesis addresses. Chapter 3 presents the methodology: the dataset and its preparation, the proposed generalization framework and each of its components,

the storage-optimization framework applied to the locked model, and the experimental protocol and evaluation metrics—including the proposed generalization-per-byte measure—under which every result is obtained. Chapter 4 reports the empirical studies as a chronological progression: from the initial sharp drop in cross-dataset accuracy, through the representation, invariance, and fusion studies, to the adaptive-graph result and the storage–generalization trade-off study, with negative results retained as evidence throughout. Chapter 5 presents the conclusion and revisits the research questions and contributions in light of the evidence, states the limitations of the work, and outlines directions for future research.

Chapter 2

Background & Literature Review

2.1 Human Activity Recognition

Human Activity Recognition (HAR) is the task of inferring what physical activity a person is performing – walking, sitting, climbing stairs, running, and so on – from a stream of sensor measurements. The problem has attracted sustained attention because an accurate, continuously running activity classifier is a building block for a wide range of applications: remote health monitoring and fall detection in eldercare, rehabilitation tracking, fitness and wellness applications, context-aware mobile computing, and industrial ergonomics monitoring, among others [3]. What distinguishes HAR from adjacent recognition problems, such as gesture recognition or gait analysis, is its focus on classifying extended, repetitive, whole-body movement patterns from short windows of continuous sensor data, typically on the order of a few seconds, rather than discrete one-off events.

HAR systems can draw on several families of sensing modality. Ambient or environmental sensors (cameras, pressure mats, infrared motion detectors) infer activity from changes an occupant induces in a fixed environment; video-based HAR in particular has its own large and mature computer-vision literature, but ambient sensing is intrinsically tied to instrumented spaces and raises privacy concerns that limit its use for continuous, everyday monitoring. Wearable inertial sensing takes the opposite approach: an accelerometer and, increasingly, a gyroscope are attached

to or carried by the body, and activity is inferred from the resulting motion signal wherever the person happens to be. This is the modality this thesis is concerned with, and for good reason – it is the modality on which the datasets used throughout this work were collected, and it is by a wide margin the modality most compatible with everyday, unobtrusive, privacy-preserving activity monitoring, because the sensor package is already present in essentially every smartphone and smartwatch in general circulation.

An inertial HAR pipeline built on a smartphone or wrist-worn device typically has access to a triaxial accelerometer, which measures linear acceleration (including the constant contribution of gravity) along three orthogonal body-fixed axes, and often a triaxial gyroscope, which measures angular velocity about those same axes. Six raw channels – three accelerometer axes and three gyroscope axes – sampled at rates typically between 20 and 100 Hz are therefore the common starting point for the great majority of inertial HAR systems, including the six datasets used in this thesis. Because accelerometers alone are cheaper, lower-power, and present on effectively every consumer device, much of the earlier HAR literature restricted itself to accelerometer-only signals; the addition of a gyroscope channel, now standard on modern smartphones, supplies complementary information about rotational motion that is particularly useful for distinguishing activities with similar translational but different rotational signatures (for example, walking versus walking upstairs). The datasets studied in this thesis all provide this now-standard six-channel accelerometer-plus-gyroscope signal, windowed into short, fixed-length segments – the representation that every model described in the remainder of this chapter, and built in Chapter 3, ultimately consumes.

A HAR pipeline built on this signal conventionally proceeds through four stages: data acquisition (as just described), signal preprocessing (filtering, resampling, and windowing the raw stream into fixed-length segments), feature representation (deriving a description of each window that a classifier can consume), and activity classification (mapping that representation to one of a fixed set of activity labels). The two sections that follow this one trace how the feature-representation and classification stages of that pipeline have evolved, from handcrafted statistical features paired with classical machine-learning classifiers (Section 2.2), through end-to-end deep architectures

that learn their own features directly from the windowed signal (Section 2.3), to the graph-based representations that motivate this thesis’s own architecture (Section 2.4).

2.2 Traditional Machine Learning Approaches for HAR

Before deep learning became the dominant paradigm, HAR systems relied on a feature-engineering pipeline: a fixed window of raw accelerometer and gyroscope samples was reduced to a compact vector of handcrafted statistics, and a classical supervised learning algorithm was trained on that vector. Common handcrafted features included time-domain statistics computed per axis and per window – mean, standard deviation, minimum, maximum, and various measures of signal energy and correlation between axes – together with frequency-domain features obtained from a Fourier or wavelet transform of the window, such as the dominant frequency, spectral energy, and spectral entropy [3]. The appeal of this approach was that it produced a low-dimensional, interpretable feature vector that classical algorithms with modest training-data requirements could handle well.

Three classifiers recur throughout this literature and remain useful reference points precisely because they are still reported as baselines in modern benchmark papers: k -Nearest Neighbours (KNN), which assigns a window the majority label among its k nearest neighbours in feature space; Random Forest (RF), an ensemble of decision trees trained on bootstrapped samples and random feature subsets; and Support Vector Machines (SVM), which learn a maximum-margin separating hyperplane, typically with a non-linear kernel, between activity classes. A foundational example is the original UCI-HAR study[5] which trained a multiclass, hardware-friendly SVM on handcrafted time- and frequency-domain features extracted from smartphone accelerometer and gyroscope signals and is itself the origin of one of the six datasets aggregated into the benchmark used throughout this thesis.

These three classical classifiers remain informative not as historical curiosities but because the benchmark this thesis builds on, DAGHAR, reports them directly alongside its deep-learning baselines under an identical evaluation protocol [1]. On DAGHAR’s standardized view – the

six-dataset, unit- and sampling-rate-harmonized version of the data used throughout this thesis – Random Forest reaches a six-dataset mean accuracy of 78.8% on raw time-domain windows and 85.5% on frequency-domain features, SVM reaches 70.2% and 76.6% respectively, and KNN reaches 56.7% and 81.5% respectively, evaluated *within* each dataset (i.e., train and test partitions drawn from the same collection, not across datasets). These numbers are competitive with, and in the frequency domain occasionally exceed, several of the deep architectures benchmarked alongside them; Random Forest on frequency-domain features, for instance, outperforms four of the twelve deep models DAGHAR benchmarks on its own mean-accuracy column. This is an important anchor for what follows in this chapter and in Chapter 4: traditional feature-engineering-plus-classical-classifier pipelines are not a weak baseline that deep learning trivially dominates when the evaluation is confined to a single dataset. Their well-documented weakness, and the reason this thesis (like the deep-learning HAR literature more broadly) largely leaves them behind, is instead a generalization one – handcrafted features and classical classifiers tuned to one collection device, sensor placement, and user population transfer poorly to a new one, a limitation examined directly once cross-dataset evaluation is introduced in Section 2.5.

2.3 Deep Learning Approaches for HAR

The past decade has seen HAR research shift decisively toward deep learning architectures that consume the windowed sensor signal directly and learn their own internal feature representation end to end, rather than relying on a separate handcrafted feature-extraction stage [4]. This shift mirrors a broader pattern across sensor- and signal-processing domains and is motivated by the same basic argument: a sufficiently expressive learned representation, trained jointly with the classifier that consumes it, can capture regularities in the raw signal that a fixed set of handcrafted statistics was never designed to expose. The remainder of this section surveys three deep architecture families relevant to this thesis, in roughly the order they came to dominate the HAR literature: convolutional networks, recurrent and hybrid convolutional-recurrent networks, and, more

recently, transformer-based sequence models.

2.3.1 CNN-based models

Convolutional neural networks (CNNs) were the first deep architecture family to see wide adoption in HAR, largely by treating a window of inertial data as a short, multi-channel time series (or, in a common variant, reshaping it into a two-dimensional time-by-channel image) and applying learned convolutional filters along the time axis to detect local, translation-invariant motion motifs – the acceleration signature of a single footfall, for instance – which are then pooled and aggregated across the window to produce a fixed-length representation for classification. This family is deep and continues to grow: DAGHAR alone benchmarks nine distinct convolutional variants (one- and two-dimensional CNNs, several convolutional architectures adapted from the broader time-series literature, and residual convolutional networks with and without squeeze-and-excitation blocks), and convolutional architectures occupy nearly all of the top positions in that comparison – a one-dimensional CNN is the strongest time-domain model DAGHAR reports on its standardized view (83.1% six-dataset mean accuracy within-dataset), while the single best model overall, at 87.6% on frequency-domain input, is the convolutional-*recurrent* hybrid introduced in the next subsection [1]. The convolutional complementary branch built for this thesis (Section 3.2.7) sits squarely in this lineage, though – as the next subsection explains – it is more precisely a convolutional-recurrent hybrid than a pure CNN.

2.3.2 Recurrent and hybrid models

Because an activity window is fundamentally a short time series, an alternative and complementary line of work models it with recurrent architectures – most commonly Long Short-Term Memory (LSTM) networks, which maintain and update an internal hidden state as they process the signal step by step, making them naturally suited to capturing temporal dependencies that a convolutional filter’s fixed receptive field can only approximate. In practice, purely recurrent HAR models are less common than *hybrid* architectures that combine a convolutional front end, responsible for

extracting local per-timestep or per-short-window features, with a recurrent back end that aggregates those features across the sequence. The architecture that established this pattern, and that is directly relevant to this thesis, is DeepConvLSTM: a stack of convolutional layers followed by LSTM layers and a final classification layer, introduced by Ordóñez and Roggen [2] for multimodal wearable activity recognition and shown to outperform a purely convolutional baseline on standard HAR benchmarks. The convolutional complementary branch used throughout this thesis’s fusion architecture (Section 3.2.7) is a direct reimplement of this convolutional-then-recurrent design – four convolutional blocks operating on the window’s frequency-domain view, followed by a stacked-LSTM recurrent block and a linear classifier on the final aggregated hidden state – chosen specifically because it gives the graph-based branch developed in this thesis a complementary, non-graph baseline architecture to fuse against, rather than fusing two variants of the same representation.

Before settling on this DeepConvLSTM-style branch, our own earlier work [6] explored a closely related hybrid design directly on one of the datasets later folded into the DAGHAR benchmark used throughout this thesis. That study combined convolutional layers for spatial (cross-axis) feature extraction with stacked LSTM and Gated Recurrent Unit (GRU) layers for temporal modelling, and added an attention mechanism intended to let the network weight the most informative timesteps within a window dynamically rather than aggregating the whole sequence uniformly; evaluated on the RealWorld-Waist smartphone dataset – the same collection later standardized as one of the six DAGHAR sources – the hybrid CNN-LSTM-GRU-with-attention model achieved strong within-dataset cross-validation accuracy relative to comparable architectures in the literature. That study was, however, a single-dataset evaluation: the model was trained and tested exclusively on RealWorld-Waist data, with no assessment of how it would perform on a smartphone from a different collection, a different sensor placement, or a different user population. It was precisely this gap – how much of a HAR model’s apparent accuracy survives when the training and test data come from genuinely different sources – that motivated the shift, in the present thesis, from single-dataset model design toward the cross-dataset generalization problem formalized later

in this chapter and evaluated throughout Chapter 4.

2.3.3 Transformer-based models

Transformer architectures, built around a self-attention mechanism that lets every position in a sequence attend directly to every other position rather than propagating information step by step as a recurrent network does, have more recently been adapted to HAR. Applied to a window of inertial data, self-attention allows a model to relate distant timesteps within the window directly, in principle capturing long-range temporal dependencies that a convolutional filter’s local receptive field or a recurrent network’s sequential state update might dilute; Dirgová Luptáková et al. [7], for example, adapt a transformer encoder to raw accelerometer windows and report accuracy competitive with, though not uniformly exceeding, convolutional and recurrent baselines on standard HAR benchmarks. This family is included in DAGHAR’s own baseline comparison as well, but notably underperforms there: the transformer variant DAGHAR benchmarks reaches only a 63.8% (time-domain) to 71.0% (frequency-domain) six-dataset mean accuracy, the weakest of the twelve deep architectures reported, trailing even the classical Random Forest baseline discussed in Section 2.2 [1]. Because transformer-based HAR is not the direction pursued in this thesis, and because the specific benchmark this thesis builds on offers no evidence that it is currently the strongest architecture family for this task at this data scale, it is discussed here only for completeness; the architectural direction this thesis instead pursues – representing a window’s multiple sensor channels not as a flat multi-channel sequence but as an explicit graph – is introduced next.

2.4 Graph Neural Networks for HAR

The architectures surveyed so far – convolutional, recurrent, and transformer-based – all share an implicit assumption: that the six raw sensor channels (three accelerometer axes, three gyroscope axes) can be treated as an undifferentiated multi-channel time series, with any relationship between channels left for the network to discover indirectly, if at all, through its filters or attention weights.

Graph Neural Networks (GNNs) offer a different starting point, one that motivated this thesis’s own line of experimentation from an early stage: they let the relationships between sensing channels be represented *explicitly*, as edges in a graph, and let the model’s computation follow that explicit structure. Our own first engagement with this literature was a systematic review surveying how GNN-based models have been applied across the sensor-based HAR pipeline, from data acquisition and preprocessing through feature extraction to activity classification, framed around the broader context of AI-driven, sensor-based healthcare monitoring [8]. That review – covering, among others, exactly the message-passing formalism and graph-representation choices discussed in the two subsections below – is what first surfaced the specific design question this thesis eventually answers: sensor-based HAR papers using GNNs almost universally represent *timesteps*, not sensing channels, as the nodes of the graph, an architectural choice examined critically in Section 2.4.2 and revisited directly in the adaptive sensor-graph design developed in Section 3.2.6.

2.4.1 GNN fundamentals

A graph neural network operates on data structured as a graph $G = (V, E)$, a set of nodes V connected by a set of edges E , where each node $v \in V$ carries a feature vector h_v . The unifying computational principle behind essentially every GNN architecture is *message passing*: at each layer, every node aggregates information from its neighbours, combines that aggregated information with its own current representation, and produces an updated representation for the next layer. Gilmer et al. [9] formalize this pattern explicitly as the Message Passing Neural Network (MPNN) framework, in which a layer’s update for node v with neighbourhood $\mathcal{N}(v)$ takes the general form

$$h_v^{(l+1)} = \text{UPDATE}\left(h_v^{(l)}, \text{AGGREGATE}(\{h_u^{(l)} : u \in \mathcal{N}(v)\})\right), \quad (2.1)$$

and show that a wide range of previously proposed graph architectures are instances of this single template, differing only in their choice of aggregation and update functions. Four specific instantiations of Equation (2.1) recur throughout the GNN and HAR-on-graphs literature and are directly

relevant to the architecture developed in this thesis.

The Graph Convolutional Network (GCN) of Kipf and Welling [10] defines the aggregation step as a fixed, degree-normalized average over a node’s neighbours (including itself), computed for an entire graph in one matrix operation as $H^{(l+1)} = \sigma(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(l)}W^{(l)})$, where $\tilde{A}$ is the graph’s adjacency matrix with self-loops added, $\tilde{D}$ its corresponding degree matrix, $W^{(l)}$ a learned weight matrix, and σ a non-linearity; this fixed-weight neighbour averaging makes GCN simple and effective but leaves it unable to learn that some neighbours should matter more than others. The Graph Attention Network (GAT) of Veličković et al. [11] addresses exactly this limitation by learning an attention coefficient α_{vu} for each edge, computed as a softmax-normalized function of the two endpoint features, so that the aggregation step becomes a weighted sum $h_v^{(l+1)} = \sigma(\sum_{u \in \mathcal{N}(v)} \alpha_{vu} W h_u^{(l)})$ with edge weights learned jointly with the rest of the network rather than fixed in advance. GraphSAGE, introduced by Hamilton et al. [12], instead emphasizes *inductive* generalization to graphs (or edge sets) unseen during training by learning a fixed aggregation function – the original paper explores mean, LSTM, and pooling aggregators – rather than a full node-specific weight matrix per graph, together with a neighbourhood-sampling scheme that keeps aggregation cost bounded on large or variable-degree graphs; the two-layer GraphSAGE backbone is the specific message-passing operator this thesis’s own sensor graph model (Section 3.2.6) is built on, chosen precisely for this inductive property, since the six datasets underlying this thesis’s cross-dataset evaluation protocol never share identical graph structure across LODO folds. Finally, the Graph Isomorphism Network (GIN) of Xu et al. [13] approaches the same aggregation choice from a theoretical angle, showing that a GNN’s ability to distinguish non-isomorphic graphs is bounded by the classical Weisfeiler–Lehman graph-isomorphism test, and that a sum aggregator combined with a multi-layer perceptron update is, under mild assumptions, sufficient to match that theoretical upper bound – a result less directly load-bearing for this thesis’s six- or twelve-node sensor graphs (Section 2.4.2) than for very large, structurally diverse graphs, but included here because it clarifies precisely what a message-passing GNN can and cannot, in principle, distinguish.

2.4.2 Graph representations of sensor data

Applying a GNN to inertial HAR data requires an explicit decision that has no equivalent in the CNN or recurrent architectures of Section 2.3: what, exactly, should be a *node*? The systematic review discussed at the opening of this section found that the great majority of GNN-based HAR work adopts what can be called a *timestep-as-node* representation: each timestep (or each short sub-window) of the signal becomes a node, and edges are constructed between temporally adjacent, or otherwise similar, timesteps, so that the graph essentially re-encodes the same sequential structure a recurrent network would process, with message passing substituting for the recurrent state update [8]. This is a natural first choice, since it requires no assumption about which sensing channels relate to one another, but it also means the resulting graph is graph-structured in name only – its structure is still fundamentally a sequence, and it forfeits the one property a graph representation is uniquely positioned to offer, an explicit representation of relationships *between* sensing channels rather than between successive moments in time.

An alternative, considerably less common in the literature but directly adopted throughout this thesis, is what can be called a *sensor-as-node* representation: each sensing channel (or, as adopted in this thesis’s own architecture, each of the six accelerometer and gyroscope axes) becomes a single node, its feature vector summarizing that channel’s behaviour across the whole window, and edges represent physical or statistical relationships between channels rather than between moments in time. Yang et al. [14] adopt exactly this representation for a graph convolutional domain-adaptation model applied to sensor-based HAR, constructing a graph over sensor channels and explicitly motivating the choice as a way to let the model exploit cross-sensor correlation structure directly rather than only temporal structure; that paper is the specific point of departure this thesis’s own adaptive sensor-graph construction builds outward from, extending a fixed sensor-channel graph into a graph whose edge structure is itself learned per input window from a blend of statistical correlation and a physical prior over sensor placement. The trade-off between the two representations is, in short, that a timestep-as-node graph keeps temporal structure explicit at the cost of channel structure, while a sensor-as-node graph does the reverse – and it is the sec-

ond of these trade-offs, together with the domain-shift and cross-dataset evaluation considerations introduced next, that this thesis’s architecture is built around.

2.5 Cross-Dataset Generalization and Domain Shift in HAR

Every architecture surveyed so far was, in its original presentation, evaluated *within* a single dataset: the training and test partitions were drawn from the same collection campaign, recorded on the same devices, at the same body positions, under the same protocol, from overlapping or at least demographically similar user populations. Under those conditions the numbers are impressive – as Section 2.2 noted, even a Random Forest on frequency-domain features exceeds 85% mean accuracy on the benchmark used in this thesis. The situation changes qualitatively when the test data come from a genuinely different source. A HAR dataset is not a neutral sample of “human activity” but the product of a specific device (with its own sensor hardware, sampling rate, and preprocessing firmware), a specific body placement (pocket, waist, wrist, thigh), a specific set of instructed activities and recording protocols, and a specific user population; change any of these and the joint distribution of signals and labels shifts, often substantially. In the machine-learning literature this phenomenon is called *domain shift* or *distribution shift*: the training data are drawn from one or more *source domains* and the deployment data from a *target domain* whose distribution differs, violating the independent-and-identically-distributed assumption under which supervised learning’s generalization guarantees hold [15]. In smartphone HAR the practical consequence is well documented and severe – models that exceed 85–90% accuracy within their own dataset routinely lose ten to twenty accuracy points when evaluated on a different HAR dataset, a drop the DAGHAR benchmark of Section 2.6 was constructed specifically to measure cleanly [1]. Since the deployment scenario that motivates smartphone HAR in the first place – a pre-trained activity model shipped to phones, placements, and users never seen during development – is precisely the cross-domain scenario, this gap between within-dataset and cross-dataset accuracy, not within-dataset accuracy itself, is the real open problem. This section introduces the concepts

and methods the field has developed for it, all of which reappear, as adopted components or as evaluated-and-rejected alternatives, in Chapters 3 and 4.

2.5.1 Domain Adaptation versus Domain Generalization

Two research programmes address domain shift, and the boundary between them – defined by *what the model is allowed to see of the target domain during training* – matters enough to this thesis that it organizes which components of the final system were kept and which were rejected. *Domain adaptation* (DA) assumes that some data from the target domain, usually unlabeled, are available at training or adaptation time, and uses them to align the model’s representation with the specific target it will be deployed on [16]. The theoretical foundation is due to Ben-David et al. [15], who bound a classifier’s target-domain error by its source-domain error plus a measure of divergence between the two domains’ feature distributions – a bound that directly licenses the dominant DA strategy of learning representations that minimize this measured divergence to the target. *Domain generalization* (DG), by contrast, assumes the target domain is entirely unseen: the model is trained on multiple source domains and must generalize to a new domain about which nothing – not even unlabeled data – is available before deployment [17, 18]. DG methods therefore cannot align to the target; they can only exploit the diversity among the sources, learning representations, augmentations, or training dynamics that are invariant across the source domains in the hope that this invariance extends to the unseen one.

The distinction is not academic bookkeeping; it encodes two different deployment scenarios. DA fits situations where each deployment target can be visited before use – unlabeled data collected on the new device, an adaptation pass run per target. DG fits the scenario that actually characterizes consumer smartphone HAR: a single model shipped to millions of heterogeneous devices, placements, and users, with no per-target data collection or adaptation step available. For this reason the present thesis adopts the DG formulation as its governing protocol – the unseen target dataset contributes nothing to training or model selection – while still evaluating several DA techniques along the way, both because the DA literature is far larger and because a DA method’s

failure *even when granted unlabeled target access* is informative evidence about the nature of the shift. Chapter 4 keeps the two families explicitly labelled and separated throughout, and its recurring finding – that target-aware DA methods fail to beat a well-constructed target-blind DG system on this benchmark – is one of the observations the thesis’s structural-bottleneck argument rests on.

2.5.2 Leave-One-Dataset-Out as an Evaluation Protocol

Evaluating a DG claim requires an evaluation protocol in which the test domain is genuinely unseen. The DG literature’s standard instrument is *leave-one-domain-out* cross-validation: given K domains, train on $K - 1$ of them, test on the held-out domain, and rotate the held-out role through all K domains, reporting the mean. The DomainBed study of Gulrajani and Lopez-Paz [19] established this as the field’s reference protocol and, importantly, showed how easily it is undermined by careless *model selection*: if hyperparameters, checkpoints, or design choices are tuned on the held-out domain’s data, the evaluation silently leaks target information and overstates generalization – under rigorous source-only model selection, they found, no DG algorithm in their survey convincingly beat well-tuned empirical risk minimization. The methodological lesson, which this thesis adopts as a hard rule, is that a DG evaluation is only as credible as its model-selection discipline: every choice must be made on source-domain validation data alone.

Instantiated at the dataset level for HAR, the protocol becomes *Leave-One-Dataset-Out* (LODO): each of the six DAGHAR datasets in turn serves as the unseen target while the other five serve as sources, treating each dataset – with its distinct devices, placements, protocols, and populations – as one domain [1]. Dataset-level held-out evaluation is a deliberately harsh test: unlike *leave-one-user-out* protocols common in earlier HAR work, which hold out individuals but keep device, placement, and protocol fixed, LODO shifts all of these factors at once, which is exactly what makes it a faithful proxy for real deployment. LODO is used in two places in this thesis and is accordingly introduced twice: conceptually in Section 3.1.4, and as a full procedural specification – including the source-only model-selection rule motivated above – in Section 3.4.1.

2.5.3 Domain-Adversarial Training

Among methods for learning domain-invariant representations, the most influential is *Domain-Adversarial Neural Network* (DANN) training, introduced by Ganin et al. [20] as a direct operationalization of the Ben-David et al. [15] bound: if target error is bounded by source error plus inter-domain divergence, then train a representation that a classifier *cannot use to tell the domains apart*. DANN attaches a second head to the feature extractor – a *domain classifier* trained to predict which domain each example came from – and connects it through a *gradient reversal layer* that negates the domain-classification gradient as it flows back into the feature extractor. The result is a minimax dynamic trained by ordinary backpropagation: the domain classifier gets better at identifying domains while the feature extractor is pushed to produce representations on which that identification fails, converging toward features that are discriminative for the task label but uninformative about domain membership.

Although DANN was proposed as a DA method – its original formulation discriminates source from *target* examples, requiring unlabeled target data – the mechanism transfers directly to the multi-source DG setting: the domain classifier is instead trained to distinguish the source domains *from one another*, which requires no target data at all and encourages representations invariant across the sources [17]. It is in exactly this target-blind, multi-source configuration that domain-adversarial training is used in this thesis, as one adopted component of the framework of Chapter 3; the loss and the gradient-reversal weighting schedule are formalized in Section 3.2.5, and the empirical evaluation that led to its adoption – including the measured characterization of *why* its gains, though consistent, are modest – appears in Chapter 4.

2.5.4 Other Domain Adaptation and Domain Generalization Techniques

Beyond adversarial training, four further families from the DA/DG literature are background for this thesis, because Chapter 4 evaluates a representative of each on the cross-dataset HAR problem.

Statistical feature alignment methods replace DANN’s learned domain critic with an explicit statistical distance between domain feature distributions, minimized as an auxiliary loss. The

two standard choices are Maximum Mean Discrepancy (MMD), a kernel-based distance between distribution embeddings [21] used as a training objective in the Deep Adaptation Network of Long et al. [22], and correlation alignment (CORAL), which matches the second-order statistics – the feature covariance matrices – of source and target, extended to deep networks as a differentiable loss in Deep CORAL [23]. A related line moves the alignment into the normalization layers themselves: domain-specific batch normalization maintains separate batch-normalization statistics (or entire normalization branches) per domain [24], and Transferable Normalization redesigns the normalization layer so that domain alignment happens there rather than in an added loss [25].

Test-time and inference-time adaptation methods defer adaptation until deployment, adjusting a trained model using only the unlabeled test data it encounters. The simplest and most widely used variant is Adaptive Batch Normalization (AdaBN), which recomputes the batch-normalization running statistics on the target data while leaving all learned weights untouched [26]; more aggressive variants such as Tent additionally update the normalization layers’ affine parameters online by minimizing the entropy of the model’s own test-time predictions [27].

Self-training with pseudo-labels takes yet another route to exploiting unlabeled target data: a trained model assigns provisional labels to the target examples it is most confident about, and a student model is then retrained on the source data augmented with these pseudo-labeled target examples [28]. Its well-known failure mode is confirmation bias – when the teacher’s confident predictions are wrong, the student learns the teacher’s errors – which makes the technique sensitive to exactly how far the target domain is from the sources.

Purpose-built DG techniques, finally, require no target data of any kind and instead manipulate the training distribution or the training dynamics. MixStyle synthesizes novel “styles” during training by mixing feature statistics between instances drawn from different source domains, on the premise that domain identity is largely carried by those statistics [29]; SWAD seeks flat minima of the training loss by densely averaging model weights along the training trajectory, on the evidence that flatter solutions generalize better across domain shift [30].

All four families are evaluated in Chapter 4 – statistical alignment and normalization-based

methods in Section 4.4.1 and Section 4.4.4, test-time adaptation in Section 4.4.3, pseudo-label self-training in Section 4.6.2, and the purpose-built DG pair among the robustness attempts of Arc III – and none of them survives the thesis’s keep/revert criterion on this benchmark. Their systematic failure, including that of the methods granted unlabeled target access that the governing DG protocol denies to the final system, is what motivates this thesis’s shift of attention away from *what alignment objective is added to the model* and toward *how the model’s input representation – the sensor graph itself – is constructed*.

2.6 The DAGHAR Benchmark

The concepts of the previous section presuppose something the HAR field long lacked: a collection of datasets that can legitimately be treated as multiple domains of the *same* task. Public inertial HAR datasets accumulated over more than a decade, but each was collected with its own devices, sampling rates, accelerometer units, window lengths, and – most disruptively – its own activity vocabulary and labelling conventions. Any attempt to train on one dataset and test on another therefore confounded two very different things: the genuine domain shift of interest (device, placement, protocol, population) and trivial format incompatibilities that no deployed model would actually face, such as one dataset reporting acceleration in g and another in m/s^2 , or one sampling at 100 Hz and another at 20 Hz. Cross-dataset results computed over unharmonized datasets are consequently difficult to interpret, and before 2024 the field had no standard instrument for separating the two effects.

The DAGHAR benchmark of Napoli et al. [1] was constructed to close exactly this gap. It curates six established smartphone HAR datasets – KuHar, MotionSense, the thigh and waist placements of RealWorld (treated as two separate domains, since placement itself is a domain factor), UCI-HAR, and WISDM – spanning waist, pocket, and thigh placements, user populations from 15 to 90 subjects, and original sampling rates from 20 to 100 Hz. The curation was selective, not exhaustive: candidate datasets whose sensor data could not be trusted were evaluated and discarded,

one because its gyroscope channel proved to be a delayed and noisy copy of its accelerometer signal, another because its in-the-wild self-labelling produced heavy class imbalance and unreliable annotations. Each retained dataset is published in two forms. The *baseline view* applies only each dataset’s own conventional preprocessing, while the *standardized view* – the one used throughout this thesis – harmonizes everything that is format rather than substance: accelerometer units are converted to m/s^2 , all signals are resampled to 20 Hz, gravity is removed with a third-order high-pass Butterworth filter, recordings are sliced into non-overlapping three-second windows of 60 timesteps, labels are mapped onto a shared six-activity vocabulary (sit, stand, walk, stair-up, stair-down, run), and each dataset is split at the *user* level into balanced train, validation, and test partitions, so that no individual contributes windows to more than one partition. Two coverage gaps survive standardization because they are properties of the original collections: UCI-HAR contains no *run* class and WISDM no *stair* classes. The per-dataset specifics, as this thesis consumes them, are tabulated in Section 3.1.1.

A benchmark that harmonizes formats is only useful if the harmonization does not also erase the domain differences it is meant to expose, and DAGHAR’s most important validation result addresses precisely this concern: model performance on the standardized view is close to identical to performance on the baseline view, with performance ratios near 1.0 across its whole classifier suite [1]. Standardization, in other words, removes the trivial incompatibilities without flattening the real domain gaps – which licenses the central methodological premise of this thesis, namely that a cross-dataset accuracy drop measured on the standardized view reflects genuine domain shift rather than a formatting artifact. The benchmark paper’s own distributional analyses characterize that remaining shift directly: a t-SNE analysis shows activities separating into low-, mid-, and high-energy bands far more cleanly in the frequency domain than in the time domain; a placement analysis shows static activities occupying a common region across datasets while dynamic activities form placement-specific clusters; and an MMD analysis between datasets identifies KuHar as the most dissimilar domain from all others – despite sharing a waist-level placement with UCI-HAR and RealWorld-Waist – indicating that demographic and protocol factors can outweigh sen-

Table 2.1: Cross-dataset evaluation collapses within-dataset accuracy on DAGHAR. Six-dataset mean accuracy of representative classifiers from the benchmark’s own suite [1], on the standardized view, under within-dataset evaluation versus the LODO protocol, for time-domain and frequency-domain input. ConvNet denotes the convolutional-recurrent DeepConvLSTM architecture of Ordóñez and Roggen [2].

Model	Within-dataset		Cross-dataset (LODO)	
	Time	Frequency	Time	Frequency
KNN	56.7%	81.5%	43.5%	66.6%
Random Forest	78.8%	85.5%	54.9%	72.2%
SVM	70.2%	76.6%	56.0%	68.6%
CNN (1D)	83.1%	84.3%	72.2%	73.9%
ConvNet	83.3%	87.6%	60.3%	77.1%
MLP (3 layers)	74.7%	75.8%	60.8%	75.8%
IMU Transformer	63.8%	71.0%	56.2%	72.9%

sensor placement itself. These analyses are reproduced and examined further, in the specific form this thesis uses them, in the exploratory analysis of Section 3.1.2.

Finally, DAGHAR benchmarks fifteen classifiers – the three classical baselines of Section 2.2 and twelve deep architectures spanning the families of Section 2.3 – on both views, in both the time and frequency domains, under both within-dataset evaluation and the LODO protocol of Section 2.5.2. Table 2.1 condenses the standardized-view results for representative models, and three observations from it frame everything that follows in this thesis. First, the within-to-cross-dataset drop is large for every model family: the best within-dataset mean of 87.6% falls to a best LODO mean of 77.1%, and the classical classifiers – so competitive within-dataset – fall considerably further, confirming that their handcrafted-feature pipelines transfer worst of all. Second, the drop afflicts even dataset pairs with similar placements and low measured MMD, which is the benchmark’s own evidence that external factors – demographics, collection protocols – dominate the shift, and its stated motivation for calling for dedicated adaptation and generalization research. Third, convolutional models are consistently the most robust under LODO – the best time-domain LODO mean belongs to a one-dimensional CNN (72.2%) and the best frequency-domain LODO mean to the convolutional-recurrent ConvNet (77.1%) – while the benchmarked transformer, weakest within-dataset, remains unremarkable cross-dataset. That 77.1% frequency-domain ConvNet

mean is the strongest published cross-dataset result on this benchmark, and it is the external reference point against which this thesis’s own systems are judged throughout Chapters 3 and 4; consistent with the framing of Chapter 1, the aim there is stated as *matching* this reference under a stricter, target-blind protocol, with the contribution lying in how the result is reached.

2.7 Storage and Model Optimization for Deep Learning and Graph Neural Networks

The literature surveyed so far concerns what a HAR model can recognize; this section concerns what it costs. The deployment scenario that motivates smartphone HAR – continuous inference on battery-powered consumer devices, potentially alongside many other applications – makes model size, memory footprint, and computational cost first-class constraints rather than afterthoughts, and a substantial literature has grown around reducing them. That literature divides broadly into two strategies: *compressing* a trained model after (or during) training, and *designing* architectures that are small from the outset. Both reappear, adapted to the graph setting, in the storage-optimization framework of Section 3.3.

The most established compression family is *parameter pruning*: removing weights judged unimportant, classically by magnitude, and retraining to recover accuracy. Han et al. [31] showed that iterative magnitude pruning removes the large majority of a network’s weights with negligible accuracy loss, and the follow-up Deep Compression pipeline [32] combined pruning with weight quantization and entropy coding to shrink storage by an order of magnitude or more. The lottery-ticket hypothesis of Frankle and Carbin [33] sharpened the interpretation of such results, showing that dense networks contain sparse subnetworks – winning tickets – that can be trained in isolation to full accuracy, implying that much of a trained network’s parameter budget is redundant with respect to its final function. A second family, *quantization*, reduces the precision rather than the number of parameters: Jacob et al. [34] demonstrated 8-bit integer inference with accuracy close to full precision, and the technique is now standard in on-device deployment toolchains. Knowledge

distillation [35] instead trains a small student network to mimic a large teacher’s output distribution, and *efficient architecture design* bypasses compression altogether – MobileNets [36] and the compound width/depth/resolution scaling of EfficientNet [37] established that capacity can be scaled down as a design axis, a strategy that corresponds directly to the architecture-capacity arm of this thesis’s storage framework (Section 3.3.1).

One finding from this literature deserves particular emphasis here, because it foreshadows the question this thesis asks. Hooker et al. [38] showed that headline test accuracy conceals what compression actually does: pruned and quantized models match their dense counterparts on average while degrading disproportionately on a narrow subset of atypical, underrepresented examples – the long tail of the training distribution. Compression, in other words, preferentially sacrifices exactly the capacity that handles inputs unlike the typical training example. Under domain shift, *every* input from an unseen target domain is, in a sense, atypical of the training distribution – which makes the interaction between compression and cross-domain generalization an obvious concern, and one that the compression literature, whose evaluations are almost universally conducted in-distribution, has largely not measured. What little evidence exists is not even uniformly negative: Diffenderfer et al. [39] found that certain carefully compressed networks can *improve* out-of-distribution robustness relative to their dense counterparts. Mixed findings of this kind do not settle the question; they sharpen the case that the compression–generalization interaction must be measured, per task and per architecture family, rather than assumed in either direction.

Graph neural networks add a storage object that the CNN-oriented compression literature does not contemplate: the graph itself. Alongside weight-oriented techniques, a GNN’s cost can be reduced by sparsifying its edge set, coarsening its nodes, or quantizing under the irregular numerical conditions message passing creates. Each has a developed line of work: NeuralSparse learns to drop task-irrelevant edges jointly with the downstream GNN [40]; the unified lottery-ticket formulation of Chen et al. [41] extends iterative magnitude pruning to prune the adjacency matrix and the model weights simultaneously; and Degree-Quant addresses GNN-specific quantization error, showing that high-degree nodes disproportionately degrade under naive low-precision aggregation

[42]. Two features of this literature matter for what follows. First, it is evaluated almost exclusively on large graphs – citation networks, social graphs, molecule datasets – where edges number in the millions and sparsification is a computational necessity; a sensor graph with six nodes inverts that regime, making the *choice* of which few edges to keep, rather than the number removed, the interesting variable. Second, like the CNN compression literature, it is evaluated in-distribution: what edge sparsification does to a GNN’s ability to generalize across domains is, to the best of our knowledge, uncharacterized.

Efficiency-oriented work exists within HAR as well, though it is comparatively recent and modest in volume. TinyHAR [43] is representative: a deliberately lightweight architecture several times smaller than DeepConvLSTM that meets or exceeds its accuracy on standard within-dataset benchmarks, part of a broader push toward HAR models that fit comfortably on wearable and edge hardware. What this line of work shares with the general compression literature is its evaluation regime: efficiency is traded against *within-dataset* accuracy, on the same benchmarks whose cross-dataset fragility Section 2.5 documented. The two constraints that jointly define realistic smartphone HAR deployment – the model must be small, and it must survive domain shift – are thus each well studied alone, but their interaction is not: whether the capacity a compressed model gives up is capacity that cross-dataset generalization needed is an open empirical question, and answering it for graph-based HAR under a strict DG protocol – including a metric that expresses generalization delivered per byte of storage (Section 3.5.2.2) – is the second contribution of this thesis, developed in Section 3.3 and evaluated in Section 4.7.

2.8 Research Gap

The strands surveyed in this chapter each come close to this thesis’s territory without occupying it, and it is worth being precise about where each one stops. Within GNN-based HAR (Section 2.4), the dominant timestep-as-node representation forfeits explicit cross-channel structure, and the sensor-as-node alternative of Yang et al. [14] – the nearest methodological ancestor of this

thesis’s graph – uses a fixed channel graph and is developed for domain *adaptation*, with target data available. The nearest recent neighbours are the companion frameworks of Ye and Wang [44] and Ye and Wang [45], which do combine a sensor-graph GNN with gradient-reversal adversarial training under a target-blind generalization protocol; but both address cross-*user* generalization within single multi-sensor body-network datasets (OPPORTUNITY, DSADS), their graph topologies are fixed anatomical priors selected among or cycled through during training rather than constructed per input window from the data, and neither considers deployment cost. On the evaluation side, the HAROOD benchmark of Lu et al. [46] confirms at scale, across sixteen OOD methods and four shift scenarios including cross-dataset, the pattern this chapter has already noted twice – no existing DA/DG algorithm consistently beats a well-tuned baseline – while benchmarking only CNN- and transformer-based architectures, with no graph-based models and no storage dimension. And the storage-optimization literature (Section 2.7), including its GNN-specific branch, evaluates indistribution, on graphs whose scale makes sparsification a computational necessity rather than a representational choice, with the compression–generalization interaction contested in general and unmeasured for HAR.

The gap, stated tightly: *no prior work combines data-driven, per-window sensor-graph construction, domain-adversarial training of a sensor-graph GNN, cross-architecture late fusion, and systematic storage optimization – evaluated together under a strict leave-one-dataset-out domain-generalization protocol on a standardized cross-dataset benchmark.* Each element exists somewhere in the literature; their combination, and the two questions it makes answerable, do not. Those questions are the research questions of Section 1.4: whether the construction of the sensor graph itself – rather than the choice of DA/DG objective layered on top – is the dominant lever for cross-dataset generalization in graph-based HAR; and what cross-dataset generalization actually costs in storage, i.e., which reductions of a generalizing model are free, which are cheap, and which destroy precisely the capacity that generalization requires. The first is answered by the adaptive prior-anchored graph of Section 3.2.6 and its evaluation in Section 4.5; the second by the storage-optimization framework of Section 3.3, the generalization-per-byte metric of Section 3.5.2.2, and

the trade-off characterization of Section 4.7.

2.9 Summary

This chapter positioned the thesis within five bodies of literature. It introduced HAR and the smartphone accelerometer-plus-gyroscope signal this thesis works with; traced the field’s classifier evolution from handcrafted features with classical models – still competitive within-dataset – through convolutional, recurrent, and transformer-based deep architectures; and introduced GNNs as the family that makes inter-channel structure explicit, together with the pivotal representational choice between timestep-as-node and the rarer sensor-as-node design this thesis adopts. It then formalized the problem that motivates everything else: domain shift makes within-dataset accuracy a poor predictor of cross-dataset performance, the DA/DG distinction separates methods by target-data access, LODO with source-only model selection is the credible evaluation instrument, and the DAGHAR benchmark supplies six standardized domains on which the strongest published cross-dataset result is a 77.1% frequency-domain ConvNet mean. Finally, it surveyed storage optimization for deep networks and GNNs, whose evaluations remain almost entirely in-distribution, and synthesized the resulting gap: the four-way combination of adaptive graph construction, domain-adversarial sensor-graph learning, cross-architecture fusion, and storage optimization under a strict DG protocol that this thesis is the first to evaluate. The next chapter specifies the methodology built to fill that gap.

Chapter 3

Methodology

This chapter describes how the thesis pursues its two connected goals: better cross-dataset generalization, and a cheaper model that keeps that generalization. The two research problems motivating these goals are stated in Chapter 1. Section 3.1 describes the DAGHAR benchmark, what the data actually look like, and how it is prepared. Section 3.2 presents the proposed generalization framework itself, and Section 3.3 extends it with the storage-optimization framework. Section 3.4 specifies the experimental protocol, Section 3.5 defines the evaluation metrics, and Section 3.6 closes the chapter with a preview of Chapter 4.

3.1 Dataset and Data Preparation

3.1.1 DAGHAR Benchmark

All experiments use the DAGHAR benchmark [1], a collection of six smartphone-based HAR datasets—KuHar, MotionSense, RealWorld-Thigh, RealWorld-Waist, UCI-HAR, and WISDM— assembled specifically to study domain adaptation and generalization. It was chosen because it is, to date, the benchmark that most directly targets the cross-dataset problem this thesis studies: rather than treating each dataset as a stand-alone task, DAGHAR standardizes all six onto a common format precisely so that they can be compared and combined, while still preserving the genuine

differences between them.

Each dataset is treated as a distinct *domain*: the six differ from one another in sensor placement, subject population, and data-collection protocol, which is exactly the kind of cross-dataset shift this thesis studies (Chapter 1). DAGHAR provides two versions of the data, a *baseline view* and a *standardized view*; this thesis uses the standardized view throughout, since it removes trivial format differences between datasets (units, sample rate, window length) while leaving the real domain gaps intact. Standardization converts every recording to a common unit and sample rate (20 Hz), removes gravity with a high-pass filter, and slices the signal into non-overlapping three-second windows. Every window therefore consists of six channels—tri-axial accelerometer (x, y, z) and tri-axial gyroscope (x, y, z)—each 60 timesteps long, and the prediction target is one of six activities: sit, stand, walk, stair-up, stair-down, and run.

Table 3.1 summarizes the six domains: their sensor placement, number of subjects, standardized-view sample sizes, and which activity classes each one actually contains. Two domains do not cover all six activities—UCI-HAR has no *run* class, and WISDM has neither *stair* class—which is a property of how those datasets were originally collected, not a modelling choice. This matters for evaluation: when one of these two domains is the held-out target, accuracy is necessarily computed only over the classes actually present, and per-class scores can be more volatile than for the full-coverage domains. This is why Section 3.5.1 reports macro-F1 alongside accuracy, and why this thesis treats KuHar and WISDM as the domains most likely to show volatile, small-sample behaviour when interpreting per-fold results later.

Table 3.1: The six DAGHAR domains used throughout this thesis. Sample counts are for the standardized view; “classes present” lists the activities available in each domain.

Domain	Placement	Subjects	Train / Val / Test	Classes present
KuHar	Waist bag	90	1386 / 426 / 144	{0, 1, 2, 3, 4, 5}
MotionSense	Pocket	24	3558 / 420 / 1062	{0, 1, 2, 3, 4, 5}
RealWorld-Thigh	Thigh	15	8400 / 1764 / 2628	{0, 1, 2, 3, 4, 5}
RealWorld-Waist	Waist	15	10332 / 1854 / 2592	{0, 1, 2, 3, 4, 5}
UCI-HAR	Waist	30	2420 / 340 / 690	{0, 1, 2, 3, 4} (no run)
WISDM	Pocket	51	8736 / 944 / 2596	{0, 1, 2, 5} (no stairs)

3.1.2 Exploratory Data Analysis (EDA)

Before designing the model, we examined how activities and domains are actually distributed in the raw signal space. Two findings from this analysis directly shaped the design decisions of the rest of this chapter, and both are numerical/statistical by nature, so this subsection is reported with the same rigor as any other data-description material even though later sections deliberately keep a lighter, more conceptual tone.

Fig. 3.1 shows a two-dimensional t-SNE embedding of the standardized windows, coloured by activity: raw time-series input produces heavily overlapping clusters, but three well-separated energy bands emerge once frequency content is considered—low-energy (sit, stand), mid-energy (walk, stairs), and high-energy (run). This pattern holds both within individual datasets and when all six are pooled together. It is the direct motivation for the hybrid time+frequency node feature introduced in Section 3.1.3: **if frequency content is what actually separates activities, a feature that only carries time-domain information is discarding useful signal.**

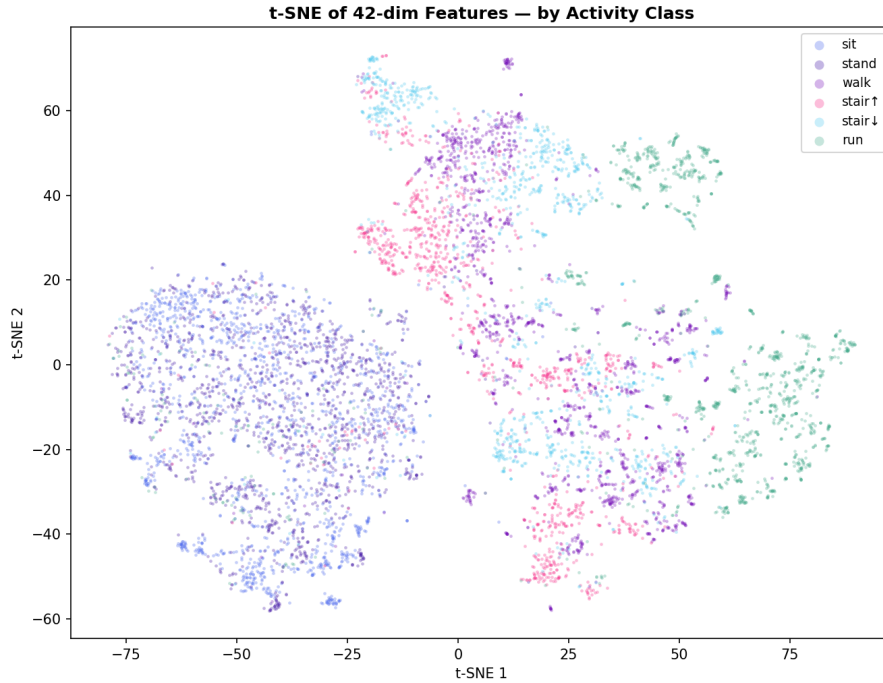


Figure 3.1: Activities separate into energy bands once frequency content is considered. A t-SNE embedding of standardized windows coloured by activity class shows the low-, mid-, and high-energy groupings that motivate frequency-aware node features.

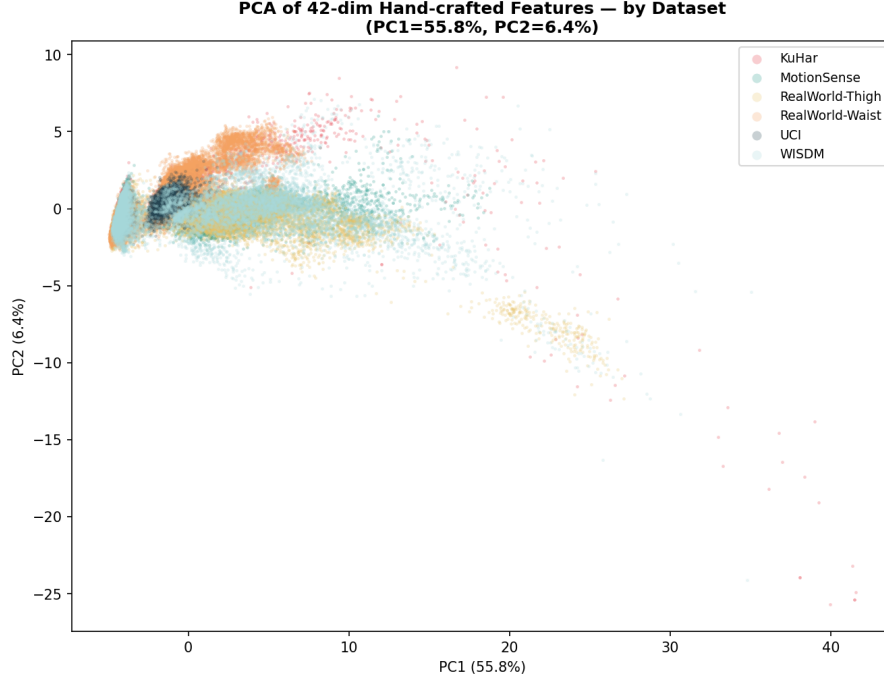


Figure 3.2: Domain shift is driven primarily by device placement. A projection of the windows coloured by source dataset shows overlap for static activities but placement-specific separation for dynamic ones.

Fig. 3.2 shows a projection of the same windows, this time coloured by source dataset rather than activity, and it shows the opposite pattern for static activities and dynamic ones: low-energy activities (sit, stand) occupy roughly the same region of feature space regardless of which dataset they came from, but dynamic activities (walk, stairs) form placement-specific clusters, since pocket, thigh, and waist placements produce systematically different motion signatures for the same activity. A maximum-mean-discrepancy (MMD) analysis across the six domains further identifies KuHar as the most dissimilar domain from the rest of the benchmark, despite sharing a broadly similar waist-level placement with UCI-HAR and RealWorld-Waist—consistent with it being, empirically, the hardest held-out target in Chapter 4. Together, these two observations indicate that **the shift this thesis studies is structural, rooted in how and where the sensor is worn and who is wearing it, rather than a superficial scaling difference between datasets, and they are the reason the proposed framework includes an explicit domain-invariance component, rather than treating it as an optional add-on.**

3.1.3 Data Preprocessing and Feature Preparation

The standardization pipeline itself—unit conversion, resampling to 20 Hz, gravity removal, and windowing into non-overlapping three-second segments—is inherited directly from the DAGHAR standardized view and is not a contribution of this thesis; it is cited rather than re-derived here. What follows describes the feature representation built on top of that pipeline, and the one normalization rule that keeps the whole protocol honest.

From time-domain to hybrid features. Earlier stages of this project considered only time-domain representations of the signal. Motivated by the separability finding of Section 3.1.2, frequency-domain features were subsequently investigated, and the framework in its final form uses a *hybrid* representation that combines both: each sensor channel’s feature is the standardized 60-step time window concatenated with its magnitude spectrum, obtained by a fast Fourier transform. Time-domain information captures how the signal changes moment to moment; frequency-domain information captures the underlying rhythm and energy of the motion, which Section 3.1.2 showed to be the more activity-discriminative view. The exact dimensionality of this feature and how the graph branch’s frequency view differs from the convolutional branch’s own frequency view are specified together with the rest of the architecture in Section 3.2.

This is also where an important nuance in the research work’s own history belongs. Combining time- and frequency-domain features was not, on its own, a reliable improvement: an earlier representation—one that placed individual timesteps, rather than sensor channels, on the graph’s nodes—saw hybrid features perform *worse* than frequency features alone. The hybrid representation only became a genuine improvement once it was paired with the sensor-as-node graph of Section 3.2.1. The lesson drawn from this is more general than either result on its own: *the effectiveness of a feature representation depends on the graph representation used to consume it*, not only on the information the feature itself carries. Section 3.2.2 states only the final, adopted design; this contrast is not repeated there.

Normalization and leakage control. Section 3.1.4 defines what counts as a “source” domain for a given evaluation fold; every normalization statistic used anywhere in this thesis—channel means and standard deviations, or the equivalent for the frequency component—is estimated exclusively on the training portion of those source domains, and then applied unchanged to that fold’s validation split and to the held-out target domain. No statistic is ever fit on target data, at any point. This is what keeps the cross-dataset evaluation of this thesis strictly fair in practice, not only in name: the target domain is unseen in every sense, including through its own first- and second-order statistics.

3.1.4 Cross-Dataset Evaluation Setting

Because the goal is generalization to a dataset the model has never encountered, this thesis evaluates every configuration with a leave-one-dataset-out (LODO) protocol. In one LODO experiment, five of the six datasets are used for model development—training and validation—while the sixth is held out entirely and used only for evaluation. This process is repeated so that each of the six datasets serves once as the unseen target, giving six independent train/test folds per configuration. A configuration’s headline result throughout this thesis is the average accuracy across all six folds, so that every domain contributes equally regardless of how many test samples it happens to have.

This subsection introduces LODO only at the level needed to understand the rest of this chapter; the exact procedure—how the source domains’ splits are used, and the rule that the target domain must never influence model selection—is specified formally in Section 3.4.

3.2 Proposed Generalization Framework

Figure 3.3 summarizes the proposed system in full. Windows from the six DAGHAR domains, prepared as described in Section 3.1, are represented by hybrid time- and frequency-domain features (Section 3.1.3); each sensor channel becomes a node in a small graph (Section 3.2.1), encoded by a per-modality CNN (Section 3.2.3) and passed through a GraphSAGE backbone (Section 3.2.4)

trained with a domain-adversarial objective (Section 3.2.5); the graph’s own edges are chosen adaptively, per window, by blending a physical prior with the data’s own correlation structure (Section 3.2.6); and the graph branch’s prediction is late-fused with an independently trained convolutional branch (Section 3.2.7) to produce the final activity prediction. The whole locked system is later subjected to the storage-optimization framework of Section 3.3. One note on ordering: this walkthrough follows the order in which the subsections below explain the components, which is not quite the order in which the system computes them—the adaptive graph of Section 3.2.6 is in fact constructed before and alongside the CNN encoding step, since it determines the topology over which the encoded features are later aggregated.

3.2.1 Sensor-as-Node Graph Representation

An early version of this project’s graph placed each *timestep* on a node. That representation produced weak cross-dataset performance and is kept in Chapter 4 only as a historical point of comparison. The research work instead investigated whether representing individual *sensor channels* as graph nodes would better capture relationships between different sensing channels and improve cross-dataset generalization—and this is the representation the final framework uses.

Concretely, each of the six sensor channels—accelerometer x, y, z and gyroscope x, y, z —becomes one node, so every graph has exactly six nodes, and the network reasons over relationships *between sensors* rather than between moments in time, as Fig. 3.4 illustrates. This change does not, by itself, raise accuracy; what it does is make every later component in this section possible, including the domain-adversarial training and the adaptive edge construction. It is also where a fixed set of nine sensor pairs, grouped by physical sensor-type adjacency—three accelerometer-only pairs, three gyroscope-only pairs, and three paired across the two sensor types—originates; this fixed grouping is used again, as a prior, in adaptive graph construction.

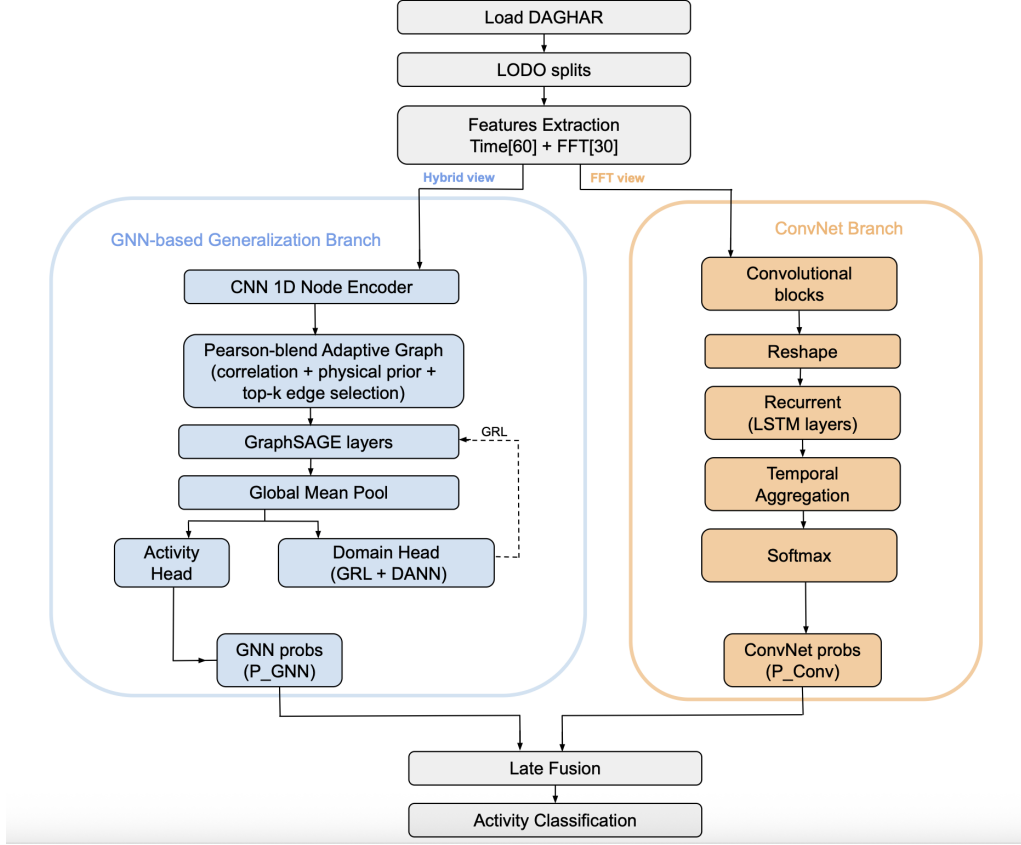


Figure 3.3: The proposed system, pursuing both research goals on one locked model. A shared feature-extraction step produces two views of each window: a hybrid time-plus-frequency view consumed by the graph branch, whose edges are chosen by a prior-anchored adaptive rule, and a frequency view consumed by the convolutional branch. The two branches’ class probabilities are combined by late fusion, and the resulting system is then the subject of the storage-optimization study. The dotted arrow denotes the training-only adversarial gradient reversed through the GRL; the domain head plays no part in inference.

3.2.2 Time-Domain and Frequency-Domain Features

Each of the six nodes needs its own feature vector, and the final framework builds this from both views of the signal available at each sensor channel. Time-domain information captures how the signal changes moment to moment; frequency-domain information captures the underlying rhythm and energy of the motion, which EDA showed to be the more activity-discriminative view for this data. The framework therefore uses a hybrid feature: the standardized 60-step time window, concatenated with its magnitude spectrum from a fast Fourier transform (30 frequency bins, the zero-frequency bin dropped), for 90 values per node in total. Both components are normalized

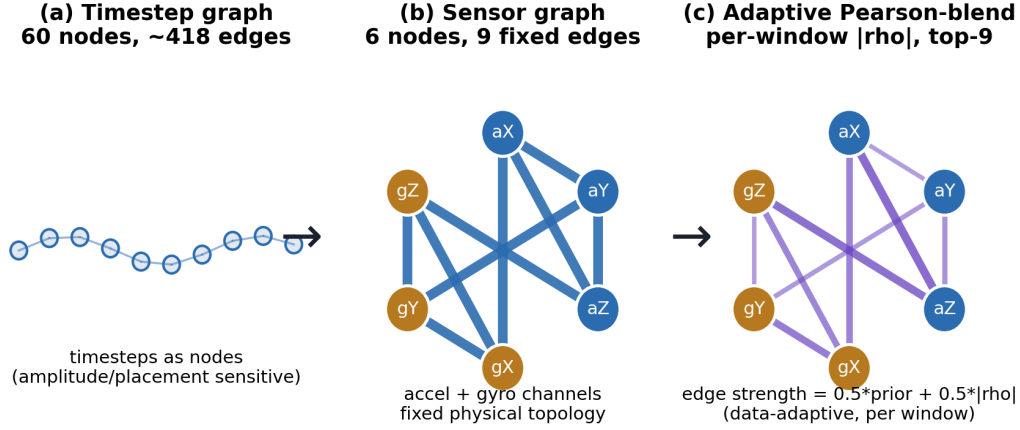


Figure 3.4: From timestep graphs to a sensor graph. Each of the six accelerometer and gyroscope channels becomes a node, and edges express inter-sensor relationships rather than temporal ones.

per channel using statistics fit only on the source-training data, following the discipline set out in Section 3.1.3.

This hybrid representation is only stated here in its final, adopted form.

3.2.3 CNN-Based Feature Encoding

Before any information is exchanged between nodes, each node’s 90-dimensional feature is first encoded independently by a small one-dimensional convolutional network. The purpose of this step is to let the model learn useful local patterns in each channel’s own signal—for example, characteristic shapes in the time series or characteristic peaks in its spectrum—before that channel’s information is combined with any other channel’s.

Weight sharing in this encoder follows sensor modality rather than being fully separate per node or fully shared across all six: one encoder is learned for the three accelerometer nodes, and a separate encoder is learned for the three gyroscope nodes, reflecting the physical grouping already present in the graph. Each encoder is a small stack: two one-dimensional convolutional layers (kernel size 5, channel widths 32 then 64, each followed by batch normalization and a ReLU nonlinearity), an average-pooling step that collapses the temporal axis, and a final linear projection, producing a 64-dimensional embedding per node. Before the message-passing step, each edge’s

type—accelerometer-only, gyroscope-only, or cross-modal, assigned during graph construction — is looked up in a small learned embedding table and added, at reduced strength, to the embedding of that edge’s source node. This gives the network access to coarse structural context even though the message-passing operator itself, described next, does not use edge weights.

3.2.4 GraphSAGE Backbone

The encoded node embeddings are passed through a two-layer GraphSAGE network, the graph branch’s backbone. Each layer combines a node’s own embedding with an average of its neighbours’ embeddings, so that information can flow between connected sensor channels; two such layers let information reach every node from every other node it is connected to, directly or through one intermediate node. GraphSAGE was adopted as the backbone after a broader comparison against alternative graph neural network architectures found it to be the most consistent performer once combined with the domain-adversarial training.

One property of GraphSAGE matters more than any other for the rest of this chapter: its neighbourhood aggregation uses only which nodes are connected, not any weight or strength assigned to a connection. This is what makes the adaptive graph construction act purely as a *topology selector*—it decides *which* of the fifteen possible sensor pairs are connected in a given window, and the backbone treats every connection it is given identically, regardless of how strongly correlated the two channels happened to be.

After message passing, the six node embeddings are combined by averaging them into one 64-dimensional graph embedding.

3.2.5 Domain-Adversarial Training

Domain-adversarial training was incorporated to encourage the model to learn activity-relevant representations that are less dependent on source-dataset-specific characteristics.

The mechanism follows the standard domain-adversarial approach of Ganin et al. [20]. In addition to the activity classifier, the graph embedding also feeds a second, small network whose job is

to guess which of the five *source* domains a training window came from—never the held-out target, since guessing the target’s identity would require having seen it. Between the graph embedding and this domain-guessing network sits a gradient-reversal layer (GRL): it passes information through unchanged in the forward direction, but on the backward pass, it flips the sign of the gradient before it reaches the shared encoder. The practical effect is that improving domain classification is actively discouraged in the shared representation, rather than rewarded—the encoder is pushed to represent activities in a way that makes the five source domains hard to tell apart from one another.

The strength of this adversarial pressure, λ , is not constant; it ramps up smoothly over the course of training following

$$\lambda(p) = \frac{2}{1 + e^{-10p}} - 1, \quad p \in [0, 1], \quad (3.1)$$

where p is training progress from 0 to 1, so that the model is not asked to satisfy a strong invariance constraint before it has learned to classify activities at all. The total training loss combines the two objectives,

$$\mathcal{L} = \text{CE}(\text{activity}) + \lambda \cdot \text{CE}(\text{domain}), \quad (3.2)$$

where CE denotes cross-entropy loss. This form of domain-adversarial training is deliberately moderate in strength: Chapter 4 shows that it helps modestly and is kept in the final framework, while heavier invariance mechanisms evaluated on top of it consistently make results worse, a pattern Chapter 4 also explains mechanistically.

3.2.6 Adaptive Pearson-Blend Graph Construction

This is the central methodological contribution of the thesis, and the component that most directly targets the thesis’s first research problem, cross-dataset generalization (Chapter 1): how should the graph’s edges—which sensor channels are considered related to one another—actually be chosen?

There are two different kinds of knowledge available for this decision, and neither is enough on its own. The first is *physical* knowledge: certain sensor pairs are related simply because of how

the device and body work—the three sensor-type groupings already introduced in Section 3.2.1. This kind of relationship is stable across datasets by construction, but it never changes, no matter what the person is actually doing. The second is *data-driven* knowledge: within a given window, two channels may move together closely (for example, during a rhythmic activity like walking) or not at all (for example, while sitting still), and which pairs move together can depend on the activity and can vary window to window. A graph built only from the physical prior ignores this useful, activity-dependent signal; a graph built only from the data risks overfitting to correlations that happen to hold on the training datasets but do not transfer to an unseen one. The proposed construction combines both: for every window, it rebuilds the graph’s edges using a score that blends the observed Pearson correlation between channels with the fixed physical prior, and the configuration selected for the final framework uses an equal, 50/50 blend of the two, keeping the nine highest-scoring sensor pairs as edges.

For each window, and for every candidate pair of sensor channels (i, j) —there are $\binom{6}{2} = 15$ such pairs among the six channels—the absolute Pearson correlation $|\rho_{ij}|$ is computed on the *raw* 60-step time signal, not the 90-dimensional hybrid feature: correlation is unaffected by the per-channel normalization applied to that feature, and the frequency bins are not a meaningful space in which to correlate two channels directly. Each pair is then scored by blending this data-driven term with the physical-prior indicator:

$$s_{ij} = \alpha |\rho_{ij}| + (1 - \alpha) \pi_{ij}, \quad \alpha \in [0, 1], \quad (3.3)$$

where $\pi_{ij} \in \{0, 1\}$ indicates whether pair (i, j) belongs to the physical-prior grouping and α controls the balance between the two terms. The nine highest-scoring pairs are kept as the window’s edges, made bidirectional, and tagged with their type (accelerometer-only, gyroscope-only, or cross-modal) for the embedding lookup.

The adopted configuration, as stated above, is $\alpha = 0.5$ with the top 9 of the 15 candidate pairs retained. This is a deliberate design choice: comparing this configuration against others on source-

validation data, following the protocol set out in Section 3.4, shows it to be the clear winner, and shows that the purely data-driven extreme, $\alpha = 1$, actually performs worse on at least one held-out fold—the physical prior is therefore an essential ingredient of the method, not a fallback that a strong enough data-driven signal would render unnecessary. Chapter 4 reports the full comparison and the mechanism behind this result.

Two properties of this construction are worth stating plainly. First, a window’s adjacency is computed strictly from that window’s own signal, so there is no leakage across windows, folds, or domains. Second, because the GraphSAGE backbone of Section 3.2.4 ignores edge weights entirely, the score s_{ij} never influences the network directly—it only decides which edges exist. The central empirical claim of this thesis, developed across Chapter 4, is that this choice of *which edges to build*, more than the choice of invariance method or model size, is what governs how well the system generalizes to an unseen dataset.

Figure 3.5 assembles every component introduced in this section into a single diagram of the complete graph branch: the per-modality convolutional encoder along the top path, the adaptive Pearson-blend edge construction along the bottom path, the two resulting streams meeting at the GraphSAGE layers, and the pooled graph embedding finally splitting into the activity head and, through the gradient-reversal layer, the domain-adversarial head.

3.2.7 ConvNet-Based Complementary Branch and Late Fusion

The graph branch described above captures relationships *among* sensor channels through its graph representation. A second, independently trained branch provides a complementary view of the same window, and the two branches’ predictions are combined through late fusion.

This second branch is a faithful reimplement of DeepConvLSTM [2]. Chapter 2 discusses why convolutional-recurrent architectures are the model family shown by the DAGHAR benchmark to be most robust under cross-dataset shift, which is the reason it was chosen for this complementary branch. Rather than the hybrid feature, this branch consumes a frequency view of the window on its own—a log-magnitude FFT that keeps the zero-frequency bin the graph branch

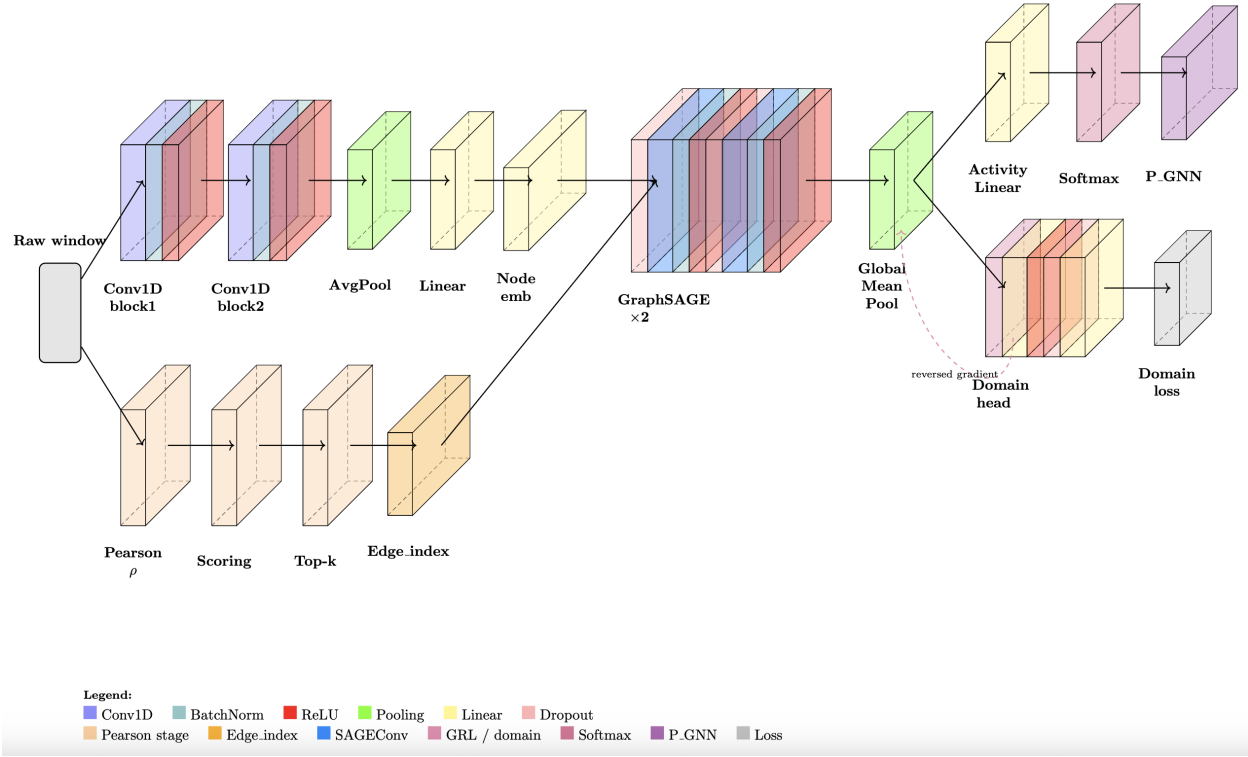


Figure 3.5: The complete graph branch, from raw window to both output heads. The convolutional node encoder (top path) and the adaptive Pearson-blend edge construction (bottom path) feed the two-layer GraphSAGE backbone; the pooled graph embedding is read by an activity classifier and, through the gradient-reversal layer, a domain discriminator trained adversarially against it.

drops—normalized using the same source-only statistics discipline as the rest of this chapter. The two branches therefore deliberately see the signal differently: this is part of why combining them helps, since the branches tend not to make the same mistakes. Architecturally, four convolutional layers mix information across time while keeping each sensor channel’s own feature maps separate, followed by two stacked LSTM layers that read the resulting sequence, and a final linear classifier on the last timestep’s hidden state. Figure 3.6 lays out this pipeline block by block.

This branch is trained separately from the graph branch and then *frozen*: its predicted class probabilities are computed once and reused, unchanged, for every configuration that fuses against them. The two branches are combined by simple probability averaging,

$$P = w \cdot P_{\text{gnn}} + (1 - w) \cdot P_{\text{conv}}, \quad w \in [0, 1], \quad (3.4)$$

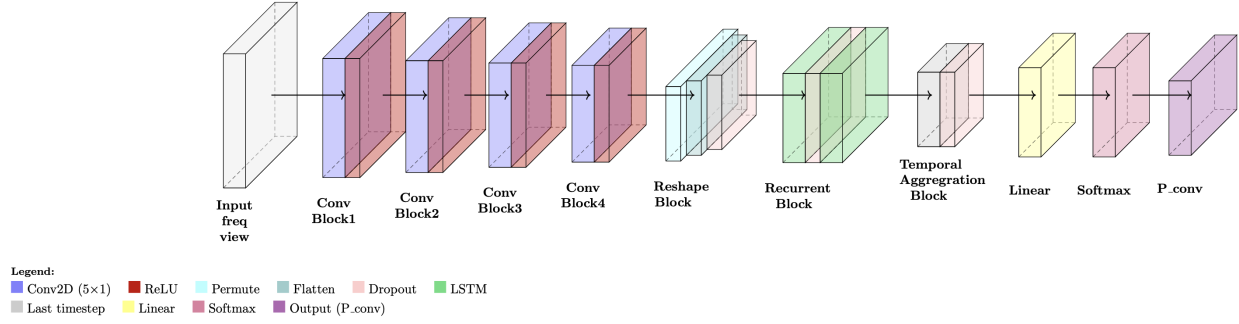


Figure 3.6: The convolutional complementary branch, a DeepConvLSTM reimplementation. Four convolutional blocks read the window’s frequency view, a recurrent block of stacked LSTM layers aggregates the resulting sequence, and a linear classifier on the final aggregated state produces this branch’s class probabilities.

with the blend weight w swept over a fixed grid and set to whichever value maximizes mean accuracy across the six leave-one-dataset-out folds; the adopted value favours the graph branch ($w \approx 0.6$). This is a deliberate, and the only, departure from the target-blind selection discipline of Section 3.4.2: because the two branches are frozen and only a single global blend weight is being set, rather than an architecture or a hyperparameter being trained, the weight is tuned directly against the aggregate leave-one-dataset-out result rather than against source-validation performance. Every other decision in this thesis, including which graph construction, which backbone, and which compression operator to adopt, is still made without ever consulting the target domain; this fusion weight alone is not, and it is flagged here as a limitation rather than left implicit. Because the convolutional branch is frozen rather than trained jointly with the graph branch, the fused results reported throughout this thesis should also be read as a conservative lower bound on what a jointly trained system could achieve—both limitations are returned to in Chapter 5.

Training Configuration Summary

Table 3.2 consolidates the regularization and optimization settings of the graph branch, held fixed across every configuration in Chapter 4 unless a specific ablation states otherwise. The convolutional branch’s own training configuration (frozen and reused from a single training run) is summarized in Section 3.2.7 above.

Table 3.2: Regularization and training configuration for the graph branch. These settings are frozen across the model-family and adaptive-graph ablations of Chapter 4.

Mechanism	Setting
Dropout	$p = 0.2$ before each GraphSAGE layer; $p = 0.2$ in the domain discriminator
Batch normalization	after each convolutional layer (encoder) and each GraphSAGE layer
Weight decay (L2)	5×10^{-4} (Adam)
Gradient clipping	max-norm 1.0
Gradient reversal	Ganin schedule, Equation 3.1, ramping $\lambda : 0 \rightarrow 1$
Learning-rate schedule	Adam, lr = 10^{-3} , cosine annealing to $\eta_{\min} = 10^{-5}$ over 100 epochs
Early stopping	patience 20 on source-validation accuracy; best checkpoint restored
Loss	$\mathcal{L} = \text{CE}(\text{activity}) + \lambda \cdot \text{CE}(\text{domain})$
Batch size / epochs	64 / up to 100
Model selection	best source-validation accuracy (target never observed)

3.3 Storage Optimization Framework

Having obtained a single locked model that generalizes across unseen datasets, this section asks how much of it can be removed before that ability breaks. Compression is treated not as one technique but as a family of operators acting on five distinct axes of the same locked system—architectural capacity, numerical precision, graph edges, graph nodes, and temporal window length—so that the cost of generalization can be characterized along each axis separately rather than folded into a single number. Every operator below is evaluated against the adaptive-graph configuration at a matched storage or edge budget, so that the effect of a *particular* compression choice can be told apart from the effect of the budget itself: a naive operator and a principled operator can reach the same model size by very different routes, and only comparing them at equal budget reveals which one actually matters. Chapter 4 reports the trade-off curves and the joint accuracy-and-size metric, used to rank operating points on a single scale; this section defines what each operator does. Figure 3.7 summarizes the general structure of this framework: the five operator families divide into two groups according to *when* each one acts on the locked model—*retrain-based* operators (capacity reduction, node coarsening, and adaptive windowing) change the model or its input specification and therefore retrain it from scratch under the identical protocol, whereas *inference-based* operators (post-hoc compression and edge-policy substitution) act on the already-

trained model with no retraining or fine-tuning—and every resulting operating point feeds a single storage–generalization trade-off analysis. Throughout, the operators act on the graph branch only; the convolutional branch remains frozen and identical across every operating point.

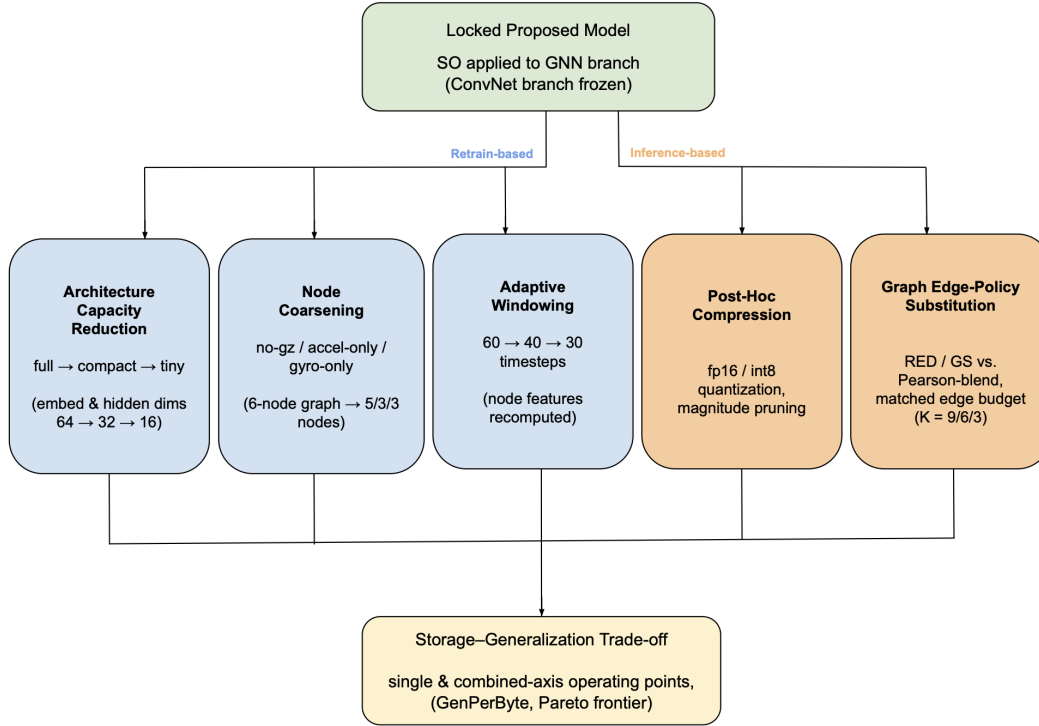


Figure 3.7: The storage-optimization framework. Five operator families act on the locked model of Section 3.2, divided by when they act: retrain-based operators (capacity reduction, node coarsening, adaptive windowing) modify the model or its input and are retrained from scratch under the identical protocol, while inference-based operators (post-hoc compression, edge-policy substitution) act on the already-trained model without retraining. All resulting single- and combined-axis operating points feed one storage–generalization trade-off analysis.

3.3.1 Architecture Capacity Reduction

The most direct lever on model size is simply to make the model smaller before training it at all. This axis retrains the graph branch at reduced embedding and hidden dimensionality—below the 64-dimensional default—producing a family of smaller models trained from scratch under the identical protocol and selection discipline as the full model, following the experimental protocol. Unlike every other operator in this section, capacity reduction changes the model *before* training

rather than acting on already-trained weights, and it is the primary lever on total parameter count: the other axes below all act on a model whose capacity has already been fixed.

3.3.2 Post-Hoc Model Compression

Two further operators act on already-trained weights, with no additional fine-tuning, so that the reported cost of compression is not flattered by extra training:

- **Precision reduction.** Half-precision (fp16) and dynamic int8 quantization lower the number of bytes used per parameter, without changing the graph or the architecture.
- **Magnitude pruning.** Zeroing out the smallest-magnitude weights across the network at a chosen sparsity level, leaving the architecture and parameter count nominally unchanged but reducing how many non-zero values must actually be stored.

Because neither operator is followed by fine-tuning, the resulting numbers in Chapter 4 should be read as a lower bound on what compression-aware retraining could achieve—a limitation discussed further in Chapter 5.

3.3.3 Graph-Level Optimization

This axis asks a sharper version of the budget-versus-structure question above: at the *same* edge budget as the adaptive graph, does the particular choice of which edges to keep matter, or would any topology of the same size do about as well?

Two classic, non-adaptive baselines are compared against the Pearson-blend construction at a matched number of edges. The first, random edge deletion, simply keeps a random subset of the fifteen candidate sensor pairs, independent of both the data and the physical prior—the naive control that any principled edge-selection rule must outperform to justify its added complexity. The second, graph sparsification, keeps edges by a fixed, deterministic priority order (for instance, favouring same-modality pairs over cross-modal ones, or the reverse) rather than at random or per-window—structured, hand-designed edge selection that stays fixed for every window and every

domain, unlike the adaptive construction’s per-window, data-driven choice. Because the adaptive graph is computed fresh per window rather than fixed at training time, these alternative policies can be substituted into an already-trained checkpoint at inference time, with no retraining at all, which is what isolates edge-selection policy as the single variable under comparison: the trained weights, and everything else about the model, are held identical across every policy compared at a given budget.

This subsection also covers a complementary question: how sensitive is the adaptive graph to its own edge budget? Independently retrained configurations at several edge counts are compared as a separate diagnostic, reported alongside the adaptive graph’s own construction in Chapter 4, to show whether a smaller budget costs accuracy on its own, apart from any effect of which edges are chosen. Comparing budget and policy this way isolates whether the adaptive graph’s contribution comes from having *any* principled, non-random structure, or specifically from being data-driven and computed per window; Chapter 4 reports the comparison and a mechanistic explanation for the result.

3.3.4 Input-Level Optimization

Two further operators act on the input itself rather than on the graph’s edges. Node coarsening removes whole sensor-channel nodes from the graph—for example dropping the gyroscope *z*-axis to obtain a five-node graph, or keeping only the accelerometer or only the gyroscope channels—which targets the node axis of storage directly and, as a side effect, probes which sensor modalities actually carry signal that transfers across domains. Adaptive windowing instead shortens the temporal window itself, from the standard 60 timesteps down to 40 or 30, recomputing the hybrid node features on the shorter signal—trading temporal resolution for storage and inference compute.

3.3.5 Storage–Generalization Trade-off

Because every operator above is compared against the locked model at a matched budget, the results in Chapter 4 can be read together as a single trade-off surface rather than as five separate

experiments: for a given storage budget, which axis gives up the least generalization to reach it, and does combining axes compose additively or interact? Ranking operating points that differ in both accuracy and size requires a metric that captures both at once, rather than favouring the single most accurate configuration regardless of size; this generalization-aware metric is defined formally in Section 3.5. Table 3.3 summarizes all five axes and the operators that act on each.

Table 3.3: The storage-optimization operators evaluated on the locked model of Section 3.2, grouped by the axis of the model each one acts on.

Axis	Operator(s)	Acts on
Architectural capacity	Capacity reduction (§3.3.1)	Model, before training
Numerical precision	fp16 / int8 quantization, pruning (§3.3.2)	Trained weights, post-hoc
Graph edges	Random deletion, sparsification, Pearson (§3.3.3)	Graph topology
Graph nodes	Node coarsening (§3.3.4)	Input channels
Temporal window	Adaptive windowing (§3.3.4)	Input length

3.4 Experimental Protocol

This section specifies exactly how models are trained, evaluated, and selected throughout the empirical studies. It is deliberately procedural rather than mathematical: the ideas themselves (evaluate on an unseen dataset; don’t peek at that dataset while choosing a configuration; check that a result isn’t a fluke of one random seed) are simple, and stating them as plain rules keeps them easy to apply consistently across every result in Chapter 4.

3.4.1 Leave-One-Dataset-Out Evaluation

Section 3.1.4 introduced the leave-one-dataset-out (LODO) idea at a conceptual level: five datasets for development, the sixth held out for testing, repeated six times. Procedurally, for each of the six folds, the model is trained using only the pooled training splits of the five source domains; each source domain’s own validation split—never the target’s—is used for early stopping and for the model-selection decisions of Section 3.4.2; and the held-out target domain’s test split is used exactly once, after training is complete, to compute that fold’s accuracy. No part of training or

selection ever observes the target domain, which is what makes the six folds comparable to one another as six independent instances of the same generalization problem, rather than six separately tuned models that happen to share an architecture.

3.4.2 Model Selection and Validation

The rule that matters most in this thesis is simple to state: *the unseen target domain is never used to choose a model, a hyperparameter, or a configuration*. Every comparison behind a result in Chapter 4—which backbone, which graph construction, which invariance method, which compression operator—is decided using only source-validation performance, never target performance. The one narrow exception is the late-fusion blend weight discussed in Section 3.2.7, which combines two already-trained, frozen branches rather than selecting among them, and is tuned directly against the leave-one-dataset-out result; it is flagged there explicitly rather than folded into this rule.

In practice, every other decision happens in two stages. First, candidate configurations for a given decision (for example, the graph’s edge count and blend parameter, or the edge-selection policies) are compared using only source-validation accuracy, and the best-performing candidate is carried forward as a single winner—this screening stage typically uses a single seed, purely to keep the search affordable. Second, that candidate winner alone is trained and evaluated under the full LODO protocol across multiple seeds, and it is this multi-seed result that Chapter 4 reports.

A candidate is only adopted in place of the reigning configuration if it clears a straightforward bar at this second stage: its mean accuracy across the six folds must improve by at least one percentage point, and no individual fold may drop by more than two percentage points relative to the reigning configuration. A candidate that misses either condition is rejected, and the reigning configuration remains the standard for every later stage of the thesis. This bar, applied the same way every time, is why several plausible-looking ideas in Chapter 4 are ultimately reported as not adopted despite showing a positive average effect—a positive mean alone is not sufficient if it comes at the cost of a badly regressed fold.

3.4.3 Multi-Seed Validation

With only six LODO folds, seed-to-seed variation from random initialization and training stochasticity can be large relative to the effect sizes under study. Every configuration reported in Chapter 4 as a final result is therefore trained and evaluated independently across multiple random seeds, and we report the seed mean and standard deviation of the LODO accuracy rather than a single run. The thesis’s central results, most importantly the adaptive graph construction of Chapter 4, are confirmed across five seeds; a small number of later results in Chapter 4’s final architecture search were confirmed across three seeds rather than five before being set aside as negative results, a difference in how many seeds were spent confirming an already-rejected candidate rather than in the underlying recipe, and Chapter 4 states the seed count explicitly wherever it differs from five. This is also why the screening stage of Section 3.4.2 deliberately uses fewer seeds still, typically one: multi-seed confirmation is reserved for the one candidate that has already shown promise, rather than spent on the full grid of candidates.

3.5 Evaluation Metrics

Because the thesis pursues two distinct goals—cross-dataset performance and storage efficiency—its metrics are grouped accordingly. Primary metrics quantify how well a model generalizes; secondary metrics quantify what that generalizing model costs to store and run; and Section 3.5.3 specifies how differences in either are tested for reliability given the small number of LODO folds. All metrics are computed on the held-out target domain and, unless stated otherwise, aggregated across the six LODO folds and the seeds used for that result, per the seed-count discussion of Section 3.4.3.

3.5.1 Classification Performance

3.5.1.1 Accuracy

The headline figure throughout this thesis is the average accuracy across the six LODO folds, with every domain weighted equally regardless of its test-set size.

3.5.1.2 Macro-F1

The F1 score averaged over the activity classes actually present in the target domain, as defined in Section 3.1.1, so every present class contributes equally regardless of its frequency. This is the primary imbalance-aware measure, and it matters given the partial class coverage of UCI-HAR and WISDM.

3.5.1.3 Precision and Recall

Macro-averaged precision and recall are reported alongside F1 to expose a configuration’s error profile—whether it errs mainly through false positives or missed detections—which a single aggregate F1 figure can mask.

3.5.1.4 Confusion Matrices

Per-fold confusion matrices are used in Chapter 4 as a qualitative diagnostic, to identify which activities are most often confused under a given domain shift, complementing the aggregate metrics above.

3.5.2 Storage and Efficiency

Because the convolutional branch is frozen and identical across every operating point in the storage study, the metrics below are computed on the graph branch, which is where every operator of Section 3.3 acts.

3.5.2.1 Parameter Count and Deploy Size

The number of trainable parameters and the serialized model size in bytes (the deploy size). Together, these give the basic storage footprint of a configuration.

3.5.2.2 Generalization-per-Byte

Ranking operating points that differ in *both* accuracy and size requires a metric that captures both at once. We define generalization-per-byte as

$$\text{GenPerByte} = \frac{\text{mean LODO accuracy}}{\text{deploy bytes}}, \quad (3.5)$$

the accuracy achieved per unit of stored model. Unlike a raw parameter count, GenPerByte rewards a model for staying accurate *and* small, so a slightly less accurate but far smaller model can be recognized as the better deployment choice rather than always losing to the single most accurate configuration regardless of size.

3.5.2.3 Compression Factor

The ratio of a reference model’s deploy size to that of a compressed operating point, expressing how many times smaller a configuration is (for example, a $6\times$ smaller model). This makes the storage saving of each operator in Section 3.3 directly comparable to the others.

3.5.2.4 Inference Latency

The mean CPU time per window, in milliseconds, measured on a fixed sample. Latency complements the storage metrics with a runtime-cost view, since a smaller model is not always a faster one.

3.5.3 Statistical Analysis

The seed mean and standard deviation of Section 3.4.3 is the primary way this thesis reports variability. For specific pairwise contrasts singled out in Chapter 4—for example, the adaptive graph against its static baseline, or Pearson-based edge selection against random deletion at matched budget—we additionally apply a paired Wilcoxon signed-rank test across the six LODO folds, pairing each fold’s result under the two configurations being compared. Six folds gives limited statistical power, so p -values are reported as a secondary, corroborating signal; the primary evidence for any claim in this thesis is the effect size itself—the seed-mean percentage-point difference and, where relevant, its consistency across every individual fold—not statistical significance taken alone.

3.6 Experimental Roadmap

This section previews the empirical narrative of Chapter 4 in outline form, without yet reporting a numeric result. Table 3.4 lists every stage of the experimental programme retained in the thesis narrative, grouped by the six chronological arcs used to organize Chapter 4, together with a one-line description of what each stage changed and its outcome. Internal project phase labels are not used anywhere in the thesis text.

Table 3.4: Roadmap of the experimental programme retained in this thesis, grouped by the chronological arcs of Chapter 4.

Arc	Stage	Outcome
Arc I	Single-dataset storage-optimization baseline	Established SO probes on a single dataset
	DAGHAR adoption	Same pipeline validated as a cross-dataset benchmark

Arc	Stage	Outcome
	LODO reframing as domain generalization	Accuracy cliff relative to within-dataset performance; motivates the rest of the thesis
Arc II	Frequency-domain node features	Partial recovery from the Arc I cliff
	Hybrid time+frequency features, timestep graph	Negative —worse than frequency features alone
	Sensor-as-node graph rewrite	Structural change; origin of the physical-prior topology (Section 3.2.1)
	DANN adoption and backbone convergence	Adopted ; GraphSAGE emerges as the consistent backbone
	Configuration sweep	Negative —capacity/configuration is not the lever
	Recipe finalization	Hybrid features finally help, on the sensor graph; the control configuration for later comparisons is locked
Arc III	Feature/normalization variants	Negative (reported as a compact comparative treatment)
	Convolutional (DeepConvLSTM) late fusion	Adopted (Section 3.2.7)
	Inference/training-time adaptation	Negative

Arc	Stage	Outcome
	Transductive domain-adaptation family	Negative —motivates rejecting target-aware DA in favour of strict DG
Arc IV	Adaptive Pearson-blend graph construction	Main accuracy contribution (Section 3.2.6)
Arc V	Weight-aware GNN backbone Pseudo-labelling	Negative Negative ; closes the domain-adaptation-family exploration
Arc VI	Storage-optimization framework applied to the locked model	Main storage contribution —the compression-generalization trade-off, GenPerByte, and the gradient-conflict diagnostic (Section 3.3)

3.7 Summary

Section 3.1 described the DAGHAR benchmark, the exploratory findings on separability and domain shift that motivate the framework’s design, the data-preparation pipeline built on top of DAGHAR’s own standardization, and the cross-dataset evaluation setting at a conceptual level. Section 3.2 presented the proposed framework itself—a sensor-as-node graph encoded by a per-modality CNN and a GraphSAGE backbone, trained with domain-adversarial regularization, whose edges are chosen per window by a prior-anchored adaptive rule, and late-fused with an independently trained convolutional branch. Section 3.3 defined the storage-optimization framework

that acts on this same locked model along five axes of compression. Section 3.4 specified the experimental protocol—the LODO procedure, the rule that the target domain is never used for model selection, and the multi-seed validation strategy. Section 3.5 defined the classification and storage-efficiency metrics, including the generalization-per-byte measure, used to report results. Section 3.6 previewed how these components come together across the six chronological arcs of the empirical programme. Chapter 4 now applies this methodology: it establishes the headline accuracy result, justifies each retained component by ablation, presents the evidence that the cross-dataset bottleneck is structural rather than a matter of capacity or target access, and reports the storage-optimization trade-off study in full.

Chapter 4

Empirical Studies

Chapter 3 proposed a framework for cross-dataset generalization and a storage-optimization treatment of that framework, evaluated under a strict leave-one-dataset-out protocol. This chapter presents the evidence for that framework: not as a single experiment, but as a body of work that arrived at it through a long sequence of trials, several of which failed instructively before the final system converged. Section 4.1 lays out that sequence as a whole. The remaining sections then walk through it arc by arc, in the order the work actually happened.

4.1 Overview of the Experimental Arc

The system proposed in Chapter 3 did not begin as a domain-generalization framework at all. It began as a storage-optimization study on a single, familiar dataset, and only became a cross-dataset generalization project once an early result made clear that storage was not the binding constraint on deployability—generalization was. From that turn onward, the work proceeds through five further arcs: a broad search over feature representations and graph structure, a sequence of domain-adaptation and robustness attempts (most of which failed), the adaptive graph construction that produces the thesis’s best model, a final architecture search that also failed, and a return to storage optimization now applied to a model worth compressing. Figure 4.1 plots the resulting accuracy trajectory across this arc, and Table 4.1 previews every stage with its headline metric.

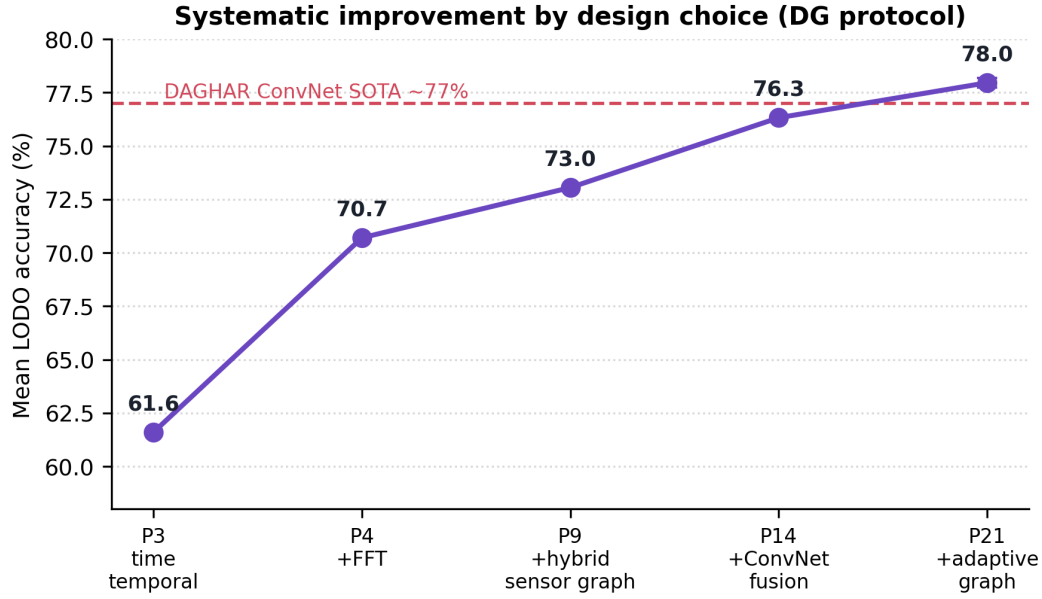


Figure 4.1: Leave-one-dataset-out accuracy across the experimental arc. The early collapse on first exposure to the six-dataset benchmark, the long recovery through representation and robustness work, and the final gain from the adaptive graph are the three inflection points that organize the rest of this chapter.

Two conventions carry over from Chapter 3 and hold for the rest of this chapter. First, every accuracy figure is a leave-one-dataset-out mean across the six DAGHAR datasets unless stated otherwise, and every model change is evaluated against the keep/revert rule described in Section 3.4.2: adopted only if it improves the mean by at least one percentage point with no individual target regressing by more than two. Second, results that did not clear that bar are not omitted; they are reported compactly, because the fact that a plausible-sounding idea failed is itself part of the evidence this thesis relies on—particularly in Section 4.4, where four of five attempts to adapt to the target domain more aggressively make the case for the stricter, target-blind protocol this thesis ultimately adopts.

Table 4.1: Detailed roadmap of the experimental arc. Each row is one stage of the work described in this chapter; accuracy is the leave-one-dataset-out mean across the six DAGHAR datasets once the benchmark was adopted (Section 4.2), and the within-dataset mean beforehand.

Arc	Stage	Accuracy	Verdict
Arc I	Single-dataset storage-optimization baseline	91.2%	compression free
Arc I	DAGHAR-standardized adoption	91.4%	rankings not portable
Arc I	LODO reframing (the accuracy cliff)	61.6%	motivates Arcs II–VI
Arc II	Frequency features, timestep graph	70.7%	recovers some ground
Arc II	Hybrid features, timestep graph	67.1%	negative (too early)
Arc II	Sensor-as-node graph rewrite	68.5%	topology matters
Arc II	Domain-adversarial training adopted	70.7%	helps GraphSAGE only
Arc II	Recipe finalized (control, “Arm A”)	73.1%	baseline for Arcs III–V
Arc III	Feature/normalization/alignment variants	59–73%	all revert (single-model ceiling)
Arc III	Convolutional late fusion	76.3%	kept
Arc III	Test-time and training-time adaptation	75–77%	negative (both revert)
Arc III	Transductive domain-adaptation family	76–77%	rejected; closes the DA line
Arc IV	Adaptive Pearson-blend graph	77.96%	best model (kept)
Arc V	Weight-aware GNN / pseudo-labeling	77.4–77.9%	both revert
Arc VI	Storage optimization revisited	77.97%	near-free at matched budget

4.2 Arc I: From Storage Optimization to Domain Generalization

This thesis did not set out to study domain generalization. It set out to ask a narrower question: how much can a graph-based HAR model be compressed before its accuracy suffers? That question was first answered on a single, familiar dataset, then re-answered once the same pipeline was ported onto the standardized six-dataset benchmark, and finally overturned once the evaluation protocol was changed from within-dataset cross-validation to leave-one-dataset-out. That overturn is the origin of everything that follows in this chapter.

4.2.1 Single-Dataset Storage-Optimization Baseline

The starting point was the UCI-HAR dataset in its original form, evaluated with five-fold cross-validation within the dataset: the same participants and collection protocol appear in both training and test folds, so this setting asks only whether a model fits the data it was built on, not whether it generalizes beyond it. Two graph backbones were compared, a graph convolutional network (GCN) and a graph attention network (GAT), each representing a window of sensor data as a graph over *timesteps* rather than sensor channels, an early representation later abandoned in favor of the sensor-as-node graph of Section 3.2.1. Both backbones reached similar accuracy on their own (GCN $91.16\% \pm 1.45$, GAT $91.24\% \pm 0.48$), giving a strong within-dataset reference point against which to test compression.

Four storage-optimization operators were then applied on top of each backbone: random edge deletion, spectral graph sparsification, node coarsening, and adaptive windowing, the same operator family later formalized in Section 3.3. The headline result was node coarsening on the attention backbone (GAT with node coarsening), which reached 91.20%, statistically indistinguishable from the uncompressed GAT baseline, while removing 63.1% of the graph’s edges and cutting real deployment size by roughly 54%. Two smaller findings from this first pass carried forward into every later storage-optimization stage. First, node-level reduction saved far more real stor-

age than edge-level reduction: coarsening nodes cut deployment size by about 54% because node features, not edges, dominate the model’s storage footprint, whereas edge-deletion techniques that removed a comparable fraction of edges cut real size by under 10%. Second, the two backbones absorbed compression differently—the attention backbone tolerated it with almost no accuracy loss, while the convolutional backbone lost more, a pattern that reappears when the same operators are revisited on the final system in Section 4.7. No individual comparison in this stage survived a multiple-comparison correction, a limitation of the five-fold sample size that recurs throughout this chapter and is addressed directly in Section 3.5.3.

4.2.2 Adoption of the DAGHAR Benchmark

The same pipeline was then ported onto the DAGHAR-standardized version of the UCI-HAR data: the same participants, but resampled to the benchmark’s fixed 20 Hz, six-channel, sixty-timestep window format described in Section 3.1, and still evaluated with within-dataset cross-validation rather than leave-one-dataset-out. This stage was a bridge, not yet the full domain-generalization setting, but it already produced the first sign that the single-dataset findings above would not simply carry over intact.

The backbone ranking reversed: the convolutional backbone now reached $90.26\% \pm 1.37$ against the attention backbone’s $86.43\% \pm 1.38$, a roughly five-point drop for the backbone that had been the stronger of the two in Section 4.2.1. The best result overall was the convolutional backbone with spectral sparsification, at 91.39%, a 1.13-point improvement over its own uncompressed baseline achieved while still removing edges—compression acting as a mild regularizer rather than a cost, a theme that resurfaces with the node-coarsened model in Section 4.2.3. Adaptive windowing, by contrast, became clearly harmful here for the first time in the project: it discards the parts of a window with the least short-term variance, which reliably biases against static postures, and both backbones lost significant accuracy under it (Bonferroni-corrected $p = 0.011$ for the convolutional backbone, $p = 0.014$ for the attention backbone). The standing class was hit hardest, with the convolutional backbone’s standing accuracy collapsing from 80.1% to 49.1% once

adaptive windowing was applied. Taken together, this stage’s lesson was that neither the backbone ranking nor the storage-technique ranking from a single dataset could be assumed to hold once the data changed, even before the evaluation protocol itself changed.

4.2.3 LODO Reframing as a Domain Generalization Problem

The pipeline was carried over unchanged into the setting this thesis is actually built around: all six DAGHAR datasets, evaluated leave-one-dataset-out as described in Section 3.1.4, so that every fold’s target dataset is entirely unseen during training and model selection. This single change in protocol, with no change to the model itself, produced by far the largest effect observed anywhere in this thesis.

Table 4.2 reports the full leave-one-dataset-out comparison. Both backbones’ uncompressed accuracy fell to roughly 60% (GCN $61.60\% \pm 9.31$, GAT $60.39\% \pm 10.15$), a drop of roughly thirty percentage points from the 86–91% within-dataset numbers of Section 4.2.2, on the same architectures that had looked essentially solved a stage earlier. The best single result in this table, the convolutional backbone with node coarsening at $62.45\% \pm 6.91$ (also the best macro-F1, 56.29), again shows compression acting as a regularizer rather than a liability under domain shift, mirroring the sparsification result of Section 4.2.2. But no storage-optimization operator, in either direction, moved the mean by more than about 2.6 percentage points, and none of the resulting differences reached statistical significance at this sample size ($n = 6$ folds). The generalization gap dwarfs every storage-optimization effect observed in this chapter.

The failure behind these numbers is not uniform noise; it concentrates in specific, interpretable class confusions. When the UCI-HAR dataset served as the held-out target, for example, the baseline convolutional backbone misclassified 116 of 138 standing windows as sitting, and scattered a large share of its walking windows into the two stair classes. Difficulty was also not evenly spread across target datasets: KuHar was the hardest target throughout this stage, with accuracy frequently at or near 50%, while MotionSense was consistently the easiest, in the low-to-high seventies. Both patterns recur, and are examined more closely, in the later arcs of this chapter.

Table 4.2: Leave-one-dataset-out accuracy after the benchmark’s first exposure to all six DAGHAR datasets, backbone by storage-optimization operator. Mean and standard deviation are computed across the six leave-one-dataset-out folds.

Backbone – Operator	Accuracy (%)	Macro-F1 (%)
GCN – none (baseline)	61.60 ± 9.31	53.85
GCN – random edge deletion	61.60 ± 6.44	53.45
GCN – adaptive windowing	59.10 ± 5.36	52.69
GCN – graph sparsification	59.14 ± 3.43	52.91
GCN – node coarsening	62.45 ± 6.91	56.29
GAT – none (baseline)	60.39 ± 10.15	53.71
GAT – random edge deletion	57.83 ± 8.43	50.99
GAT – adaptive windowing	58.35 ± 7.90	51.11
GAT – graph sparsification	59.98 ± 10.85	53.24
GAT – node coarsening	58.38 ± 9.67	51.15

This is the point in the project’s history where the central research problem stated in Chapter 1 was actually discovered rather than assumed. A thirty-point accuracy cliff between within-dataset and cross-dataset evaluation, on the same architectures, made clear that storage optimization was not the binding constraint on deployability; a model that fails to generalize is not made more deployable by also being smaller. This finding redirected the remainder of the project from a pure storage-optimization study toward the cross-dataset generalization problem that occupies Sections 4.3 through 4.6, with storage optimization returning only in Section 4.7, once a model existed that was worth compressing.

4.3 Arc II: Feature and Graph-Representation Exploration

The accuracy cliff of Section 4.2.3 left two possible culprits: the time-domain features fed to the model, and the timestep-as-node graph structure itself. This arc works through both, in that order, before arriving at the sensor-as-node architecture and training recipe that Chapter 3 presents as the project’s proposed framework. Two of the six stages below are negative results, reported in the same detail as the positive ones, because each failure directly motivated the stage that followed it.

Table 4.3: Leave-one-dataset-out accuracy after switching to frequency-domain node features, still on the timestep-as-node graph. Gains are measured against the time-domain results of Table 4.2.

Backbone (frequency features)	Accuracy (%)	Macro-F1 (%)	Gain over time-domain
GCN	64.64 ± 8.22	58.32	+3.04 pp
GAT	69.38 ± 7.96	64.09	+8.99 pp
GraphSAGE (new)	70.70 ± 8.64	65.85	—
GIN (new)	67.72 ± 10.99	63.14	—
GCN + node coarsening	65.71 ± 8.00	61.30	+3.26 pp

4.3.1 Frequency Features on the Timestep Graph

The first response to the accuracy cliff targeted representation rather than architecture. Time-domain statistics are sensitive to per-dataset amplitude scale, device placement, and orientation—exactly the nuisance variation a domain-shift benchmark is built to expose—whereas the frequency content of human motion, such as step cadence and its harmonics, should be comparatively invariant across devices and placements. The forty-two-dimensional time-domain node features were therefore replaced with a seventy-two-dimensional frequency-domain representation: a Hann-windowed local Fourier transform computed per channel, yielding eight magnitude bins plus spectral energy, entropy, dominant frequency, and centroid. The graph itself was still built over sixty timesteps, unchanged from Section 4.2.3; only the node features and the resulting nearest-neighbour edges changed. Two additional backbones, GraphSAGE and graph isomorphism networks (GIN), were introduced alongside the existing convolutional and attention backbones on the hypothesis that aggregation style might itself matter for cross-domain robustness.

Every carried-over backbone improved, by three to nine percentage points, confirming that spectral descriptors transfer across datasets better than local time-domain statistics. The convolutional-attention ranking flipped again: the attention backbone, which had fallen behind in Section 4.2.3, gained the most (nearly nine points) and moved back ahead of the convolutional backbone, consistent with the idea that attention coefficients computed on stable spectral features are themselves more stable across domains than attention computed on dataset-specific amplitude patterns. GraphSAGE, introduced here for the first time, immediately took the lead at 70.70%, with the best macro-

Table 4.4: Leave-one-dataset-out accuracy with concatenated time- and frequency-domain node features (114-dimensional), still on the timestep-as-node graph. Every backbone loses ground relative to the frequency-only features of Table 4.3.

Backbone (hybrid features)	Accuracy (%)	Macro-F1 (%)	Change vs. frequency-only
GCN	60.37 ± 9.84	53.28	−4.27 pp
GAT	67.05 ± 7.98	61.23	−2.33 pp
GraphSAGE	66.38 ± 4.46	59.87	−4.32 pp
GIN	66.12 ± 7.37	61.46	−1.60 pp

F1 of any configuration so far. And node coarsening again beat its own uncompressed baseline under domain shift (65.71% versus 64.64%), the second consecutive stage in which compression behaved as a regularizer rather than a cost, the first instance being the node-coarsened result of Section 4.2.3. As with every leave-one-dataset-out result in this chapter, these gains should be read as consistent trends across six targets rather than as significance-tested claims; the sample size limitation discussed in Section 3.5.3 applies throughout.

4.3.2 Hybrid Features on the Timestep Graph

The natural next question was whether combining time- and frequency-domain information would help further, since the two views plausibly carry complementary information—the frequency view separates low- from high-energy activities, while the time view helps distinguish activities of similar energy, such as walking from climbing or descending stairs. The two feature blocks were concatenated into a single 114-dimensional node vector (forty-two time-domain plus seventy-two frequency-domain) and fed to the same four backbones, still on the timestep-as-node graph.

This is the arc’s first clean negative result. Every backbone lost ground relative to the frequency-only features of the previous stage, and the convolutional backbone even fell slightly below its own original time-domain result from Section 4.2.3 (60.37% versus 61.60%), with its worst single target collapsing to 43.8%. The likely mechanism is that concatenation re-introduces, as a fixed input, exactly the dataset-specific amplitude and placement information that the frequency-only representation had removed; the model has no way to tell which of its 114 input dimensions will generalize,

so training happily exploits the source-discriminative time-domain block, and the result is source overfitting rather than the hoped-for complementarity. The damage tracked each architecture’s capacity to down-weight noisy inputs: the convolutional backbone, with uniform aggregation and no attention or gating, lost the most, while the attention backbone, which can suppress unstable feature directions, lost the least and became the best hybrid configuration. Tellingly, hybrid features did still win on specific dataset targets, precisely those where the frequency-only view had been weakest, which suggested the time-domain information was not useless, only that naive concatenation was the wrong way to combine it. This diagnosis, together with the added storage cost of a 114-dimensional feature (roughly 42% more data per fold than the frequency-only representation), is what motivated the architectural rewrite of Section 4.3.3 rather than further feature engineering on the same graph. The apparent contradiction with the positive hybrid result of Section 4.3.6 later in this arc is only apparent: that later result combines the same two views through a learned, per-channel-normalized encoder under domain-adversarial pressure, not through fixed concatenation, and the mechanism section below explains why that distinction is what makes the difference.

4.3.3 Sensor-as-Node Graph Rewrite

With representation gains from the timestep-as-node graph exhausted, this stage changed the most basic design decision in the pipeline: what a node represents. Rather than sixty timesteps connected by nearest-neighbour edges over hand-crafted statistics, each graph now has exactly six nodes, one per physical sensor channel (three accelerometer axes and three gyroscope axes), and each node’s input is its entire raw sixty-step signal, encoded by a learned one-dimensional convolutional network rather than by hand-crafted statistics at all. This is the sensor-channel graph and per-modality CNN encoder presented as the proposed architecture in Section 3.2.1 and Section 3.2.3. Edges were fixed rather than learned: every pair of channels is connected, but each edge carries one of three types—intra-accelerometer, intra-gyroscope, or cross-modal—reflecting the physical relationships between the channels rather than any statistic computed from the data. This fixed, physically grounded topology is precisely what later becomes the physical-prior term

Table 4.5: Leave-one-dataset-out accuracy of the sensor-as-node graph, four backbones plus the no-message-passing control, before domain-adversarial training is introduced.

Configuration	Accuracy (%)	Note
CNN-only (no graph)	68.51 $\pm$ 5.21	best; message passing adds nothing here
Attention backbone	68.12 $\pm$ 5.88	
GraphSAGE	67.94 $\pm$ 6.11	
Convolutional backbone	67.83 $\pm$ 6.04	
GIN	67.41 $\pm$ 6.32	

blended into the adaptive graph of Section 3.2.6; its origin is here, not in the frequency-feature work of the previous two stages. A CNN-only ablation, with no message passing at all, was introduced alongside the four graph backbones as a deliberate control, to make sure any apparent benefit of the new graph could be attributed to message passing rather than to the new per-channel CNN encoder alone.

The rewrite is also, by construction, an order-of-magnitude storage reduction before any compression technique is applied: thirty edges and six nodes per graph, versus the previous representation’s several hundred edges and sixty nodes.

All five configurations cluster within about one percentage point of each other, and the no-message-passing control wins outright: at this stage, message passing over the six-node graph contributes nothing measurable. With only six fully connected nodes and two graph layers, every node already sees every other node twice, leaving little relational structure left for message passing to exploit; a learned, per-channel CNN encoder is doing essentially all of the work. None of these five configurations beats the frequency-feature record of Section 4.3.1 (70.70%) either, so this stage’s contribution is architectural and storage-related rather than a new accuracy record: it converts the problem into a far more compact and physically interpretable shape, on which the still-untested domain-adversarial mechanism could plausibly do more work than it could on the old graph. That test is the subject of the next stage.

Table 4.6: Leave-one-dataset-out accuracy with domain-adversarial training added to each sensor-graph configuration, and the change relative to the non-adversarial results of Table 4.5.

Configuration	Accuracy (%)	Macro-F1 (%)	Change vs. no adversary
GraphSAGE	70.73 $\pm$ 7.95	64.73	+2.79 pp
GIN	67.80 $\pm$ 6.58	62.51	+0.39 pp
CNN-only (no graph)	66.49 $\pm$ 6.63	60.69	−2.02 pp
Convolutional backbone	66.35 $\pm$ 3.96	61.74	−1.48 pp
Attention backbone	64.60 $\pm$ 4.09	60.16	−3.52 pp

4.3.4 Domain-Adversarial Training and Backbone Convergence

The sensor-as-node graph left a specific puzzle: a compact, physically interpretable architecture whose graph component added no value on its own. The working hypothesis was that the missing ingredient was not model capacity but domain invariance—nothing in the training objective so far had asked the encoder to ignore which source dataset a window came from. This stage applies the domain-adversarial objective of Section 3.2.5 to all five sensor-graph configurations from Section 4.3.3, using the Ganin gradient-reversal schedule with the adversarial weight ramped to its maximum value over training.

Domain-adversarial training helped exactly one backbone. GraphSAGE gained nearly three points and set a new project-best accuracy of 70.73%, edging past the frequency-feature record of Section 4.3.1 on a graph roughly ten times smaller. Every other backbone was hurt or left unchanged, so much so that the mean across all five configurations actually fell slightly once the adversarial objective was added. This divergence is the reason the framework proposed in Chapter 3 describes only a GraphSAGE backbone in Section 3.2.4: convolutional, attention, and isomorphism-network backbones were all tried under the same domain-adversarial objective on the same graph, and none of them tolerated it as well as GraphSAGE did. A plausible mechanism is that GraphSAGE’s degree-independent mean aggregation produces embeddings in which domain information and activity information are separable enough for the gradient-reversal layer to strip out the former without damaging the latter, whereas the attention backbone’s coefficients appear to entangle the two—attention patterns are themselves domain-specific, so suppressing domain

information along with them removes useful structure as well. This stage also produced the first evidence, in this architecture, that message passing has genuine value: under adversarial pressure, GraphSAGE beat the no-message-passing control by more than four points, reversing the control’s advantage from Section 4.3.3. Both of the original aspiration targets set for this line of work, an intermediate 74% mean and an eventual 77% mean matching the DAGHAR ConvNet baseline, were still missed at this stage, but the combination of the sensor graph, GraphSAGE, and domain-adversarial training was fixed from this point on as the architecture every later stage builds from.

4.3.5 Configuration Sweep

Before changing anything conceptual, one cheap question remained: was the roughly fifty-thousand-parameter GraphSAGE backbone simply too small? Four scaled-up configurations were tested, spanning a deeper convolutional encoder, a wider one, a combined deeper-and-wider encoder with an additional graph layer, and a larger temporal kernel, holding the graph, the input features, and the domain-adversarial training setup fixed. None of the four beat the small default’s 70.73% mean; the closest came within half a point but did so with substantially higher variance across targets, propped up by a single unusually strong result on one dataset while losing ground on three others. Extra capacity, under leave-one-dataset-out evaluation, was evidently spent fitting source-specific detail that the unseen target then penalized, rather than improving generalization. This negative result closed the architecture-scaling question for the rest of the project: no later stage attempts to enlarge the backbone again, and the compact default remains the reference architecture, reinforcing this thesis’s second research goal of keeping the model cheap without sacrificing the first.

4.3.6 Recipe Finalization

With capacity ruled out, the one lever not yet tried on the sensor-as-node graph was input representation, the same lever that helped substantially on the earlier timestep-as-node graph in Section 4.3.1 but backfired when naively combined in Section 4.3.2. This stage retries both frequency

Table 4.7: Leave-one-dataset-out accuracy of the finalized recipe: sensor graph, GraphSAGE, domain-adversarial training, and frequency or hybrid input features with source-only per-channel normalization.

Configuration	Accuracy (%)	Macro-F1 (%)	Change vs. raw time (Table 4.6)
Hybrid (time + frequency)	73.05 $\pm$ 6.23	68.20	+2.32 pp
Frequency only	72.53 $\pm$ 7.52	67.97	+1.80 pp
Frequency, softer adversarial weight	69.21 $\pm$ 8.18	65.71	−1.52 pp

and hybrid features on the now-fixed sensor graph, GraphSAGE backbone, and domain-adversarial objective, with one further change: features are normalized per channel using statistics fit only on source training data, so the target dataset is never touched during normalization.

Hybrid features now help, reversing the negative result of Section 4.3.2 and setting a new project-best mean of 73.05%, with the lowest cross-target variance of any leading configuration so far. This is not a contradiction of the earlier negative result but its resolution: in Section 4.3.2, time and frequency statistics were concatenated as fixed, hand-crafted features, and the model had no mechanism to suppress the dataset-specific time-domain component. Here, the raw time signal and its frequency magnitude spectrum are normalized per channel on source-only statistics and passed together into a learned convolutional encoder trained under domain-adversarial pressure, which can weight the two views adaptively rather than being forced to treat both as fixed and equally trustworthy inputs. The general lesson carried forward from this comparison is that feature fusion fails as concatenation of fixed features but succeeds as joint input to a learned, invariance-regularized encoder. A third variant, which softened the adversarial weight and lowered the learning rate on the frequency-only input, lost ground rather than gaining it, confirming that the training recipe fixed in Section 4.3.4 was already close to a local optimum rather than under-tuned.

This configuration—the sensor-as-node graph, the small GraphSAGE backbone, domain-adversarial training, and hybrid time-and-frequency input features with source-only normalization—is fixed for the remainder of the chapter as the control recipe against which every later architectural change is measured, referred to throughout the rest of this chapter as Arm A. Against the approximately 77% ConvNet reference accuracy on this benchmark, the gap has narrowed to roughly four per-

centage points; closing that remaining gap is the explicit agenda of Arc III and Arc IV, taken up next in Section 4.4.

4.4 Arc III: Domain Adaptation and Robustness Attempts

Arm A, fixed at the end of Section 4.3, left a roughly four-point gap to the approximately 77% accuracy reference set by the DAGHAR ConvNet baseline. This arc is the project’s most sustained attempt to close that gap without changing the underlying graph structure. It tries four different classes of intervention, in roughly chronological order: alternative feature and normalization choices within the single model, fusing the model with an independently trained convolutional network, adapting the model at or near test time, and, finally, giving the model limited access to the target dataset’s unlabeled data under a relaxed, transductive protocol. Only one of these four classes of intervention survives; the rest are reported compactly, because their consistent failure is itself the arc’s main finding: closing this gap by refining the single model further, or by giving it more access to the target domain, did not work, and that negative result is what ultimately justifies the stricter, target-blind protocol this thesis adopts.

4.4.1 Feature, Normalization, and Alignment Variants

Four single-model modifications to Arm A were tried in this stretch, each changing exactly one component while holding everything else fixed, and each measured against the keep/revert rule of Section 3.4.2. None cleared it.

The statistical-alignment attempt replaced the domain-adversarial objective itself with explicit distribution-matching losses between source datasets, on the reasoning that matching feature covariances directly might be more stable across backbones than a discriminator that has to be fooled. It underperformed the domain-adversarial result by a wide margin, most likely because covariance matching aligns the marginal distribution of pooled embeddings while the actual failure mode under domain shift, discussed in Section 4.2.3, is a conditional one—class boundaries such as sitting

Table 4.8: Single-model modifications to Arm A, all measured against the leave-one-dataset-out mean established in Section 4.3.6. Every row reverts.

Modification	What changed	Accuracy (%)	Change
Statistical source alignment	DANN replaced by CORAL + MMD + domain-specific batch norm	67.21	−5.84 pp*
Recurrent temporal pooling	mean-pool replaced by a bidirectional recurrent encoder	71.29	−1.76 pp
Learned attention pooling	mean-pool replaced by a learned attention pool	73.11	+0.06 pp (noise)
Dual-view gated encoder	separate time/frequency encoders with a learned gate	70.35	−2.70 pp
Per-bin-channel normalization	per-channel z-score replaced by a finer per-frequency-bin z-score	69.23	−3.82 pp
Source-domain augmentation	jitter, scaling, warping, and shifting applied to source-train windows	70.53	−2.52 pp

*Evaluated on a separate data pipeline with different per-target test-set sizes; the comparison is indicative rather than exact, but the gap is large enough that the qualitative conclusion is unaffected.

versus standing shifting across datasets—which matching means and covariances does not fix. The remaining five modifications kept the domain-adversarial objective fixed and instead tried to improve what feeds it: two alternative ways of pooling the temporal signal, a gated two-branch encoder that let the model weight time-domain and frequency-domain information separately, a finer normalization granularity, and finally training-set augmentation. All five failed in a similar way: any added flexibility was spent fitting source-specific detail rather than target-transferable structure, and the damage concentrated on the single hardest target dataset in every case, with source-domain augmentation actually the most damaging (a drop of over fifteen points on that target alone at its strongest setting), because the transformed source windows sharpened exactly the source-specific patterns the model was already prone to overfit, and because the added augmented batches complicated the domain-adversarial objective’s own training dynamics. The one exception, learned attention pooling, moved the mean by an amount well within single-seed noise and offered no reliable gain. Taken together, this stretch closes the single-model line of attack: nothing that only changes how Arm A processes its own inputs measurably improves it, and the model’s

Table 4.9: Convolutional late fusion, compared against each model alone. Two independent training runs are reported; both confirm the same qualitative result.

Run	Graph model alone	Convolutional model alone	Fused (best weight)
First training run	71.15%	72.67%	75.69% ($w = 0.6$)
Second training run	70.72%	74.84%	76.32% ($w = 0.5$)

compact mean-pooling and per-channel normalization defaults are better understood as regularizers that prevent over-specialization to the source datasets, not as underpowered defaults waiting to be replaced.

4.4.2 Convolutional Late Fusion

With single-model refinement exhausted, this stage asks a different question: rather than changing how the graph model processes its inputs, can it be combined with an independently trained model that processes the same data differently, so that the two disagree on different windows and a simple combination beats either alone? A faithful reimplementation of DeepConvLSTM [2], the convolutional-recurrent architecture behind the DAGHAR ConvNet reference accuracy, was trained independently on each leave-one-dataset-out fold, using a log-magnitude frequency view of the same windows rather than the graph branch’s hybrid time-and-frequency features. The two models’ output probabilities are then combined by a simple weighted average, with no joint training and no shared parameters: this is exactly the late-fusion rule presented in Section 3.2.7.

Fusion beat both individual models in both training runs, reaching 76.32% in the stronger of the two runs, a gain of more than three points over the graph model’s own 73.05% mean. An oracle upper bound—crediting a correct answer whenever either model alone got it right—sits around 84%, showing real complementary signal remains unexploited by simple probability averaging, but the practical gain from fusion alone already exceeds anything achieved by the single-model modifications of Section 4.4.1. The two models fail on different folds for an interpretable reason: the graph model tends to lead on the datasets closest to the overall source pool, while the convolutional model, with its recurrent encoder reading long-range temporal dependencies from a single contin-

uous sequence, tends to lead on datasets where that kind of temporal structure matters more than cross-sensor relational structure. Because the two architectures encode genuinely different aspects of the signal, the samples they get wrong do not fully overlap, which is the necessary condition for averaging to help. This complementarity, not any refinement internal to either model, is what finally moves accuracy past the single-model ceiling reached in Section 4.3.6, and convolutional late fusion is kept as a permanent part of the system from this point forward.

4.4.3 Test-Time and Training-Time Adaptation

The fused system still fell short of the target accuracy, so the next two attempts asked whether the graph branch specifically could be strengthened by adapting it, at or near test time, to statistics of the unlabeled target data it would eventually be evaluated on. Both attempts are reported compactly, as brief negative results that close this line before it develops further.

The first, a batch-normalization adaptation applied only to the graph branch, recomputes its normalization statistics from the unlabeled target windows at test time, leaving the convolutional branch untouched since it has no batch-normalization layers to adapt. This helped substantially on the graph branch’s two most difficult target datasets, matching the intuition that adaptation should help exactly where the model was least prepared for the target’s statistics, but it damaged the model badly everywhere else—accuracy fell by more than twenty-five points on one of the graph branch’s previously strongest targets—so the fused mean fell by roughly 1.3 points overall (75.05% versus the 76.32% fusion baseline of the previous subsection). The explanation follows directly from how the domain-adversarial objective already works: on targets where it succeeds at aligning the graph branch’s representations, the source-fitted normalization statistics are already well matched to the target, and overwriting them with statistics estimated from a comparatively small, unlabeled test set introduces noise rather than removing bias; on the targets where the adversarial objective fails to align representations, target statistics genuinely help. Because the adaptation cannot tell in advance which regime a given target falls into, its net effect is negative.

The second attempt combined two established domain-generalization regularizers, dense stochas-

tic weight averaging and feature-statistic mixing across the training batch, in the hope that a flatter training trajectory and reduced reliance on batch-specific feature statistics would generalize better without touching the target at all. Instead, it produced the single worst result of the entire project for the graph branch alone, a collapse of more than sixteen points relative to Arm A, severe enough that the optimal fusion weight for the graph branch became zero: the fused system reduced to the convolutional model running by itself, at 74.84%, below the fusion baseline of Section 4.4.2. The most likely explanation is an interaction between the two regularizers and the domain-adversarial objective itself: weight averaging blends model states from a training window in which the adversarial pressure is still actively reshaping the representation, while batch-level feature-statistic mixing scrambles exactly the domain-identifying signal the adversarial discriminator depends on, and the two effects compound rather than cancel. With this result, the project treated the supervised domain-generalization search as closed for the single graph model, and the fused result of 76.32% from Section 4.4.2 was locked in as the best domain-generalization system reached so far, ahead of the adaptive graph construction introduced in Section 4.5.

4.4.4 Transductive Domain Adaptation Family

Every attempt so far had respected a strict domain-generalization protocol: the target dataset was never seen, in any form, during training. This final stretch of the arc relaxes that protocol deliberately, to test whether giving the model legitimate, label-free access to the target domain’s own data distribution during training—not at inference, as in Section 4.4.3, but as part of training itself—could close more of the remaining gap. This is a different task from domain generalization, strictly easier because more information is available, and every result in this subsection is reported as a change relative to a matched, target-aware lower bound established under the same relaxed protocol, not against the domain-generalization numbers reported elsewhere in this chapter.

That lower bound was established first, by training the same fused system with the unlabeled target data loaded but never used, confirming the training pipeline behaved correctly by reproducing the domain-generalization fusion result almost exactly (76.88%, against 76.32% under the

Table 4.10: Transductive domain-adaptation methods, evaluated relative to the target-aware lower bound of the same protocol. Both revert.

Method	Mechanism	Best mean (%)	Change
Transductive normalization	normalization statistics fit on source and target together	76.25	−1.41 pp
Feature covariance alignment	a covariance-matching loss between source and target embeddings	77.47	+0.60 pp

stricter protocol, a difference well within normal run-to-run variation). Two adaptation methods were then tried against this lower bound, each required to clear the same keep/revert rule as everywhere else in this thesis: an improvement of at least one point in the mean, with no individual target dropping by more than two points.

The first method, which folds the unlabeled target data into the normalization statistics computed before training even begins, follows the same logic as the test-time adaptation of Section 4.4.3, and fails for the same reason: shifting the normalization toward the target helps the datasets most different from the source pool while badly hurting the datasets already close to it, and one shared set of statistics cannot serve both at once. The second method operates one level deeper, adding a loss term during training that pulls the model’s learned graph embeddings for source and target data toward the same covariance structure, without ever touching the target’s labels. This came far closer to succeeding: at a small alignment weight, it produced sizeable gains on the two hardest target datasets while keeping every other target within the keep/revert rule’s collapse guard, and its mean improvement, six-tenths of a percentage point, was positive for the first time in this arc. It still fell short of the required one-point threshold, and stronger alignment weights made the result worse rather than better, because the alignment term increasingly competed with, rather than supported, the classification objective it was meant to assist.

This near-miss marks the end of the arc’s search for gains through target-aware adaptation. Two further, more aggressive alignment techniques were planned but never run, once it became clear that the covariance-alignment result had already identified the actual bottleneck: the alignment methods were not failing because they were too weak, but because they were trying to compensate

for a graph branch whose underlying representation was itself the limiting factor. Improving that representation directly was judged the more promising direction, and the project accordingly returned to a strict, target-blind domain-generalization protocol and turned its attention to the graph construction itself, taken up next in Section 4.5. The broader lesson carried out of this arc is that every attempt to close the remaining gap by giving the model more access to the target domain, whether at test time or during training, either failed outright or fell just short of a meaningful gain, while the one successful intervention, convolutional late fusion, required no target access at all. This is the empirical case, made across five separate methods and one entire arc of the project, for committing to the stricter, target-blind protocol that the rest of this thesis follows.

4.5 Arc IV: Adaptive Graph Construction — The Best Model

Arc III closed on a specific diagnosis: alignment techniques that gave the model access to the target domain, whether at test time or during training, kept falling just short of a meaningful gain, and the covariance-alignment result of Section 4.4.4 suggested why—those methods were compensating for a graph branch whose own representation was the actual limiting factor, not for a lack of target information. This arc tests that diagnosis directly, by changing the one part of the graph branch that had been held fixed since its introduction in Section 4.3.3: the topology itself. Every experiment up to this point used the same fixed nine-edge topology, grouped into intra-accelerometer, intra-gyroscope, and cross-modal pairs. This arc replaces that fixed topology with one computed per window from the data itself, and the resulting adaptive graph construction becomes the thesis’s best model and its central contribution, corresponding to Section 3.2.6 of the proposed framework.

4.5.1 Design and Screening

The design question was whether a correlation-based adjacency—computed independently for every three-second window from the six raw sensor channels, and blended with the fixed physical-prior topology by the score defined in Equation 3.3—would give the GraphSAGE backbone a more

Table 4.11: Screening sweep over candidate adaptive-graph configurations, single seed, selected on source-validation accuracy only.

Configuration	Edges per window	Blend weight α	Source-validation accuracy (%)
Blend, top-9	9	0.5	82.23
Pure correlation, top-9	9	1.0	80.51
Pure correlation, top-6	6	1.0	79.95
Pure correlation, top-12	12	1.0	79.88

informative topology to work with than the single static structure used everywhere in Arc II and Arc III. Because GraphSAGE, as established in Section 3.2.4, aggregates over whichever edges are present but ignores any weight attached to them, the Pearson-correlation scores in this design act purely as a topology selector: the highest-scoring pairs for a given window become that window’s edges, and everything else about the training recipe, the backbone, and the frozen convolutional branch is untouched.

Following the two-stage discipline set out in Section 3.4, a small screening sweep was run first, at a single seed, with the winner chosen purely on source-validation accuracy—the held-out target dataset was never consulted at this stage.

The blended configuration, keeping nine edges per window at an equal fifty-fifty blend between the correlation score and the physical prior, won this screening sweep clearly, by nearly two full points over the next-best configuration. All three of the runner-up configurations relied on the correlation score alone, at three different edge budgets; none matched the blended winner, an early hint of the mechanism examined more closely in Section 4.5.3. This blended, nine-edge configuration was then carried into the second stage: full confirmation across five random seeds, alongside the fixed-topology control and, as an informative ablation, the pure-correlation alternative at the same edge budget.

4.5.2 Seed Confirmation and the Keep/Revert Decision

The adaptive blend clears both parts of the keep/revert rule from Section 3.4.2 comfortably: a gain of 1.28 percentage points in the mean, well above the required one point, and, critically,

Table 4.12: Five-seed leave-one-dataset-out accuracy: the fixed-topology control, the winning adaptive blend, and the pure-correlation ablation. Only the blend clears the keep/revert rule.

Configuration	Fused accuracy (%)	GNN branch alone (%)	Change vs. control
Fixed topology (control)	76.68 ± 1.36	72.05	—
Adaptive blend ($\alpha = 0.5$)	77.96 ± 0.21	74.41	+1.28 pp
Pure correlation ($\alpha = 1.0$)*	76.98 ± 0.41	71.42	+0.30 pp

*Three seeds rather than five; run as a diagnostic ablation once the blended winner was confirmed, not as a second candidate for adoption.

no individual target dataset drops at all relative to the fixed topology—the smallest gain, on the hardest-to-move target, is a fold that still improves by a fraction of a point rather than regressing. This is a stronger result than the rule requires, which only forbids a drop of more than two points; here every single target improves. The pure-correlation ablation, by contrast, fails the rule: although its mean also rises slightly, its worst target drops by nearly two points, and the required one-point mean threshold is not met either. The adaptive blend, not pure data-driven correlation, is therefore adopted as the graph construction method for the remainder of this thesis.

One presentational detail is worth fixing here to avoid confusion with Section 4.4.2. The fused results in this arc use the same grid-searched late-fusion rule as Arc III, with the blend weight re-selected for the new pairing at $w = 0.6$, favouring the graph branch—this is the adopted value stated in Section 3.2.7, and it is the weight behind every fused figure from this point forward, including the headline 77.96%. The shift from the $w = 0.5$ optimum of Arc III’s stronger fixed-topology run is not an inconsistency but a consequence of the pattern that arc already observed: the optimal mix tracks the branches’ relative strength, and the adaptive graph raises the graph branch above the frozen convolutional branch for the first time (Section 4.5.3), so the weight correspondingly tilts toward it.

The largest gains land on MotionSense and WISDM, with RealWorld-Waist and KuHar close behind; the smallest gains, still positive, are on RealWorld-Thigh and UCI, which were already among the easier targets under the fixed topology. KuHar, the target that has been the hardest throughout this chapter, both in this arc and in every arc before it, still improves by more than a

Table 4.13: Per-target leave-one-dataset-out accuracy, fixed topology versus the adopted adaptive blend (fused system, five-seed means). Every target improves.

Target dataset	Fixed topology (%)	Adaptive blend (%)	Change
KuHar	66.11	67.50	+1.39 pp
MotionSense	84.93	87.06	+2.13 pp
RealWorld-Thigh	74.85	74.93	+0.08 pp (smallest gain)
RealWorld-Waist	74.75	76.49	+1.74 pp
UCI	86.09	86.38	+0.29 pp
WISDM	73.33	75.38	+2.04 pp
Mean	76.68	77.96	+1.28 pp

point, consistent with the idea that a topology allowed to adapt per window has more room to help on a target whose signal characteristics differ most from the rest of the source pool.

A second, equally important effect of adopting the adaptive blend is a sharp reduction in seed-to-seed variance: the fixed-topology control’s standard deviation across five seeds is 1.36 percentage points, while the adaptive blend’s is 0.21, roughly a sixfold tightening. Under the fixed topology, a single unlucky or lucky seed could plausibly land anywhere from the mid-seventies to the high seventies; under the adaptive blend, five independent runs land within a narrow half-point band. A plausible mechanism is that the per-window variation in which edges are selected acts as a mild implicit regularizer on the training trajectory, damping the run-to-run stochasticity that otherwise comes from the GraphSAGE backbone’s own random initialization and minibatch ordering. Practically, this also means the adaptive blend’s headline number is a far more reliable estimate than any of the single-seed results reported earlier in this chapter, which is why the five-seed protocol was adopted as standard from this stage onward.

4.5.3 Why the Blend Wins and Pure Correlation Does Not

The screening sweep and the seed confirmation both point to the same conclusion—an equal blend of data-driven correlation and physical prior beats either extreme—and the mechanism follows directly from how the backbone consumes the graph. Because GraphSAGE ignores edge weights and only aggregates over whichever edges are present, the correlation score’s entire effect is to

decide *which* pairs of channels become edges in a given window; it never influences how strongly a message is weighted once an edge is chosen. Under a purely data-driven topology, this selection is made independently for every window, with no structural constraint tying it back to anything invariant across datasets. On windows drawn from the sources the domain-adversarial objective was trained on, this is usually fine. But on a window from a genuinely unfamiliar target, ordinarily unrelated channels can become transiently, spuriously correlated simply because of an unfamiliar motion pattern, and a purely data-driven selector will happily promote that spurious pair into an edge. Because the domain-adversarial discriminator depends on the graph branch producing a broadly consistent structure across domains to learn what to suppress, a topology that can swing this unpredictably on unfamiliar data undermines exactly the mechanism the training recipe relies on—which is consistent with the pure-correlation ablation’s weakest results appearing on the same challenging target datasets that have driven every difficulty in this chapter.

The fixed physical prior—the same intra-accelerometer, intra-gyroscope, and axis-aligned cross-modal pairing introduced in Section 4.3.3—does not have this problem, because it never changes regardless of what the data looks like; but used alone, it also cannot exploit whatever genuine, useful correlation structure a window’s signal actually contains. Blending the two scores at an equal weight lets the correlation term still influence which of the prior’s neighboring or additional pairs get promoted, while preventing any single unusual window from selecting a topology disconnected from the physically grounded structure the rest of the system was built around. The practical upshot is that the physical prior established in Section 4.3.3 for entirely different reasons—interpretability and storage efficiency—turns out to be empirically load-bearing here as well: it is not a placeholder that a fully data-driven topology can simply improve on, but a stabilizing anchor that the data-driven signal needs in order to help at all.

One further consequence of the adopted blend is worth noting for the fusion result of Section 4.4.2. The graph branch alone, under the adaptive blend, reaches 74.41% mean accuracy, higher than the frozen convolutional branch’s own accuracy for the first time anywhere in this chapter. Late fusion tends to help most when its two component models are of comparable strength and

Table 4.14: Leave-one-dataset-out accuracy at each of the project’s three domain-generalization milestones. The adaptive graph is adopted as the thesis’s final system.

Milestone	Mean accuracy (%)
Single graph branch alone (Section 4.3.6)	73.05
Fixed-topology graph branch fused with the convolutional branch (Section 4.4.2)	76.32
Adaptive-blend graph branch fused with the convolutional branch	77.96 $\pm$ 0.21

make different mistakes; with the graph branch no longer the weaker of the two, the fused system benefits from a better-balanced pair of models rather than one strong branch being dragged down by fusing with a noticeably weaker one.

4.5.4 The Thesis’s Best Model

The adaptive-blend graph construction is adopted from this point forward as the thesis’s final domain-generalization system, matching the roughly 77% ConvNet reference accuracy this chapter has measured itself against since Section 4.3.4, while never using any target-domain data at any stage of training, consistent with the strict protocol Arc III argued for. It is this system, not any of the intermediate milestones in Table 4.14, that is carried into the final architecture search of Section 4.6 and the storage-optimization study of Section 4.7.

4.6 Arc V: Final Architecture Search

The adaptive graph adopted in Section 4.5.4 left two further, more targeted questions open, one on each side of the strict domain-generalization line this thesis otherwise holds to. The first stays entirely within the domain-generalization protocol and asks whether the backbone itself, still GraphSAGE at this point, was leaving information on the table by ignoring the actual strength of the correlation scores it had just been shown to select on. The second briefly reopens the relaxed, target-unlabeled protocol closed in Section 4.4.4, on the reasoning that a stronger graph branch might succeed at self-training where the earlier attempts, built on a weaker branch, had not. Both attempts are reported here as negative results, and together they close the search for further accu-

Table 4.15: Residual Chebyshev backbone versus the adopted GraphSAGE backbone, three-seed means, both using the identical adaptive-graph topology.

Backbone	Fused accuracy (%)	GNN branch alone (%)	Change
GraphSAGE (adopted)	78.10 ± 0.14	75.19	—
Residual Chebyshev convolution	77.36 ± 0.88	73.03	−0.74 pp

racy gains, leaving the adaptive-graph system of Section 4.5.4 as the thesis’s final model.

Both experiments in this section were run and gated against an earlier, three-seed measurement of the adopted adaptive-graph recipe, $78.10\% \pm 0.14\%$, rather than the fully hardened five-seed figure of $77.96\% \pm 0.21\%$ reported in Section 4.5.2. The two numbers describe the identical recipe measured with different seed counts, not two different systems, and neither of this section’s keep/revert verdicts changes under either baseline.

4.6.1 Weight-Aware Message Passing

GraphSAGE, as established in Section 4.5.3, uses the adaptive graph’s correlation scores only to decide which edges exist; it never uses the scores’ magnitude once an edge has been selected. This stage asks whether a backbone that can consume those magnitudes directly, Chebyshev spectral graph convolution with residual connections, extracts additional signal from the same adaptive topology. The graph construction itself was held fixed at the adopted winner from Section 4.5.2; the only change was the message-passing operator, from GraphSAGE to a residual Chebyshev-convolution stack that consumes the correlation scores as continuous edge weights through the graph Laplacian rather than discarding them.

Following the same two-stage discipline as every other keep/revert decision in this chapter, four spectral-backbone configurations, varying the polynomial order and the number of residual blocks, were screened at a single seed on source-validation accuracy alone. The best-scoring configuration on that screen was then confirmed across three seeds against the adaptive-graph baseline.

The spectral backbone loses on every measure that matters: its mean is nearly a point below GraphSAGE’s, its worst individual target drops by more than three and a half points, well past

the two-point collapse guard, and its seed-to-seed variance regresses sixfold, undoing most of the stability gain that came with adopting the adaptive graph in the first place. Only one target improves meaningfully, the chronically difficult target that has driven the largest gains and the largest instability throughout this chapter, and that single gain is not enough to offset the collapse elsewhere. A further, specifically informative negative finding accompanied the screening stage itself: the configuration that scored highest on source-validation accuracy during screening was, of the configurations evaluated, the weakest on the held-out target once all three seeds were run—the opposite ordering from what the screening stage was designed to predict. This inversion is a recognizable signature of a model overfitting to source-domain structure in a way that source-validation accuracy cannot detect, and it is a caution specific to spectral methods: a backbone whose filters are tuned to the eigenspectrum of a graph Laplacian can score well on validation data drawn from the same domains it was trained on while generalizing poorly to an unfamiliar one, a distinction ordinary source-validation selection has no way to see.

The mechanism is consistent with everything Arc IV established about why the adaptive graph works. A spectral method needs the graph’s structure to produce a broadly similar eigenspectrum across domains for its learned filters to transfer, but the adaptive graph’s edge weights are, by design, computed fresh per window and vary with whatever correlation pattern a given window happens to show; a spectral filter tuned to the average Laplacian implied by the source datasets has no guarantee of matching an unseen target’s typical Laplacian, and the collapse on the fold that deviates most from the source pool is consistent with exactly that mismatch. GraphSAGE’s local, weight-agnostic aggregation, by contrast, has no eigenspectrum to mismatch in the first place, which is part of why it proved robust enough to serve as the backbone from Section 4.3.4 onward. This result also directly answers the question left open at the end of Section 4.5.3: the value in the adaptive graph’s correlation scores lies entirely in which edges they select, not in their magnitude, and a backbone built to exploit that magnitude does worse, not better. GraphSAGE is confirmed as the backbone for the remainder of this thesis.

Table 4.16: Pseudo-label self-training versus the source-only adaptive-graph baseline, three-seed means.

Configuration	Fused accuracy (%)	GNN branch alone (%)	Change
Source-only (adopted baseline)	78.10 ± 0.14	75.19	—
Pseudo-label self-training	77.87 ± 0.49	74.46	−0.23 pp

4.6.2 Pseudo-Label Self-Training

The second and final attempt in this search briefly reopens the transductive protocol closed at the end of Section 4.4.4, on the reasoning that the covariance-alignment methods tried there had been limited by the graph branch’s quality at the time, and that the substantially stronger adaptive-graph branch of Section 4.5.4 might succeed where those attempts had not. Rather than aligning feature distributions directly, this stage tries teacher-student self-training: a model trained source-only with the adopted recipe serves as a teacher, assigning pseudo-labels to whichever unlabeled target windows it is confident enough about, and a freshly initialized student is then trained on the source data together with those pseudo-labeled target windows, with the pseudo-labeled windows excluded from the domain-adversarial loss so they inform classification without disturbing the source-only domain discriminator. A confidence threshold controls how many target windows are trusted, and a class-balancing guard caps how many pseudo-labels any single predicted class can contribute, specifically to prevent a target dataset with fewer activity classes than the benchmark’s full label set from being flooded with confidently wrong labels for activities it does not contain.

The same two-stage discipline was followed once more: four threshold-and-balancing configurations were screened at a single seed on source-validation accuracy, and the winner was confirmed across three seeds against the source-only baseline.

The pseudo-label student again fails the keep/revert rule, on both counts: the mean gain falls short of the required threshold, and the chronically difficult target this chapter keeps returning to drops by nearly three and a half points. The failure mode here, however, is instructive and different from anything seen in Section 4.4.4. On that same difficult target, the teacher is confident enough to assign a pseudo-label to only four of the six activity classes; the other two classes receive no

pseudo-labeled examples at all under the confidence threshold used, so the student never sees any target-flavoured example of them and its performance on those activities degrades relative to the teacher it was meant to improve on. This is a coverage failure, not a precision failure: the pseudo-labels the student does receive for that target are usually correct, there are simply not enough of them, and not across enough classes, to help. A second, distinct failure appears on the target dataset whose activity label set is a strict subset of the benchmark’s full six classes: the teacher, trained on datasets that do contain all six, confidently assigns pseudo-labels for classes that do not actually occur in this target’s data at all, and although the class-balancing guard limits the damage, a large share of the selected pseudo-labels for this target are simply wrong. Not every target suffered, however: the target with the strongest fusion synergy earlier in this chapter gained more than two points here, with pseudo-label precision above ninety-seven percent, confirming that the mechanism works cleanly when the teacher is both confident and correct across the target’s full class range—the method’s failure is fold-specific, not a fundamental flaw in the idea of self-training itself.

Taken together, Section 4.4.4’s feature-alignment attempts and this subsection’s self-training attempt fail for independent reasons rather than the same one: alignment could not bridge a distributional gap using only second-order statistics, while self-training is bounded by how good the teacher already is on the hardest target, and on that target the teacher is not accurate enough to bootstrap reliable pseudo-labels across its full range of activities. Two structurally different domain-adaptation families, tried against two different strengths of the underlying graph branch, both close without clearing the bar this thesis set for adopting a target-aware method. The adaptive-graph system fixed in Section 4.5.4, evaluated strictly under the target-blind protocol and with no further architectural change, therefore stands as this thesis’s final domain-generalization system, and the remainder of this chapter turns to what it costs to store and run.

4.7 Arc VI: Storage Optimization Revisited

The thesis’s opening motivation, established all the way back in Section 4.2.3, was a storage-optimization question that domain generalization interrupted: the thirty-point accuracy cliff between within-dataset and cross-dataset evaluation made it clear that a smaller model was worthless if it could not also generalize. Every intervening arc set that question aside to chase generalization instead, first through representation and architecture in Section 4.3, then through adaptation and fusion in Section 4.4, and finally through the adaptive graph construction of Section 4.5.4, which this chapter now treats as fixed. This section returns to the original question on the model that finally answers it: what does the thesis’s best system, at $77.96\% \pm 0.21\%$ mean accuracy, actually cost to store and run, and how much of that cost can be recovered for free. The storage-optimization operator family introduced in Section 3.3 is applied here in full, following the same five-seed protocol used throughout Section 4.5, so that every comparison in this section is measured against the same 77.96% reference used everywhere else in this chapter.

4.7.1 Architecture Capacity and Post-Hoc Compression

Two independent levers were tested first: shrinking the backbone itself before training, and compressing an already-trained checkpoint afterward with no fine-tuning at all. Three backbone sizes were defined by scaling the CNN encoder’s channel widths and the GraphSAGE hidden dimension together, from the roughly fifty-two-thousand-parameter default down to a compact variant and a substantially smaller tiny variant with under five thousand parameters, a more than tenfold reduction.

Dropping a single sensor axis, the vertical gyroscope channel, costs nothing at all: the five-node variant ties the full six-node system almost exactly, well within noise. The remaining node-coarsening variants confirm the modality hierarchy this chapter has touched on before: removing the entire gyroscope modality costs about one point, while removing the entire accelerometer modality costs nearly five, confirming that accelerometer signal carries most of the information

Table 4.17: Architecture capacity variants and input-level node coarsening, five-seed means, fused system.

Variant	Accuracy (%)	GNN branch storage	Change
Full backbone (reference)	77.96 ± 0.21	222 KB	—
Tiny backbone (91% fewer parameters)	76.97 ± 0.74	37 KB ($5.9\times$ smaller)	−0.99 pp
Drop the vertical gyroscope axis (five nodes)	77.97 ± 0.75	221 KB	+0.02 pp (ties)
Accelerometer-only (three nodes)	76.85 ± 0.40	221 KB	−1.11 pp
Gyroscope-only (three nodes)	73.14 ± 1.19	221 KB	−4.82 pp
Shorter window, forty timesteps	76.89 ± 0.62	222 KB	−1.07 pp
Shorter window, thirty timesteps	75.73 ± 0.22	222 KB	−2.23 pp

this system relies on and gyroscope is a secondary, partially redundant contributor. Reducing the input window from sixty timesteps down to forty or thirty costs a little over one point and a little over two points respectively, a moderate but real trade against a real reduction in how much raw signal must be buffered per window. The tiny backbone, at a tenth of the parameters, costs under a point; a separate compact variant with roughly three times as many parameters as the tiny backbone was also tried, at a single seed, and performed worse than the tiny variant despite its larger size, indicating that the six-node sensor graph is small enough that architectural scaling below the full backbone is not monotone, and any compact-sized deployment point should be replaced by the smaller tiny variant rather than used as an intermediate step.

Post-hoc compression, applied directly to a saved full-backbone checkpoint with no retraining or fine-tuning at all, is considerably freer still. Half-precision weights, roughly halving the on-disk size, and dynamic eight-bit quantization of the model’s linear layers both leave accuracy essentially unchanged. Global magnitude pruning is similarly free up to fifty percent sparsity, with no measurable accuracy cost at either thirty or fifty percent of weights removed. Only at seventy percent sparsity does the model collapse outright, with accuracy on two of the six target datasets falling by several tens of points; enough of the graph branch’s weight paths are removed at that point that the network can no longer route information through the graph at all, and with no fine-tuning to recover, the damage is total rather than gradual. Compression, in short, is free up to a real threshold and then fails sharply rather than gracefully, which is itself a useful characterization for

Table 4.18: Edge-selection policies at a matched nine-edge budget, applied at inference to the same trained checkpoint, five-seed means.

Edge policy (nine edges per window)	Accuracy (%)	Change vs. adopted policy
Adaptive Pearson blend (adopted; ties intra-modality priority)	78.07 ± 0.54	—
Random edge selection	69.82 ± 1.05	−8.25 pp
Cross-modal edges only	48.91 ± 5.99	−29.16 pp

anyone deploying this system: the operating range below that threshold, not the threshold itself, is where the practical deployment decisions in this section are made.

4.7.2 Edge-Level and Input-Level Storage Optimization

The node-coarsening results of Section 4.7.1 already showed that *which* sensor channels are kept matters more than how many; this stage asks the same question of the graph’s edges. Because the adaptive graph construction of Section 4.5.1 computes its topology fresh for every window at inference time, alternative edge-selection policies can be substituted into an already-trained checkpoint with no retraining at all, letting this comparison isolate the effect of edge policy alone, at a matched edge budget, on the exact same trained weights.

At the same nine-edge budget the adopted adaptive policy and a heuristic that simply prioritizes intra-modality pairs (accelerometer-to-accelerometer and gyroscope-to-gyroscope) selected identical edges and produced identical accuracy, at this graph size—a policy consistent with the physical prior established in Section 4.3.3 recovers the same result as the correlation-based selection, without needing to compute any correlation at all. Random edge selection at the same budget loses more than eight points, and a policy restricted to cross-modal edges only, deliberately excluding the intra-modality pairs entirely, collapses by nearly thirty points, confirming again that intra-modal co-movement structure, not cross-modal pairing, is what the message-passing step actually depends on. A paired statistical test across the six leave-one-dataset-out folds confirms that the gap between the adopted policy and random selection is not an artifact of averaging: every fold favors the structured selection, and the difference is significant at this sample size. Reducing the

edge budget further, to six or three edges per window, costs additional accuracy under any policy, but the gap between a well-chosen and a poorly chosen topology at a fixed budget dwarfs the gap from changing the budget itself. The headline finding of this stage, in other words, is that structure matters far more than budget: at matched storage cost, which edges are kept determines far more of the outcome than how many edges are kept, echoing and reinforcing the mechanistic account given in Section 4.5.3 for why the adaptive graph works in the first place.

4.7.3 Combined-Axis Results and the Pareto Frontier

The individual levers examined so far are each near-free or moderately costly on their own. This stage asks whether they compose: does stacking several modest storage savings together cost roughly the sum of their individual costs, or do they interact, either compounding or partially cancelling one another.

Combining the free node-coarsening variant with the forty-timestep window reduction costs about as much as the window reduction alone costs by itself, consistent with a roughly additive composition rather than any strong interaction between the two axes; since dropping the gyroscope axis was itself free, the combined system’s cost is attributable almost entirely to the shorter window. Adding eight-bit quantization on top of that combination costs a further small increment in accuracy, for a correspondingly small additional reduction in size, since only the model’s linear layers are quantized under this scheme rather than every weight in the network.

Plotting every variant from this section against its own storage footprint traces out a small Pareto frontier with exactly two points on it: the node-coarsened variant that drops the vertical gyroscope axis, and the tiny backbone. Every combined-axis point, and every other single-lever variant, is dominated by one of these two: for a given storage budget, either the coarsened five-node variant or the tiny backbone reaches higher accuracy than any stacked combination of window-length reduction, quantization, and node coarsening together. The practical conclusion is that the single most effective lever for aggressive size reduction is architecture capacity itself, not any combination of smaller, individually cheaper levers; stacking several modest reductions does not

outperform simply choosing the one lever, whether that is dropping a redundant node or shrinking the backbone, that is best suited to the storage budget at hand.

4.7.4 Gradient-Conflict Diagnostic

One loose thread remains from earlier in this chapter. The domain-adversarial objective, adopted in Section 4.3.4 and used in every subsequent model, produced only a modest, backbone-selective gain when it was first introduced, and every attempt in Section 4.4.1 and Section 4.4.3 to push the model toward stronger domain invariance, whether through additional regularization or through adaptation at or near test time, either failed outright or made things measurably worse. This closing stage does not propose a new fix for that pattern; it measures directly why the pattern exists, as a diagnostic explanation of an effect already established elsewhere in this chapter, at no additional experimental cost, rather than as a new architectural contribution.

The measurement is the cosine similarity, computed on the shared encoder parameters at each training step, between the gradient coming from the activity-classification loss and the gradient coming from the domain-adversarial loss after it passes through the gradient-reversal layer. A cosine near zero would indicate the two objectives are roughly independent on the shared parameters; a positive cosine would indicate they cooperate; a negative cosine indicates they are, at that step, pulling the shared encoder in opposing directions. Across the leave-one-dataset-out folds, the mean cosine is negative, at roughly -0.20 , and the two gradients point in opposing directions on the majority of training steps, in the neighborhood of five out of every six. This conflict is not concentrated in one part of the encoder: the accelerometer branch, the gyroscope branch, the graph convolution layers, and the remaining encoder components all show comparably negative cosines, and the conflict grows more consistently negative as the Ganin schedule of Section 3.2.5 ramps the adversarial weight up over the course of training, consistent with the two objectives genuinely competing for the same shared representation once adversarial pressure becomes substantial, rather than domain invariance behaving as a free auxiliary signal early in training and only becoming costly by coincidence.

This measurement gives a mechanistic account of a pattern this chapter has run into repeatedly. Domain-adversarial training helped only one backbone in Section 4.3.4, softening or strengthening the adversarial pressure further in Section 4.3.6 and Section 4.4.1 never reliably improved on the original recipe, and a regularizer designed specifically to push harder toward domain invariance collapsed the model outright in Section 4.4.3. All of these results are consistent with activity classification and domain invariance being genuinely opposed forces on this shared encoder rather than two objectives that happen to have found different optimal weights: asking the model for more invariance is asking it to move its shared representation in a direction that classification’s own gradient is, on average, actively pulling against. This is why the domain-adversarial objective’s contribution throughout this thesis has been real but modest, and why every attempt to extract more from it by increasing its pressure or its reach ran into the same wall.

With this diagnostic in place, the storage-optimization characterization of this section completes the account of what the thesis’s final system costs and what governs that cost. Section 4.8 draws these results together with every earlier arc’s findings into a single comparative discussion.

4.8 Cross-Phase Comparative Discussion

Table 4.19 draws every stage covered in this chapter into a single place, extending the preview given in Table 4.1 with the final verdict reached for each one. Reading down it in order retraces the whole empirical arc: a within-dataset storage study that looked solved, a first cross-dataset exposure that revealed it was not, a long search through representation and architecture, a still longer search through domain adaptation and robustness techniques that mostly failed, an adaptive graph construction that finally closed most of the remaining gap, a final architecture search that confirmed no further gain was available, and a return to storage optimization on a model that was, by that point, actually worth compressing.

Table 4.19: Every stage covered in this chapter, in order, with its final verdict.

Arc	Stage	Accuracy	Verdict
I	Single-dataset storage-optimization baseline (within-dataset)	91.2%	compression free
I	DAGHAR-standardized adoption (within-dataset)	91.4%	rankings not portable
I	LODO reframing: the accuracy cliff	61.6%	motivates Arcs II–VI
II	Frequency features, timestep graph	70.7%	kept; recovers most of the cliff
II	Hybrid features, timestep graph	67.1%	revert; too early for this graph
II	Sensor-as-node graph rewrite	68.5%	architectural pivot; no accuracy gain yet
II	Domain-adversarial training adopted	70.7%	kept for Graph-SAGE only
II	Configuration sweep (capacity scaling)	70.1%	revert; capacity is not the lever
II	Recipe finalized (control, “Arm A”)	73.1%	kept as the control recipe
III	Feature/normalization/alignment variants	59–73%	all revert

Table 4.19 – continued

Arc	Stage	Accuracy	Verdict
III	Convolutional late fusion	76.3%	kept
III	Test-time and training-time adaptation	75–77%	both revert
III	Transductive domain-adaptation family	76–77%	all revert; closes the DA line
IV	Adaptive Pearson-blend graph construction	78.0%	kept; best model; every fold improves
V	Weight-aware spectral backbone	77.4%	revert
V	Pseudo-label self-training	77.9%	revert; closes the DA line again
VI	Architecture capacity and node coarsening	73–78%	one sensor axis free; capacity dominates
VI	Post-hoc compression	up to 78.9%	free to 50% pruning; sharp cliff at 70%
VI	Edge policy at matched budget	49–78%	structure dominates budget
VI	Combined-axis and Pareto frontier	76–78%	stacking does not beat one good lever

Table 4.19 – continued

Arc	Stage	Accuracy	Verdict
VI	Gradient-conflict diagnostic	—	explanatory only; not a proposed fix

Read as a whole, the table’s clearest pattern is which family of interventions worked and which did not. Every domain-adaptation attempt that relied on giving the model some form of privileged access to the target domain—adapting normalization statistics at or before test time, aligning feature covariances, or bootstrapping pseudo-labels from a source-only teacher—failed to clear the bar this thesis set for adoption, across five separate attempts spanning two different arcs and two different strengths of the underlying graph branch. Every one of these methods shares a common shape: each computes a single, global correction, whether a batch-normalization statistic, an alignment loss, or a confidence threshold, and applies it uniformly across all six target datasets at once. That shape is also why they failed in a strikingly consistent way: the correction that helped the target dataset most different from the source pool reliably hurt the targets already close to it, because a single global correction cannot simultaneously serve a target that needs to be pulled toward the source distribution and a target that is already well served by it. No method tried in this chapter escaped that trade-off entirely; the closest, feature covariance alignment at a gentle weight in Section 4.4.4, only narrowed the mean shortfall without eliminating it.

The two interventions that did succeed, convolutional late fusion in Section 4.4.2 and the adaptive graph construction in Section 4.5.4, both sidestep this trade-off rather than resolving it. Fusion never touches the target domain at all; it succeeds by combining two models that encode genuinely different aspects of the same signal, so their errors do not fully overlap, and a simple weighted average recovers signal neither model exposes on its own. The adaptive graph likewise never touches the target domain, but it changes what a single global correction even means: rather than computing one statistic for an entire target dataset, it recomputes its topology fresh for every three-second

window, so the correction is local to each window’s own signal rather than global to the dataset it happens to come from. The one design choice that kept this local flexibility from destabilizing on unfamiliar data was blending it with a topology that never changes at all, established for entirely different reasons in Section 4.3.3. The pattern across this whole chapter, in short, is that global corrections computed from or toward the target domain kept failing, while approaches that either avoided the target domain through model diversity or replaced a single global correction with many small, locally-computed ones kept succeeding.

The same table also makes clear which target datasets were structurally hardest, and the pattern is consistent across nearly every method in this chapter. One target is the hardest almost everywhere it appears: its accuracy sits below every other target’s under the accuracy cliff of Section 4.2.3, it is the target every temporal-pooling and feature-fusion variant in Section 4.4.1 damages most, it is the target whose oracle ceiling under fusion in Section 4.4.2 is lowest of any target, and it is the target where the self-training attempt of Section 4.6.2 fails through simple lack of confident pseudo-label coverage rather than through low precision. Several factors plausibly compound here: its test set is the smallest of the six by a wide margin, which alone inflates variance, but its accuracy is also consistently lower on average, not merely noisier, across every method tried, consistent with a genuinely larger distributional gap from the rest of the source pool, whether from device placement, participant demographics, or recording protocol. A second target shows a different kind of difficulty, not low accuracy so much as structural incompatibility with methods that assume every target shares the benchmark’s full six-class label set: because this target’s activities are a strict subset of the other five datasets’, any method that must commit to a full six-way decision, most visibly the pseudo-label self-training of Section 4.6.2, risks confidently assigning labels for activities the target does not contain at all. The two easiest targets throughout this chapter, by contrast, are consistently the ones closest to the overall source pool in device placement and protocol, and they are also the targets that benefit most from fusion, since a strong ConvNet branch and a strong graph branch agreeing closely, but not perfectly, on an easy target is exactly the condition under which averaging their predictions helps most.

Taken together, these two patterns suggest a broader reading of domain shift in smartphone-based human activity recognition, beyond the specific numbers reported in this chapter. The shift between DAGHAR’s six datasets is not a single, uniform shift that any one alignment direction could correct; it is a heterogeneous mixture of near-source and far-from-source targets, and the distance to the source pool differs enough from target to target that no single global statistic, alignment loss, or confidence threshold can serve every target at once. Techniques built around a single global correction, whatever form that correction takes, are structurally mismatched to a domain-shift problem of this kind. What worked instead, across this entire empirical program, was either avoiding the need for target-specific correction altogether by combining structurally different models, or replacing one global correction with many small, local ones anchored by a stable, non-adaptive structural prior. Both of these lessons, not merely the specific accuracy numbers behind them, are the thesis’s contribution to how cross-dataset generalization in this setting should be pursued.

4.9 Summary

This chapter set out to demonstrate, empirically, the framework proposed in Chapter 3, and to show that its two goals, cross-dataset generalization and a cheap model that keeps that generalization, were pursued honestly rather than assumed. The chapter began with a storage-optimization question that looked answered on a single dataset and fell apart the moment the same model was evaluated across six datasets under a strict leave-one-dataset-out protocol, producing a thirty-point accuracy cliff that redirected the entire project toward domain generalization. A long search through feature representation and graph architecture recovered roughly a third of that gap and fixed the sensor-as-node graph, the GraphSAGE backbone, and the domain-adversarial training objective as the system’s foundation. A longer and less successful search through domain-adaptation and robustness techniques closed with only one durable gain, convolutional late fusion, after four separate families of target-aware adaptation each failed to clear the bar this thesis set for adopting

them. The adaptive Pearson-blend graph construction then delivered this thesis’s central empirical result, closing most of the remaining gap to the DAGHAR ConvNet reference accuracy while using no target-domain data whatsoever, and a further architecture search confirmed that no additional gain was available beyond it. Finally, the storage-optimization question the chapter opened with was answered on this finished system: compression is cheap up to a real limit, one sensor axis is entirely free to drop, and which parts of the model are kept matters far more than how many parts are kept, a finding that echoes the mechanism behind the adaptive graph itself. A closing diagnostic explained, without proposing a fix, why the domain-adversarial objective’s contribution stayed modest throughout: the activity-classification and domain-invariance gradients are, on this architecture, in measurable and persistent tension rather than cooperating.

The system this chapter arrives at, an adaptive sensor graph fused with an independently trained convolutional network, evaluated strictly under a target-blind protocol and matching the accuracy of prior work that was not so constrained, is the empirical evidence for the framework Chapter 3 proposed. Chapter 5 draws out what this body of work implies beyond its own numbers, and what it leaves for future work to pursue.

Chapter 5

Conclusion

5.1 Summary of the Work

This thesis set out from a deployment mismatch: sensor-based human activity recognition works well when a model is trained and tested inside a single dataset, and much worse when the same model meets a dataset it has never seen—a different device, body position, population, and collection protocol. Chapter 1 framed this as two connected problems: cross-dataset generalization is not guaranteed (Problem 1), and generalization is not useful if the model that achieves it is too expensive to deploy (Problem 2). Both were studied on the DAGHAR benchmark’s six standardized smartphone datasets, under a strict leave-one-dataset-out protocol in which the held-out dataset contributes nothing to training or model selection, and every adopted design decision was confirmed across multiple random seeds.

The scale of Problem 1 was established empirically before it was addressed: graph neural networks that exceeded 91% accuracy within a single dataset fell to roughly 62% when evaluated across datasets (Section 4.2.3)—a cliff of nearly thirty percentage points on the same architectures, which redirected the project from its original storage-optimization focus toward generalization first. The response, developed across Chapters 3 and 4, was a progression of representation choices rather than a single mechanism: frequency-domain and then hybrid time-plus-frequency

node features; a rewrite of the graph itself so that nodes represent sensor channels rather than timesteps (Section 4.3.3); moderate domain-adversarial training (Section 4.3.4); late fusion with an independently trained convolutional branch (Section 4.4.2); and, as the capstone, an adaptive graph construction that chooses each window’s edges by blending the window’s own inter-channel correlations with a fixed physical prior (Section 4.5). The final fused system reaches $77.96\% \pm 0.21\%$ mean leave-one-dataset-out accuracy over five seeds, improving on its fixed-topology counterpart on every individual fold and matching the benchmark’s strongest published convolutional baseline. Alongside this constructive line, an equally deliberate negative line—capacity sweeps, alignment and normalization variants, test-time adaptation, and transductive domain-adaptation methods given access to unlabeled target data—failed a pre-declared adoption bar under the same protocol, establishing that the bottleneck is structural rather than a matter of model size or target access.

With the model locked, Problem 2 was addressed by compressing it deliberately along five storage axes—capacity, precision, edges, nodes, and window length—at matched budgets (Section 4.7). The study found that the cost of storage is highly uneven: removing a redundant sensor channel is free, a backbone with 91% fewer parameters gives up only about one percentage point for a $5.9\times$ size reduction, and, most consequentially, at an identical edge budget the *choice* of edges is worth over eight percentage points—graph structure, not graph budget, governs generalization. A training-dynamics diagnostic (Section 4.7.4) closed the loop by explaining why invariance training helps only modestly: the activity and invariance objectives pull the shared encoder in opposing directions on most training steps. Together, the two halves answer the questions posed in Chapter 1: a graph-based model can match convolutional state-of-the-art generalization when its graph is built well, and that generalization can be made substantially cheaper to store—provided the structure that produces it is preserved.

5.2 Research Contributions

Chapter 1 summarized the thesis’s five contributions; with the evidence of Chapter 4 now in hand, each can be stated with its justification.

1. An adaptive, prior-anchored graph construction for cross-dataset HAR. The central methodological contribution (Section 4.5) is the per-window Pearson-blend edge rule: score every candidate sensor pair by an equal blend of the window’s absolute inter-channel correlation and a fixed physical prior, and keep the top nine pairs as edges. Under five-seed confirmation (Section 4.5.2), the fused system improves from $76.68\% \pm 1.36\%$ (fixed topology) to $77.96\% \pm 0.21\%$ —a +1.28 percentage-point gain realized on every one of the six held-out datasets, with a sixfold reduction in seed variance; the graph branch alone gains +2.36 points. This matches the benchmark’s strongest published convolutional baseline. The claim is sharpened by its own ablation (Section 4.5.3): the purely data-driven variant of the same rule ($\alpha = 1$) posts a competitive mean but regresses on a held-out fold, failing the every-fold standard—the physical prior is an essential ingredient of the method, not a fallback.

2. A generalization-aware efficiency metric and the first compression–generalization characterization for graph-based cross-dataset HAR. The thesis proposes *generalization-per-byte*—mean cross-dataset accuracy per unit of stored model—and applies it in a systematic characterization of the locked model along five storage axes at matched budgets (Sections 4.7.1–4.7.3). The characterization shows the cost of storage to be sharply uneven: dropping the vertical gyroscope axis is free (+0.02 points, a statistical tie); the tiny backbone trades 0.99 points for a $5.9\times$ size reduction and dominates all stacked precision-and-window combinations; and the Pareto frontier of the whole study reduces to exactly these two operating points. The practical deployment lever, on this evidence, is architectural capacity—not the accumulation of many small post-hoc reductions.

3. The finding that graph structure, not graph budget, governs generalization. At an identical nine-edge budget, with identical trained weights, the adopted edge policy reaches 78.07%

while random edge selection reaches 69.82%—an 8.25-point gap, consistent across every held-out dataset—and a deliberately adversarial policy (cross-modal edges only) collapses to 48.91% (Section 4.7.2). Meanwhile the budget itself is comparatively cheap for a well-chosen topology. *Which* connections a sensor graph keeps matters far more than *how many*: the graph’s structure is a first-class design variable for generalization, not merely a storage knob. This finding is what unifies the thesis’s two halves—the same object that delivers the accuracy contribution is the one whose preservation the storage study shows to be non-negotiable.

4. A training-dynamics explanation for the limits of invariance training. Domain-adversarial training was adopted for its modest, reliable gain, yet every stronger invariance mechanism evaluated on top of it regressed performance (Sections 4.4.1 and 4.4.3). The gradient-conflict diagnostic (Section 4.7.4) explains the pattern rather than merely reporting it: on the shared encoder parameters, the activity and domain-invariance gradients oppose each other on roughly five of every six training steps, with a mean cosine similarity near -0.20 , uniformly across the encoder’s components. Invariance pressure in this setting is purchased at the direct expense of the classification objective, which accounts for both the modest size of the gain at moderate strength and the regressions at higher strength.

5. Systematic negative results as evidence. Across the project, the plausible alternatives were not skipped but evaluated and rejected under one consistent protocol and one pre-declared adoption bar: additional capacity (Section 4.3.5), feature, normalization, and alignment variants (Section 4.4.1), test-time and training-time adaptation (Section 4.4.3), the transductive domain-adaptation family—which was allowed to see unlabeled target data and still failed the bar (Section 4.4.4)—and two final architecture candidates (Section 4.6). Reported honestly, these results carry evidentiary weight of their own (Section 4.8): they establish that the cross-dataset bottleneck is structural, and that neither parameters nor target access is what closes the gap—the graph is.

5.3 Limitations

No empirical thesis is complete without an honest account of where its evidence is thinner than its claims deserve. Six limitations are stated here; none, in our judgment, undermines a headline claim, but each bounds how far a claim should be generalized.

The fusion weight is not target-blind. As flagged where the rule is defined (Chapter 3), the late-fusion blend weight is the one parameter in this thesis tuned directly against the aggregate leave-one-dataset-out result rather than on source-validation data. The exposure is small—a single scalar selected from a coarse grid, applied to two frozen models—but it is a real departure from the otherwise strict selection discipline, and results involving fusion inherit it. The graph-branch-only results, which obey the strict protocol throughout, are unaffected.

The convolutional branch is frozen, and its reimplementations trails its published reference. The two branches are never trained jointly; the convolutional branch is trained once and reused frozen, so all fused results are a conservative lower bound on a jointly trained system—the oracle analysis in Section 4.4.2, at roughly 84%, shows how much complementary signal simple averaging leaves unexploited. Separately, the DeepConvLSTM reimplementations itself reaches 72.7–74.8% alone under this thesis’s pipeline, short of the roughly 77% that DAGHAR reports for its own convolutional reference—a gap attributable to implementation and training details rather than architecture. The thesis’s matching-SOTA claim rests on the fused system’s 77.96% against the published reference, and does not depend on the reimplementations reproducing that reference exactly; but a stronger convolutional branch could plausibly raise the fused ceiling, which cuts in the thesis’s favour and is noted as future work.

Parts of the storage study rest on fewer than five seeds. The headline storage arms (the full model, the dropped-gyroscope variant, the tiny backbone, and the edge-policy comparison) are five-seed confirmed, but the combined-axis stacking results and some intermediate capacity arms rest on a single seed, and a small number of rejected candidates elsewhere were closed out at three

seeds. Wherever a result rests on fewer seeds, Chapter 4 says so explicitly; conclusions drawn from single-seed points (in particular, the fine detail of how axes compose) should be read as indicative rather than confirmed.

The edge-policy comparison swaps policies only at inference. The structure-versus-budget result compares edge-selection policies on a model whose weights were trained under the adopted adaptive policy; the alternative policies are substituted at inference, which is precisely what isolates edge choice as the only variable, but also means the baselines are not shown at their own best—a model trained end-to-end under random or fixed edge selection might adapt its weights to its topology and recover part of the gap. The size of the observed gap (over eight points, on every fold) makes it unlikely that retraining would erase the qualitative finding, but the quantitative margin should be read with this asymmetry in mind.

Post-hoc compression is evaluated without fine-tuning. Quantization and pruning are applied to trained weights with no recovery training, so their reported costs are upper bounds on what compression-aware training could achieve. This was a deliberate choice—it keeps the comparison honest, since every axis is measured without extra training—but it means the thesis’s precision-axis numbers should not be quoted as the best achievable for this architecture.

Six domains, one benchmark. Every result in this thesis is derived from the six DAGHAR datasets; no seventh, never-touched dataset exists to test the final locked system on. Although the target dataset of each fold is never used for training or selection, the six folds were revisited throughout the project’s development, so at the level of the research programme—as opposed to any single experiment—design decisions have adapted to this benchmark. DAGHAR’s own irregularities, noted in Chapter 3, sharpen the point: two domains lack activity classes (UCI-HAR has no *run*; WISDM has no stairs), and the hardest domain, KuHar, contributes a test split of only 144 windows, making its per-fold numbers the most volatile in the study. Claims of the form “this method generalizes across HAR datasets” should therefore be read as established on, and bounded

by, this benchmark.

5.4 Future Work

Each direction below extends a specific finding or addresses a specific limitation above; none is speculative beyond the evidence.

Weight-aware message passing, attempted properly. The backbone ignores edge weights, which makes the adaptive graph a pure topology selector—and the one weight-aware architecture evaluated (Section 4.6.1) regressed and was reverted. But that negative result convicts one architecture, not the idea: a targeted attempt would keep the adopted construction and inject the blend scores s_{ij} as edge features in an architecture designed for them, with the prior anchoring the weights the way it already anchors the topology. The gradient-conflict diagnostic suggests any such attempt should be regularized gently: mechanisms that reshape the representation aggressively have consistently regressed in this setting.

Joint training and smarter fusion. The oracle gap of Section 4.4.2 (roughly 84% against 77.96% achieved) is the single largest known reserve of unexploited accuracy in the system. Joint or alternating training of the two branches, a learned per-window gating in place of one global blend weight, or distilling both branches into one model are all natural attempts—and a source-validation-tunable gate would additionally remove the fusion-weight limitation above.

Compression-aware training. The post-hoc numbers invite their counterpart: quantization-aware training and pruning with recovery fine-tuning, evaluated under the same matched-budget, multi-seed protocol. The interesting question is not whether accuracy improves—it should—but whether the *shape* of the trade-off changes: whether fine-tuning rescues the axes this thesis found expensive, or merely cheapens further the ones already cheap.

A true held-out evaluation. The most direct answer to the six-domains limitation is a seventh dataset, standardized through DAGHAR’s own published pipeline and touched exactly once, after every design decision is frozen. Position and modality extensions (wrist-worn devices, additional sensor channels) would test the physical prior’s portability, since a different sensor layout requires re-deriving the prior—a cheap, well-defined exercise that would reveal how much of the method’s value is the specific prior versus the prior-anchoring principle.

Learning the graph constructor. The adopted construction is hand-designed: a fixed blend rule with two hyperparameters. Its success, and the finding that structure governs generalization, together motivate the natural next step—learning the constructor itself, subject to an explicit edge budget, trained end-to-end with the backbone. The design space is subtle, however: a constructor trained only against the task loss has every incentive to rediscover the fixed rule it replaces, and this thesis’s own evidence—that structure transfers where optimization does not—suggests the training signal is the crux. A learned constructor rewarded directly for the cross-domain *stability* of the topology it selects, rather than for source accuracy alone, is the version of this idea most worth attempting.

5.5 Concluding Remarks

This thesis began with a storage question and was forced, by its own first cross-dataset experiment, to answer a generalization question first. That detour turned out to be the work: what stands at the end is not a collection of techniques but a single object—a small graph over six sensor channels, its edges chosen window by window from the data but anchored in how the sensors physically relate—examined from two directions. Examined for accuracy, that graph is what closed the gap to the benchmark’s state of the art, where added capacity, heavier invariance, and even access to the target’s own data had failed to. Examined for cost, it is what compression cannot be allowed to touch: the storage study’s central result is that nearly everything else about the model can be given up cheaply, but the structure cannot.

The broader lesson this work offers to graph-based recognition on wearable sensors is correspondingly simple. In the cross-dataset setting, *how a sensor graph is built matters more than how it is processed, how large the model is, or what the training procedure is allowed to see*—and building it well requires both the data and the physics, in balance. What a model of this kind needs to generalize turns out to be modest in every sense: a handful of well-chosen connections, kept small, kept structural, and kept honest.

References

- [1] Otávio Napoli, Dami Duarte, Patrick Alves, Darlinne Hubert Palo Soto, Henrique Evangelista de Oliveira, Anderson Rocha, Levy Boccato, and Edson Borin. A benchmark for domain adaptation and generalization in smartphone-based human activity recognition. *Scientific Data*, 11(1):1192, 2024.
- [2] Francisco Javier Ordóñez and Daniel Roggen. Deep convolutional and lstm recurrent neural networks for multimodal wearable activity recognition. *Sensors*, 16(1):115, 2016.
- [3] Oscar D Lara and Miguel A Labrador. A survey on human activity recognition using wearable sensors. *IEEE communications surveys & tutorials*, 15(3):1192–1209, 2012.
- [4] Jindong Wang, Yiqiang Chen, Shuji Hao, Xiaohui Peng, and Lisha Hu. Deep learning for sensor-based activity recognition: A survey. *Pattern recognition letters*, 119:3–11, 2019.
- [5] Davide Anguita, Alessandro Ghio, Luca Oneto, Xavier Parra, and Jorge L Reyes-Ortiz. Human activity recognition on smartphones using a multiclass hardware-friendly support vector machine. In *International workshop on ambient assisted living*, pages 216–223. Springer, 2012.
- [6] Nisha Dangol and Sazia Mahfuz. Hybrid cnn-lstm-gru with attention for human activity recognition. *Procedia Computer Science*, 265:99–106, 2025.
- [7] Iveta Dirgová Luptáková, Martin Kubovčík, and Jiří Pospíchal. Wearable sensor-based human activity recognition with transformer model. *Sensors*, 22(5):1911, 2022.

- [8] Nisha Dangol and Sazia Mahfuz. A review on sensor-based har models using gnn: Ai in healthcare. *Procedia Computer Science*, 272:368–375, 2025.
- [9] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural message passing for quantum chemistry. In *International conference on machine learning*, pages 1263–1272. Pmlr, 2017.
- [10] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.
- [11] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. Graph attention networks. *arXiv preprint arXiv:1710.10903*, 2017.
- [12] Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30, 2017.
- [13] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? *arXiv preprint arXiv:1810.00826*, 2018.
- [14] Jing Yang, Tianzheng Liao, Jingjing Zhao, Yan Yan, Yichun Huang, Zhijia Zhao, Jing Xiong, and Changhong Liu. Domain adaptation for sensor-based human activity recognition with a graph convolutional network. *Mathematics*, 12(4):556, 2024.
- [15] Shai Ben-David, John Blitzer, Koby Crammer, Alex Kulesza, Fernando Pereira, and Jennifer Wortman Vaughan. A theory of learning from different domains. *Machine Learning*, 79(1):151–175, 2010.
- [16] Sinno Jialin Pan and Qiang Yang. A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10):1345–1359, 2010.
- [17] Kaiyang Zhou, Ziwei Liu, Yu Qiao, Tao Xiang, and Chen Change Loy. Domain generalization: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(4): 4396–4415, 2023.

- [18] Jindong Wang, Cuiling Lan, Chang Liu, Yidong Ouyang, Tao Qin, Wang Lu, Yiqiang Chen, Wenjun Zeng, and Philip S Yu. Generalizing to unseen domains: A survey on domain generalization. *IEEE Transactions on Knowledge and Data Engineering*, 35(8):8052–8072, 2023.
- [19] Ishaan Gulrajani and David Lopez-Paz. In search of lost domain generalization. In *International Conference on Learning Representations*, 2021.
- [20] Yaroslav Ganin, Evgeniya Ustinova, Hana Ajakan, Pascal Germain, Hugo Larochelle, François Laviolette, Mario March, and Victor Lempitsky. Domain-adversarial training of neural networks. *Journal of machine learning research*, 17(59):1–35, 2016.
- [21] Arthur Gretton, Karsten M Borgwardt, Malte J Rasch, Bernhard Schölkopf, and Alexander Smola. A kernel two-sample test. *Journal of Machine Learning Research*, 13:723–773, 2012.
- [22] Mingsheng Long, Yue Cao, Jianmin Wang, and Michael Jordan. Learning transferable features with deep adaptation networks. In *International Conference on Machine Learning*, pages 97–105. PMLR, 2015.
- [23] Baochen Sun and Kate Saenko. Deep CORAL: Correlation alignment for deep domain adaptation. In *European Conference on Computer Vision Workshops*, pages 443–450. Springer, 2016.
- [24] Woong-Gi Chang, Tackgeun You, Seonguk Seo, Suha Kwak, and Bohyung Han. Domain-specific batch normalization for unsupervised domain adaptation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7354–7362, 2019.
- [25] Ximei Wang, Ying Jin, Mingsheng Long, Jianmin Wang, and Michael I Jordan. Transferable normalization: Towards improving transferability of deep neural networks. *Advances in Neural Information Processing Systems*, 32, 2019.
- [26] Yanghao Li, Naiyan Wang, Jianping Shi, Xiaodi Hou, and Jiaying Liu. Adaptive batch normalization for practical domain adaptation. *Pattern Recognition*, 80:109–117, 2018.

- [27] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations*, 2021.
- [28] Dong-Hyun Lee. Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks. In *ICML Workshop on Challenges in Representation Learning*, volume 3, 2013.
- [29] Kaiyang Zhou, Yongxin Yang, Yu Qiao, and Tao Xiang. Domain generalization with MixStyle. In *International Conference on Learning Representations*, 2021.
- [30] Junbum Cha, Sanghyuk Chun, Kyungjae Lee, Han-Cheol Cho, Seunghyun Park, Yunsung Lee, and Sungrae Park. SWAD: Domain generalization by seeking flat minima. *Advances in Neural Information Processing Systems*, 34:22405–22418, 2021.
- [31] Song Han, Jeff Pool, John Tran, and William Dally. Learning both weights and connections for efficient neural network. *Advances in Neural Information Processing Systems*, 28, 2015.
- [32] Song Han, Huizi Mao, and William J Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. In *International Conference on Learning Representations*, 2016.
- [33] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *International Conference on Learning Representations*, 2019.
- [34] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2704–2713, 2018.
- [35] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

- [36] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.
- [37] Mingxing Tan and Quoc Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *International Conference on Machine Learning*, pages 6105–6114. PMLR, 2019.
- [38] Sara Hooker, Aaron Courville, Gregory Clark, Yann Dauphin, and Andrea Frome. What do compressed deep neural networks forget? *arXiv preprint arXiv:1911.05248*, 2019.
- [39] James Diffenderfer, Brian Bartoldson, Shreya Chaganti, Jize Zhang, and Bhavya Kailkhura. A winning hand: Compressing deep networks can improve out-of-distribution robustness. *Advances in Neural Information Processing Systems*, 34:664–676, 2021.
- [40] Cheng Zheng, Bo Zong, Wei Cheng, Dongjin Song, Jingchao Ni, Wenchao Yu, Haifeng Chen, and Wei Wang. Robust graph representation learning via neural sparsification. In *International Conference on Machine Learning*, pages 11458–11468. PMLR, 2020.
- [41] Tianlong Chen, Yongduo Sui, Xuxi Chen, Aston Zhang, and Zhangyang Wang. A unified lottery ticket hypothesis for graph neural networks. In *International Conference on Machine Learning*, pages 1695–1706. PMLR, 2021.
- [42] Shyam A Tailor, Javier Fernandez-Marques, and Nicholas D Lane. Degree-quant: Quantization-aware training for graph neural networks. In *International Conference on Learning Representations*, 2021.
- [43] Yexu Zhou, Haibin Zhao, Yiran Huang, Michael Hefenbrock, Till Riedel, and Michael Beigl. TinyHAR: A lightweight deep learning model designed for human activity recognition. In *ACM International Symposium on Wearable Computers*, pages 89–93, 2022.

- [44] Xiaozhou Ye and Kevin I-Kai Wang. Graph-based adversarial domain generalization with anatomical correlation knowledge for cross-user human activity recognition. *arXiv preprint arXiv:2506.01962*, 2025.
- [45] Xiaozhou Ye and Kevin I-Kai Wang. Domain-adversarial anatomical graph networks for cross-user human activity recognition. *arXiv preprint arXiv:2505.06301*, 2025.
- [46] Wang Lu, Yao Zhu, and Jindong Wang. HAROOD: A benchmark for out-of-distribution generalization in sensor-based human activity recognition. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2026.